



Politecnico di Torino

Microelectronic Systems

DLX Microprocessor: Design & Development

Final Project Report

Master degree in Computer Engineering

Referents: Prof. Mariagrazia Graziano, Giovanna Turvani

Authors: Group 47

Gabriele Arcidiacono, Stefano Galati, Emanuele Aquilia

November 20, 2025

Contents

1	Introduction	1
1.1	Design Features	2
2	Processor Overview	4
2.1	Processor Architecture	4
2.1.1	Top-Level View	4
2.1.2	Implementation	5
2.2	The Pipeline	6
2.3	Structure of source files	9
3	Datapath	11
3.1	Overall Structure	11
3.2	ALU	11
3.2.1	Overview	11
3.2.2	Logicals	12
3.2.3	Shifter	16
3.2.4	Adder/Subtractor	19
3.2.5	Equal 0 checker	24
3.2.6	Comparator	25
3.2.7	Output Multiplexer	28
3.2.8	Complete overview of ALU operations	33
3.3	Registers	34
3.3.1	Overview	34
3.3.2	Implementation	35
3.3.3	VHDL implementation	35
3.3.4	Integration in the Datapath	35
3.4	Register File	35
3.4.1	Overview	35
3.4.2	Implementation	35
3.4.3	VHDL implementation	37
3.5	Sign and Zero Extenders	37
3.5.1	Overview	37
3.5.2	VHDL Implementation	38
4	Control Unit	39
4.1	Overview	39
4.2	Implementation	39
4.2.1	Ports description	40

4.2.2	Regular control signals	41
4.2.3	ALU control signals	42
4.2.4	Delaying the control signals	42
4.2.5	Flush and Stall handling	43
4.2.6	VHDL Implementation	44
5	Memories	45
5.1	Data Memory	45
5.1.1	Overview	45
5.1.2	Implementation	45
5.1.3	VHDL Implementation	47
5.2	Instruction Memory	47
5.2.1	Overview	47
5.2.2	Implementation	47
5.2.3	VHDL Implementation	48
5.3	Data Cache	48
5.3.1	Overview	48
5.3.2	Implementation	49
5.3.3	VHDL Implementation	52
6	Pipeline Hazards	53
6.1	Introduction	53
6.2	Data Hazards	53
6.2.1	Read-After-Write (RAW) Hazards	53
6.2.2	Handling RAW Hazards	54
6.3	Control Hazards	65
6.3.1	Handling Control Hazards	65
7	Simulation and Verification	73
7.1	Overview	73
7.2	Unit tests	73
7.3	Integration tests	73
7.4	Testing using assembly programs	74
7.4.1	Running a simulation	75
7.4.2	ALU operations test	75
7.4.3	Jump instructions test	76
7.4.4	Branch instructions test	77
7.4.5	Data hazards test	77
7.4.6	Cache functionality test	77
7.4.7	Load and store instructions variations test	78
7.4.8	Stack operations test	78
7.4.9	Branch prediction test	78
7.4.10	Complex programs test	79
8	Synthesis and Physical Design	80
8.1	Synthesis	80
8.1.1	Overview	80
8.1.2	Full script listing	80
8.1.3	Detailed Explanation	81
8.2	Generated files and directories	83

8.2.1	Running the Synthesis	83
8.3	Summary	84
8.4	Synthesis results	84
8.5	Physical Design of the DLX Processor	86
8.5.1	Design Setup and Configuration	86
8.5.2	Floorplanning	87
8.5.3	Power Planning	87
8.5.4	Standard-Cell Placement	87
8.5.5	Clock Tree Synthesis	87
8.5.6	Routing	88
8.5.7	Parasitic Extraction and Analysis	88
8.5.8	Timing and Power Reports	88
8.5.9	Final comments	88
9	Conclusions	90
9.1	Lesson Learned	90
9.2	Future Work	91
9.3	Acknowledgements	91
A	Data Hazard Instances	92
A.1	All cases of Data Forwarding	92
A.2	All cases of Stalling	97
B	Instructions encoding	101

CHAPTER 1

Introduction

In this chapter, we introduce the DLX microprocessor project, outlining its objectives, scope, and significance. Moreover, we provide an overview of the main characteristics of the DLX architecture and highlight the key design features and goals of our implementation. This project consists of the design and implementation of a **DLX** microprocessor at the RT level using **VHDL**. The DLX architecture is a RISC (Reduced Instruction Set Computer) architecture that was designed to be simple and efficient. It is widely used in academic settings for teaching computer architecture concepts due to its simplicity and clarity and it closely resembles the RISC-V architecture. The main features of the DLX architecture include:

- **Load/store** architecture: all arithmetic and logic operations are performed on registers, and only load and store instructions access memory.
- **Fixed instruction length** of 32 bits
- 32 general-purpose registers
- **Pipelined** execution: the DLX architecture supports a five-stage pipeline, which allows for overlapping instruction execution and improved performance.
- **Three instruction formats** (Figure 1.1):
 - **R-type (Register)**: Register-register ALU operations
 - **I-type (Immediate)**: instructions that include an immediate value as one of the operands (ALU immediate operations, loads, stores, branches)
 - **J-type (Jump)**: instructions that involve jumps to a target address.

The architecture supports a variety of instruction types, including arithmetic, logical, memory access, and control flow instructions. The motivation behind this project is to gain hands-on experience in designing and implementing a microprocessor using VHDL.

By working on this project, we aim to deepen our understanding of computer architecture concepts and develop practical skills in digital design and hardware description languages.

The goals of this project include designing a functional DLX microprocessor that can execute a subset of DLX instructions, implementing pipelining to improve performance, and validating the design through simulation and synthesis. We also aim to document the design process and provide a comprehensive report on our findings and experiences throughout the project.

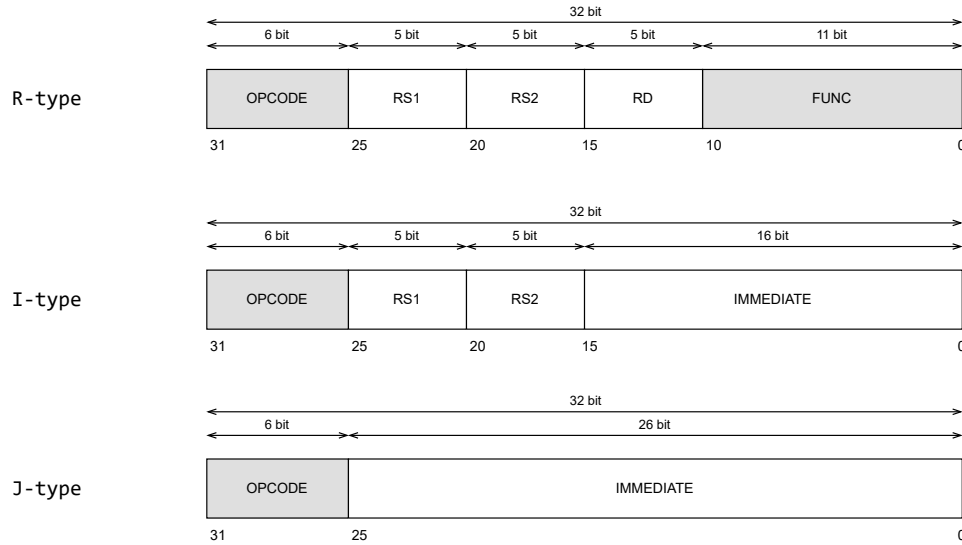


Figure 1.1: DLX Instruction Formats

1.1 Design Features

In this section, we will outline the key design features of our DLX microprocessor implementation. The **basic** design features include:

- **Five-stage pipeline:** The microprocessor is designed with a five-stage pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. This allows for overlapping instruction execution and improved performance
- **Datapath:** all datapath components needed to support the operations are implemented at RT level with VHDL
- **Control Unit:** to control the datapath components, a Hardwired Control Unit is implemented, generating the necessary control signals based on the instruction OPCODE and FUNC code
- **Basic instruction set:**
 - **Addition:** add, addi
 - **Subtraction:** sub, subi
 - **Logic:** and, andi, or, ori, xor, xori
 - **Shift:** sll, slli, srl, srli
 - **Comparison (signed):** sle, slei, sge, sgei
 - **Equality:** sne, snei
 - **Branch:** beqz, bnez
 - **Jump:** j, jal
 - **Memory (load):** lw
 - **Memory (store):** sw

The **advanced** design features include:

- **Extended instruction set** with the following additional instructions:
 - **Addition:** addu, addui
 - **Subtraction:** subu, subui
 - **Shift:** sra, srai
 - **Comparison (signed):** slt, slti, sgt, sgti
 - **Comparison (unsigned):** sltu, sltui, sleu, sleui, sgtu, sgtui, sgeu, sgeui
 - **Equality:** seq, seqi
 - **Jump:** jr, jalr
 - **Memory (load):** lb, lbu, lh, lhu, lhi
 - **Memory (store):** sb, sh
- **Forwarding:** forwarding is implemented to handle data hazards and minimize pipeline stalls.
- **Branch Prediction:** A branch prediction mechanism is implemented to improve the flow of instruction execution and minimize pipeline stalls caused by branch instructions.
- **Cache Memory:** The microprocessor includes a cache memory subsystem to reduce memory access latency and improve overall performance.

The final design will be validated through extensive **simulation** with testbenches and assembly programs that test various aspects of the microprocessor's functionality and performance. The design will also be **synthesized** and optimized. Finally, the synthesized design will be **placed** and **routed**, evaluating post physical design performance.

CHAPTER 2

Processor Overview

In this chapter, we will provide an overview of our DLX microprocessor architecture, including its key components and their interactions. We will discuss the organization of the processor, including the datapath and control unit, and explain how these components work together to execute instructions. Finally, we will describe the pipeline mechanism at a high level. By understanding the architecture of the DLX microprocessor, we can better appreciate the design choices made during the implementation phase and the challenges encountered along the way.

2.1 Processor Architecture

2.1.1 Top-Level View

Figure 2.1 shows the schematic symbol of our DLX microprocessor architecture. We may notice various ports to interface with the external memories (Instruction Memory and Data Memory) as well as the clock and reset signals. Indeed, the processor is designed to work in conjunction with external memory modules, which are not part of the processor itself but are essential for its operation.

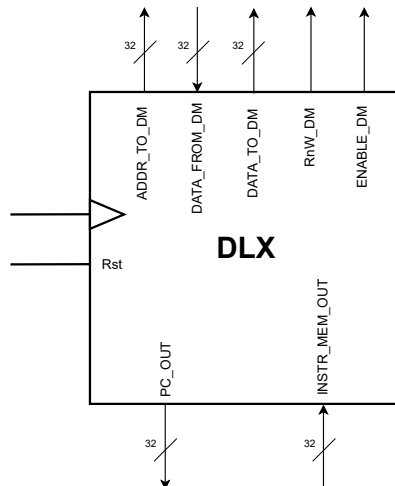


Figure 2.1: Schematic symbol of the DLX processor

Input ports:

- **Clk**: processor Clock (1-bit)
- **Rst**: Reset (1-bit)
- **INSTR_MEM_OUT**: Instruction from Instruction Memory (32-bit)
- **DATA_FROM_DM**: Data from Data Memory (32-bit)

Output ports:

- **PC_OUT**: Address to Instruction Memory (32-bit)
- **ADDR_TO_DM**: Address to Data Memory (32-bit)
- **DATA_TO_DM**: Data to Data Memory (32-bit)
- **ENABLE_DM**: Data Memory Enable (1-bit)
- **RnW_DM**: Data Memory Read/Write signal (1-bit)

2.1.2 Implementation

By looking into our processor, at the highest layer, we can see that it is entirely made by the connection of two components:

- **Datapath (DP)**
- **Control Unit (CU)**

This is shown in Figure 2.2.

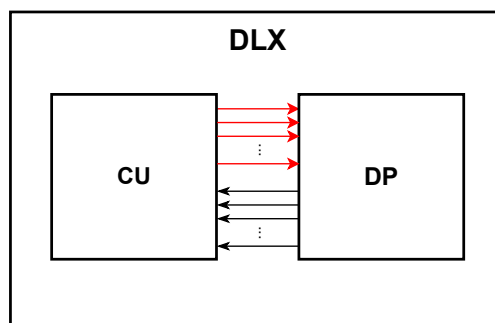


Figure 2.2: first layer view of the DLX processor: Datapath and Control Unit

The **Datapath** is the executive unit of the processor: it is the one containing all the resources to perform the requested operations needed to execute an instruction (arithmetic, logic, load, store, instruction fetch,...).

However, the Datapath alone wouldn't be able to execute instructions as it is unable to self-control its operations. This is the role of the **Control Unit (CU)**: it generates all the control signals needed to properly drive the datapath components, in order to execute the instructions currently being processed.

2.2 The Pipeline

In the DLX architecture, each instruction has to go through a series of 5 stages to be executed, just as it was an industrial product being assembled on an assembly line. These stages are the following:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory (MEM)
5. Write Back (WB)

Every instruction will go through these 5 stages in this exact order to be executed, no exceptions. Each stage lasts one clock cycle, so at each new clock tick, an instruction will move to the next stage.

In the most simple (non-pipelined) model, instruction $i + 1$ coming after instruction i would have to wait for instruction i to complete all its 5 stages before starting its execution. This would mean a **latency** of 5 clock cycles per instruction and a **throughput** of $1/5$ instructions per clock cycle.

To improve performance, the DLX architecture implements a **pipeline** mechanism. This means that while instruction i is in stage EX, instruction $i + 1$ can already be in stage ID, and instruction $i + 2$ in stage IF. A new instruction is fetched at every clock cycle and after the first one reaches the WB stage, a new instruction is also completed at every clock cycle (if no pipeline stalls occur). This way the **latency** remains 5 clock cycles per instruction, but the **throughput** becomes $5/5 = 1$ instruction per clock cycle. Figure 2.3 shows a simplified representation of the pipelined datapath. Notice that the **Instruction Memory** and **Data Memory** are represented in the figure but they are not part of the Datapath itself, as they are external memories connected to the processor. The big grey rectangles

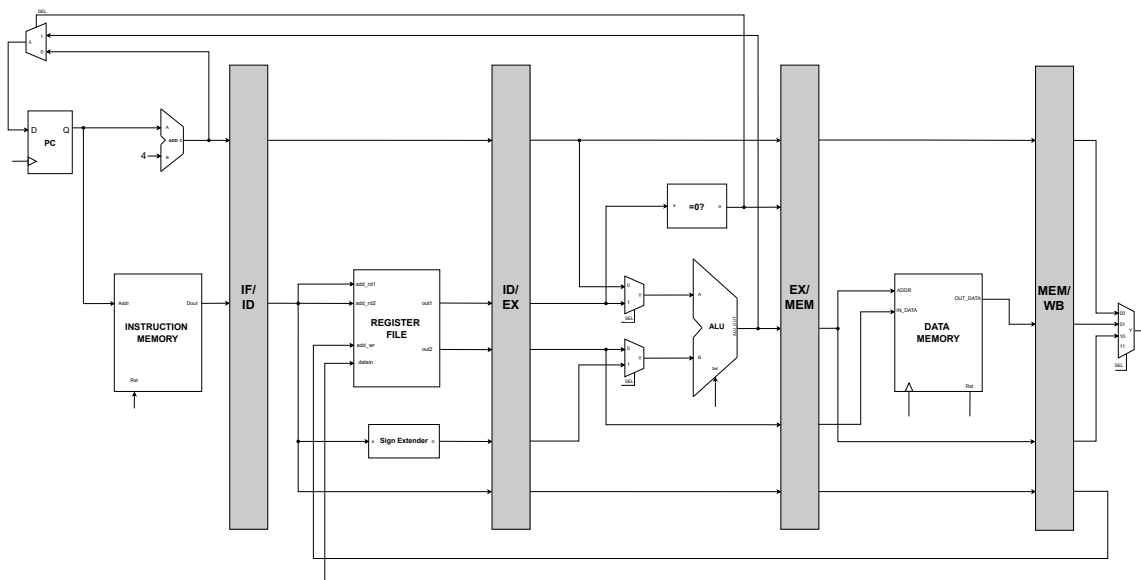


Figure 2.3: Simplified Pipelined Datapath

are the **pipeline registers**, which clearly separate the different stages of the pipeline. They are used to store the intermediate results between each stage of the pipeline. This allows each stage to

operate independently and concurrently, enabling multiple instructions to be processed simultaneously in different stages, hence the pipeline mechanism. Every instruction entering the pipeline will undergo the following journey:

1. *Instruction Fetch* (IF):

In the Instruction Fetch (IF) stage, the instruction is fetched from the **Instruction Memory** (IM) using the **Program Counter** (PC) as address, and it is loaded into the **Instruction Register** (IR).

$$\text{IF/ID.IR} \leftarrow \text{IM}[\text{PC}]$$

The PC is incremented by 4 and propagated to the next pipeline stage. The Next Program Counter (NPC), which will be used in the next clock cycle to fetch the next instruction, is computed. In normal operation, the NPC is simply $\text{PC} + 4$, but in case a taken branch instruction is in EX stage, the NPC will be updated with the branch target address computed in EX.

$$\begin{aligned} \text{PC_INCR} &\leftarrow \text{PC} + 4 \\ \text{IF/ID.PC_INCR} &\leftarrow \text{PC_INCR} \\ \text{NPC} &\leftarrow \text{PC_INCR or ALU_OUT} \\ \text{PC} &\leftarrow \text{NPC} \end{aligned}$$

2. *Instruction Decode* (ID):

In the Instruction Decode (ID) stage, the instruction in IF/ID.IR is decoded by the **Control Unit** (not shown in Figure 2.3) to determine the operation to be performed and the operands involved. In the same stage, the necessary registers are read from the register file and signed/unsigned extension of the immediate is performed.

$$\begin{aligned} \text{ID/EX.A} &\leftarrow \text{RF}[\text{IF/ID.IR}[\text{RS1}]] \\ \text{ID/EX.B} &\leftarrow \text{RF}[\text{IF/ID.IR}[\text{RS2}]] \\ \text{ID/EX.Imm} &\leftarrow \text{extend}(\text{IF/ID.IR}[\text{immediate field}]) \end{aligned}$$

Moreover, the program counter incremented and the instruction register are propagated to the next pipeline stage as they will be needed later on.

$$\begin{aligned} \text{ID/EX.PC_INCR} &\leftarrow \text{IF/ID.PC_INCR} \\ \text{ID/EX.IR} &\leftarrow \text{IF/ID.IR} \end{aligned}$$

3. *Execute* (EX)

In the Execute (EX) stage, the **ALU** (Arithmetic Logic Unit) performs the required arithmetic or logic operations based on the decoded instruction. For load/store instructions, the effective memory address is calculated by adding the base register value to the sign-extended immediate offset.

$$\begin{aligned} \text{ALU instruction: } \text{ALU_OUT} &\leftarrow \text{ALU}(\text{ID/EX.A}, \text{ID/EX.B or ID/EX.Imm}) \\ \text{Load/Store instruction: } \text{ALU_OUT} &\leftarrow \text{ID/EX.A} + \text{ID/EX.Imm} \end{aligned}$$

The ALU result is then propagated to the next pipeline stage. In case of store operations, the data to be stored is also propagated.

$EX/MEM.ALU_OUT \leftarrow ALU_OUT$

Store instruction: $EX/MEM.B \leftarrow ID/EX.B$

Moreover, branch evaluation is performed for branch instructions, to determine whether the branch is taken or not, and update the PC accordingly.

Branch instruction: if condition($ID/EX.A$) then $NPC \leftarrow ALU_OUT$

Finally, the program counter incremented and the instruction register values are propagated to the next pipeline stage.

$EX/MEM.PC_INCR \leftarrow ID/EX.PC_INCR$

$EX/MEM.IR \leftarrow ID/EX.IR$

4. *Memory* (MEM)

In the Memory (MEM) stage, if the instruction involves a memory operation (load or store), the **Data Memory** (DM) is accessed accordingly.

Load instruction: $MEM/WB.LMD \leftarrow DM[EX/MEM.ALU_OUT]$

Store instruction: $DM[EX/MEM.ALU_OUT] \leftarrow EX/MEM.B$

Moreover, the ALU output, incremented PC and IR values are propagated to the next pipeline stage.

$MEM/WB.ALU_OUT \leftarrow EX/MEM.ALU_OUT$

$MEM/WB.PC_INCR \leftarrow EX/MEM.PC_INCR$

$MEM/WB.IR \leftarrow EX/MEM.IR$

5. *Write Back* (WB)

At last, in the Write Back (WB) stage, the result of the operation is written back to the destination register in the register file.

R-type instruction: $RF[MEM/WB.IR[RD]] \leftarrow MEM/WB.ALU_OUT$

I-type ALU instruction: $RF[MEM/WB.IR[RS2]] \leftarrow MEM/WB.ALU_OUT$

Load instruction: $RF[MEM/WB.IR[RS2]] \leftarrow MEM/WB.LMD$

Jump and Link instruction: $RF[LR] \leftarrow MEM/WB.PC_INCR$

As mentioned previously, the whole path of each instruction through the 5 stages is completely controlled by the **Control Unit**, which generates all the necessary control signals at each stage to properly drive the datapath components and ensure the correct execution of the instruction.

Figure 2.4 shows the full schematic of the DLX processor, including both the Datapath and the Control Unit and also the external memory modules. The control signals generated by the CU are highlighted

in **red**, the Datapath signals are in **black**, while the Reset signal is highlighted in **blue**. The only signals that are not shown in the schematic are the **enable** signals of the pipeline registers, which come from the CU as well. In the next chapters, we will go into more detail about the individual components of the Datapath and Control Unit, explaining their design and functionality. In particular:

- Chapter 3 will focus on describing the **Datapath** components
- Chapter 4 will describe the **Control Unit** design and how it generates the control signals.
- Chapter 5 will focus on the implementation of the **Instruction Memory**, **Data Memory** and **Data Cache**
- Chapter 6 will discuss how **pipeline hazards** are detected and resolved in our pipelined architecture and the techniques used to mitigate their impact on performance.

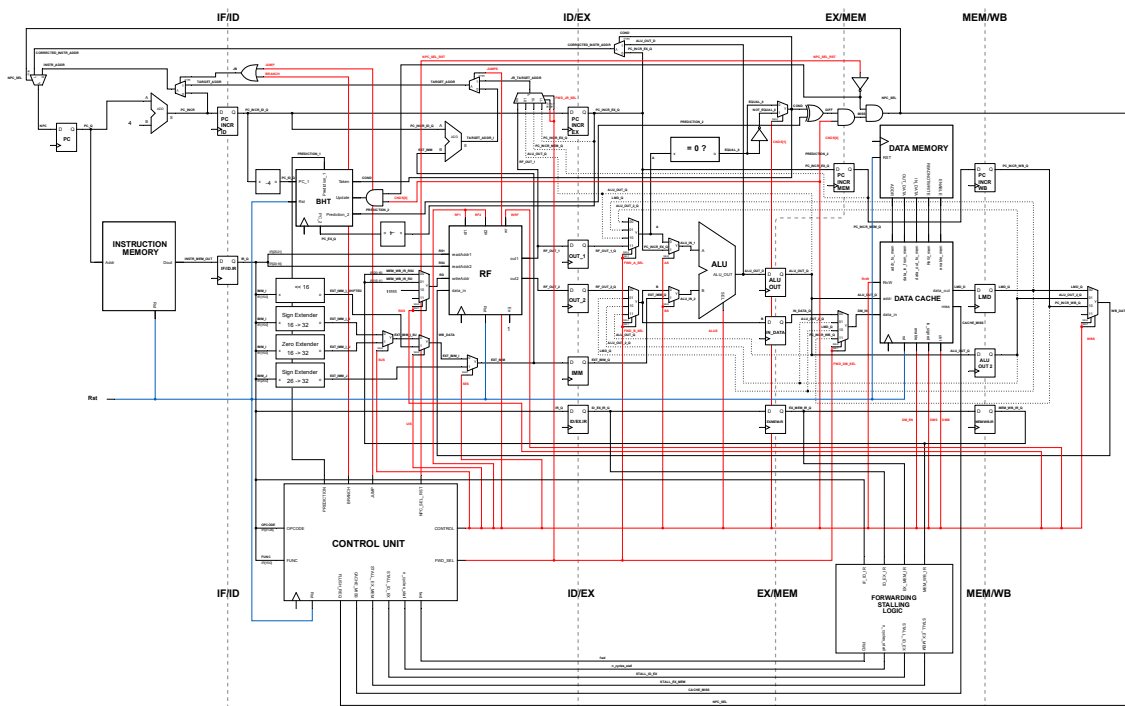


Figure 2.4: Full DLX processor schematic. In red the control signals from the CU. In blue the Reset signal. In black the datapath signals.

2.3 Structure of source files

The source files of the DLX microprocessor are organized as in Figure 2.5.

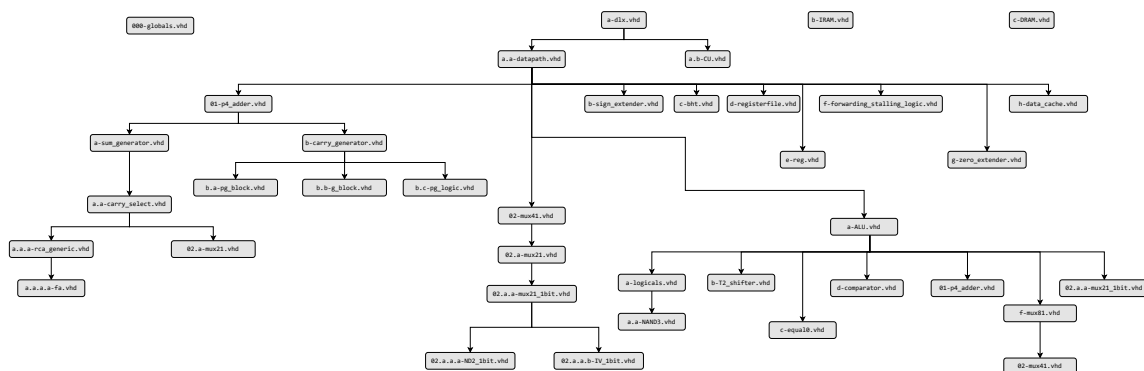


Figure 2.5: Structure of source files of the DLX microprocessor

CHAPTER 3

Datapath

3.1 Overall Structure

In the last section, we introduced the datapath component, which is the executive unit of the processor. In this section, our purpose is to give a complete description of all the sub-components forming the basic datapath of our processor. Later on, we will also describe the additional components that have been added to support hazard management and improve performance, namely the Data Cache, Branch History Table and Forwarding/Stalling Unit.

The structure of the basic datapath is shown in Figure 2.3. The datapath is composed of several modules, each one performing a specific function. The main modules are the following:

- ALU (Section 3.2)
- Registers (Section 3.3)
- Register File (Section 3.4)
- Multiplexers (Section 3.2.7)
- Zero Detector (Section 3.2.5)
- Sign and Zero Extenders (Section 3.5)

3.2 ALU

3.2.1 Overview

The ALU is a critical component of the DLX processor's datapath, responsible for executing a arithmetic, logical and comparison operations on 32-bit operands. Figure 3.1 shows its schematic symbol. It has three input ports: **A** and **B** (the input operands), and an 11-bit control signal **Se1** that determines the specific operation to be performed. The ALU produces a single 32-bit output (**ALU_OUT**) based on the selected operation.

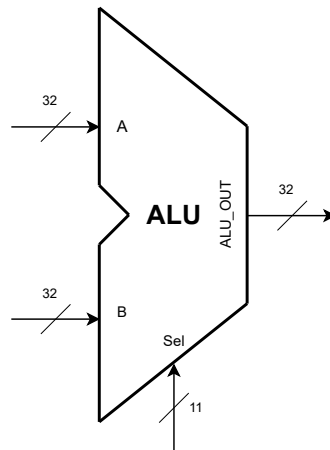


Figure 3.1: Schematic symbol of a 32-bit ALU. A and B are the operands, Sel is the 11-bit control signal and ALU_OUT is the output.

Figure 3.2 illustrates the internal logic of the ALU, highlighting its modular design. Each functional block within the ALU is responsible for a specific set of operations, and the control signal **Sel** orchestrates the selection and execution of these operations. The main units composing the ALU are the following:

- Logical Unit (Section 3.2.2)
- Shifter (Section 3.2.3)
- Adder/Subtractor (Section 3.2.4)
- Equal 0 checker (Section 3.2.5)
- Comparator (Section 3.2.6)
- Output Multiplexer (Section 3.2.7)

The previously listed components perform operations on inputs A and B (or on some intermediate signal), based on the control signals **Sel** (coming from the Control Unit).

The VHDL specification of this component can be found inside the file `a-ALU.vhd`, and its subcomponents in the folder `a-ALU.core/`, both located in the `vhdlsim/src/a.a-datapath.core/` directory. The implementation follows a structural design approach, where each subcomponent is instantiated and interconnected to form the complete ALU.

3.2.2 Logicals

Description

The Logical Unit is responsible for performing bitwise logical operations on the input operands. It supports the following logical operations:

- AND

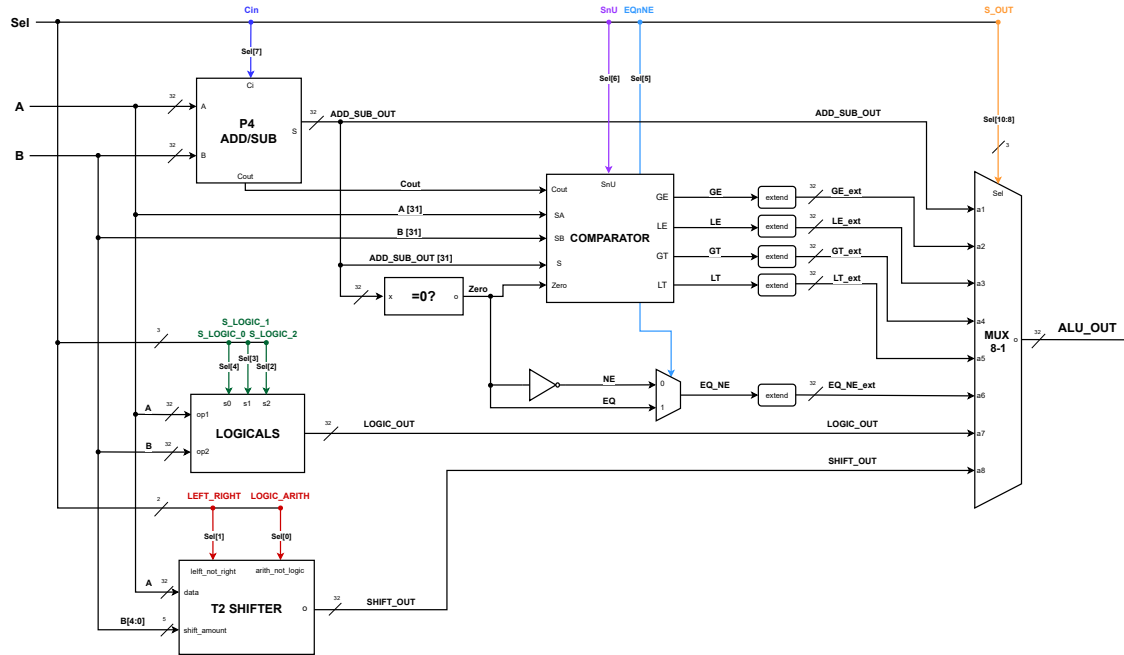


Figure 3.2: Architectural-level schematic representation of the ALU

- OR
- XOR

Its block representation can be seen in Figure 3.3.

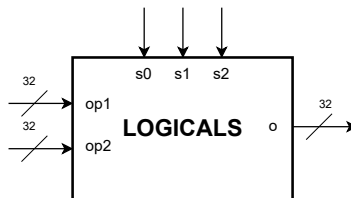


Figure 3.3: Schematic symbol of a 32-bit logical unit

Input ports:

- op1: operand 1 (32-bit)
- op2: operand 2 (32-bit)
- s0: selection signal 0 (1-bit)
- s1: selection signal 1 (1-bit)
- s2: selection signal 2 (1-bit)

Output ports:

- o: output (32-bit)

Function performed:

s0	s1	s2	o
0	0	1	op1 op2 (AND)
1	1	0	$\text{op1} \oplus \text{op2 (XOR)}$
1	1	1	$\text{op1} + \text{op2 (OR)}$

Table 3.1: Logical operations performed based on selection signals

Implementation

It is implemented using a combination of 3-input, 32-bit NAND gates, as shown in figure 3.4. The selection signals, extended to 32 bits, are used to select the desired operation. **s0**, **s1** and **s2** are extended to 32 bits before entering the NAND gates. The choice of this implementation with two levels of NAND gates is optimal both for area and performance.

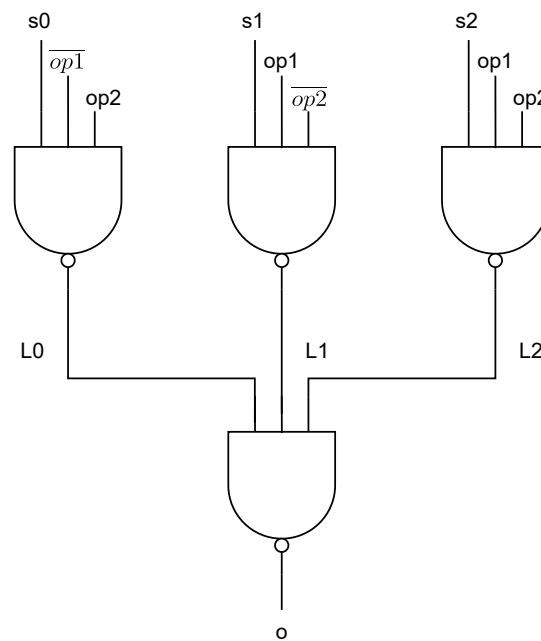


Figure 3.4: Gate-level schematic representation of a 32-bit logical unit

The output evaluates to:

$$\begin{aligned}
 o &= \overline{L0} \overline{L1} \overline{L2} \\
 &= \overline{L0} + \overline{L1} + \overline{L2} \\
 &= \overline{s0 \text{ op1 op2}} + \overline{s1 \text{ op1 op2}} + \overline{s2 \text{ op1 op2}} \\
 &= s0 \overline{\text{op1}} \text{op2} + s1 \text{op1} \overline{\text{op2}} + s2 \text{op1 op2}
 \end{aligned} \tag{3.1}$$

Table 3.2 shows how to get from Equation 3.1 to Table 3.1

s0 s1 s2	o
0 0 1	op1 op2 (AND)
1 1 0	$\overline{\text{op1}} \text{op2} + \text{op1} \overline{\text{op2}} = \text{op1} \oplus \text{op2}$ (XOR)
1 1 1	$\overline{\text{op1}} \text{op2} + \text{op1} \overline{\text{op2}} + \text{op1 op2} = \text{op1} + \text{op2}$ (OR)

Table 3.2: Truth table of logical unit

Integration in the ALU

The Logical Unit is integrated into the ALU as in Figure 3.2 and the mapping between its ports and the ALU signals is the following:

- op1 → A
- op2 → B
- s0 → S_LOGIC_0 (Sel[4])
- s1 → S_LOGIC_1 (Sel[3])
- s2 → S_LOGIC_2 (Sel[2])
- o → LOGIC_OUT

The output produced is signal LOGIC_OUT. Table 3.3 shows the truth table of the Logical Unit as a component of the ALU.

Sel[4]	Sel[3]	Sel[2]	LOGIC_OUT
	0	0	1
			A B
	1	1	0
			$A \oplus B$
	1	1	1
			$A + B$

Table 3.3: Truth table of the logical unit as a component of the ALU

The output LOGIC_OUT directly feeds into the output multiplexer of the ALU.

VHDL Implementation

The VHDL specification of this component is `a-logicals.vhd`, located in the `vhdlsrc/src/a.a-datapath.core/a-ALU.core/` directory. The implementation follows a structural approach, in which the only subcomponent used is a 3-input NAND gate, specified in the file `a.a-NAND3.vhd`, located in the same folder.

3.2.3 Shifter

Description

The **Shifter** is a functional component of the ALU that performs bitwise shifting operations on a 32-bit input operand. It supports both **logical** and **arithmetic** shifts, as well as left and right directions. The specific operation is determined by control signals. This particular shifter was designed based on the one found in the **UltraSPARC T2** architecture, seen during the course. Its schematic symbol can be seen in Figure 3.5.

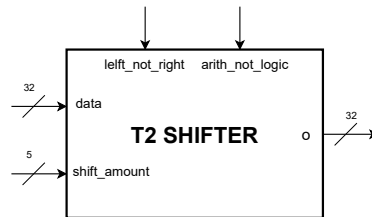


Figure 3.5: Schematic symbol of the T2-architecture-based shifter.

Input ports:

- **data**: operand (32-bit)
- **shift_amount**: shift amount (5-bit)
- **left_not_right**: left/right shift control signal (1-bit)
- **arith_not_logic**: arithmetic/logical shift control (1-bit)

Output ports:

- **o**: output (1-bit)

Function performed:

left_not_right	arith_not_logic	o
0	0	data >> shift_amount (SRL)
0	1	data >>> shift_amount (SRA)
1	—	data << shift_amount (SLL)

Implementation

The implementation is based on the UltraSPARC T2 shifter architecture, which was chosen for its performance (using shift registers wouldn't have been optimal in terms of speed and area). Figure 3.6 shows the logic circuit of the shifter. Its mechanism consists of three phases:

1. **Mask generation**
2. **Mask selection**
3. **Fine-grained shift**

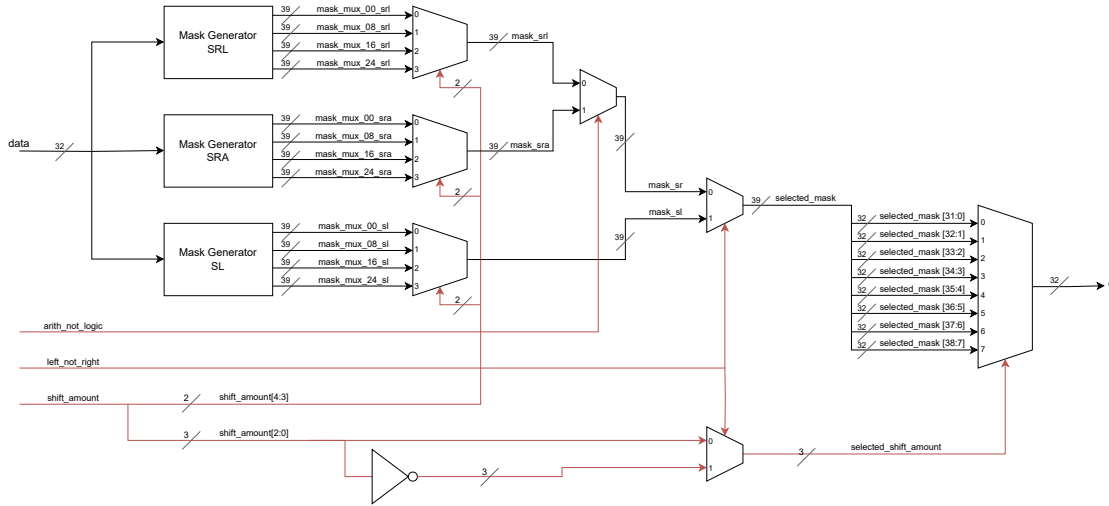


Figure 3.6: Logic diagram of the T2-architecture-based shifter.

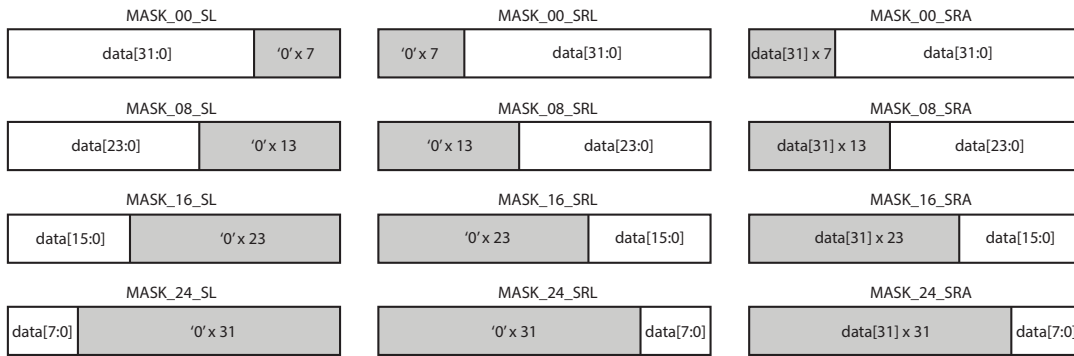


Figure 3.7: All the possible masks generated by the shifter

1. Mask generation The masks are generated by simply concatenating a subportion of **data** with a repetition of 0 (logical shift) or **data[MSB]** (arithmetic shift) bits. There are a total of 12 masks generated that account for all combinations of all shift amounts (0-31) and shift types (SLL, SRL, SRA). All the possible masks are shown in Figure 3.7. For every shift type, each mask allows for a range of 8 possible shifts:

- MASK_00_XXX → shift 0-7
- MASK_08_XXX → shift 8-15
- MASK_16_XXX → shift 16-23
- MASK_24_XXX → shift 24-31

XXX stands for either SLL (shift left logical), SRL (shift right logical) or SRA (shift right arithmetic).

2. Mask selection The 2 MSBs of **shift_amount** (**shift_amount[4:3]**) are used to select the correct mask (in terms of shift amount, not shift type), as each configuration of those corresponds to a specific shift range and therefore a specific mask. This is shown in Table 3.4.

shift_amount	shift range	selected mask
00xxx	0-7	MASK_00_XXX
01xxx	8-15	MASK_08_XXX
10xxx	16-23	MASK_16_XXX
11xxx	24-31	MASK_24_XXX

Table 3.4: Mask selection based on the 2 MSBs of shift_amount

After this initial selection, one out of three possible masks is picked, depending on the shift type. The choice is performed using the `arith_not_logic` and `left_not_right` control signals. These cascaded operations pick a certain mask, which becomes the `selected_mask`. For example if `shift_amount=10101`, `left_not_right=0` and `arith_not_logic=1`, then `selected_mask=MASK_16_SRA`.

3. Fine-grained shift The last step is the fine-grained shifting operation, that implements a specific shift among the range allowed by `selected_mask`. This operation is controlled by the 3 LSBs of `shift_amount` as there are 8 possible shifts to choose from. In practice, a 32-bit subportion of `selected_mask` is picked, and this subportion is the final shifted operand, hence the output of the shifter. The exact subportion to be picked based on the shift amount and direction is shown in Table 3.5.

If a left shift has to be performed, the 3 selection signals are inverted.

shift_amount	shift right	shift left	shifts
xx000	<code>selected_mask[31:0]</code>	<code>selected_mask[38:7]</code>	{0,8,16,24}
xx001	<code>selected_mask[32:1]</code>	<code>selected_mask[37:6]</code>	{1,9,17,25}
xx010	<code>selected_mask[33:2]</code>	<code>selected_mask[36:5]</code>	{2,10,18,26}
xx011	<code>selected_mask[34:3]</code>	<code>selected_mask[35:4]</code>	{3,11,19,27}
xx100	<code>selected_mask[35:4]</code>	<code>selected_mask[34:3]</code>	{4,12,20,28}
xx101	<code>selected_mask[36:5]</code>	<code>selected_mask[33:2]</code>	{5,13,21,29}
xx110	<code>selected_mask[37:6]</code>	<code>selected_mask[32:1]</code>	{6,14,22,30}
xx111	<code>selected_mask[38:7]</code>	<code>selected_mask[31:0]</code>	{7,15,23,31}

Table 3.5: Fine-grained shift selection based on the 3 LSBs of shift_amount

Both the mask selection and the final shifting phases are executed via multiplexers.

Subcomponents

Here's the subcomponents used to build the shifter, that we can see in Figure 3.6:

- Mask generator SL
- Mask generator SRL
- Mask generator SRA
- 8-1 multiplexer (32-bit)
- 4-1 multiplexer (32-bit)
- 2-1 multiplexer (32-bit)

The three mask generators, as previously mentioned, concatenate bits of `data` with either 0 or `data[MSB]` bits, depending on the shift type. Each mask generator produces 4 masks, one for each shift range. Their hardware implementation is hidden as they are implemented behaviorally, but can be checked after synthesis. The multiplexers instead, are thoroughly described in Subsection 3.2.7.

Integration in the ALU

The Shifter is integrated into the ALU as depicted in Figure 3.2 and its ports are mapped to the ALU signals as follows:

- `data` → `A`
- `shift_amount` → `B[4:0]`
- `left_not_right` → `LEFT_RIGHT` (`Sel[1]`)
- `arith_not_logic` → `LOGIC_ARITH` (`Sel[0]`)
- `o` → `SHIFT_OUT`

Table 3.6 shows the truth table of the Shifter as a component of the ALU.

<code>Sel[1]</code>	<code>Sel[0]</code>	<code>SHIFT_OUT</code>
0	0	<code>A >> B[4:0]</code>
0	1	<code>A >>> B[4:0]</code>
1	-	<code>A << B[4:0]</code>

Table 3.6: Truth table of the Shifter as a component of the ALU

`SHIFT_OUT` directly feeds into the output multiplexer of the ALU.

VHDL Implementation

The VHDL specification of this component can be found in `b-T2_shifter.vhd`, located in the `vhdlsim/src/a.a-datapath.core/a-ALU.core/` directory. The implementation follows a mixed structural and behavioral approach: the mask generators are implemented behaviorally, while the multiplexers are specified structurally, using the corresponding entities to be described later.

3.2.4 Adder/Subtractor

Description

The **Adder/Subtractor** is a central component of the ALU, responsible for performing both addition and subtraction on the input operands. Its design is based on the adder of the **Intel Pentium 4** processor seen during the course, that is a Sparse Tree Carry Lookahead Adder able to achieve much higher performance than a Ripple Carry Adder at the cost of a smaller area than that of a full CLA. We had already designed this adder during the laboratories, so we recycled that design, adding the logic for subtraction. The schematic symbol of the Adder/Subtractor can be seen in Figure 3.8.

Input ports:

- `A`: first operand (32-bit)
- `B`: second operand (32-bit)

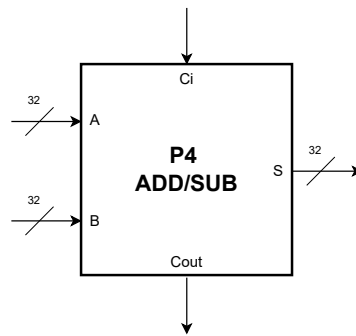


Figure 3.8: Schematic symbol of the P4 adder block.

- Ci: carry-in (1-bit)

Output ports:

- S: sum (1-bit)
- Co: carry-out (1-bit)

Function performed:

Ci	Operation
0	$S = A + B$
1	$S = A - B$

NOTE: here the + symbol indicates arithmetic addition, not bitwise OR.

Implementation

Operation Addition and subtraction are performed using 2's complement arithmetic. The subtraction operation is implemented by inverting the bits of B and setting the carry-in to 1. The SUB signal is used to decide whether an addition or a subtraction is performed. In particular, the operation performed is always the following:

$$S = A + (B \oplus Ci) + Ci \quad (3.2)$$

So that:

- when $Ci = 0$, $S = A + B$ and the operation is an addition
- when $Ci = 1$, $S = A + \bar{B} + 1$ and the operation is a subtraction

Hierarchical Composition The Adder/Subtractor is built from several reusable blocks. Following a top-down approach, we now explore them all:

- **Top-Level Adder (P4_ADDER) (01-p4.adder.vhd):** the top-level block, that is the P4-adder itself, properly connects two main sub-components: the **Carry Generator** and the **Sum Generator**, as depicted in Figure 3.9. It also handles the inversion of B based on the Ci signal.

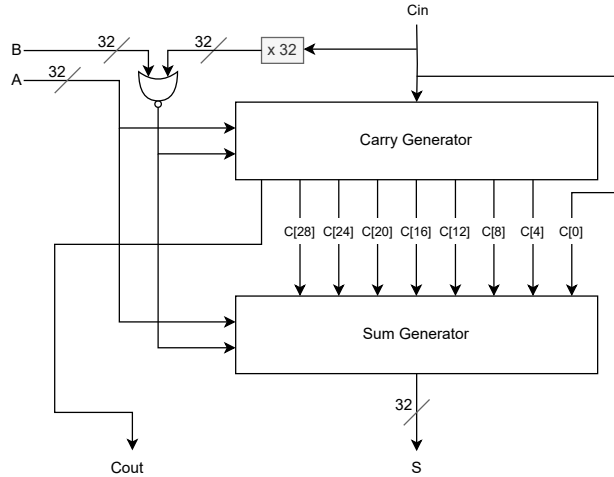


Figure 3.9: Logic diagram of the P4 adder block.

- **Carry Generator (b-carry_generator.vhd)**: the **Carry Generator** is a hierarchical module that efficiently determines the carry-in for each block of the adder, enabling fast parallel computation of sums. It is built using a combination of propagate/generate logic and block-level carry computation.

Internally, the Carry Generator uses the following components:

- **PG_LOGIC (b.c-pg_logic.vhd)**: For each bit, this module computes the *propagate* (p_i) and *generate* (g_i) signals, which indicate whether a carry will be propagated or generated at that position. Two architectures are available: one for carry computation (using OR for propagate) and one for sum computation (using XOR for propagate).
- **PG_BLOCK (b.a-pg_block.vhd)**: Combines propagate and generate signals across blocks of bits, allowing hierarchical computation of block-level propagate (P_{ij}) and generate (G_{ij}) signals, where:

$$P_{i:j} = P_{i:k} \cdot P_{k-1:j} \quad (3.3)$$

$$G_{i:j} = G_{i:k} + P_{i:k} \cdot G_{k-1:j} \quad (3.4)$$

- **G_BLOCK (b.b-g_block.vhd)**: Computes the generate signal for a block, further supporting the hierarchical structure.

The Carry Generator organizes these components in a tree-like structure, recursively combining signals to compute the carry-in for each block. This approach reduces the critical path compared to a simple ripple-carry scheme, enabling higher performance for wide adders. The output is a vector of carry signals, one for each block, which are then used by the Sum Generator to compute the final sum.

- **Sum Generator (a-sum_generator.vhd)**: Produces the final sum by combining the outputs of the carry generator and the propagate signals. The Sum Generator is designed to work hierarchically, similar to the Carry Generator, and uses the computed carry-in for each bit along with the propagate signals to determine the sum output.

Internally, the Sum Generator uses several instances of a single sub-component, the **Carry Select Adder** (see Figure 3.11); in particular, it concatenates 8 Carry Select Adders, each accounting for 4 bits of the final sum.

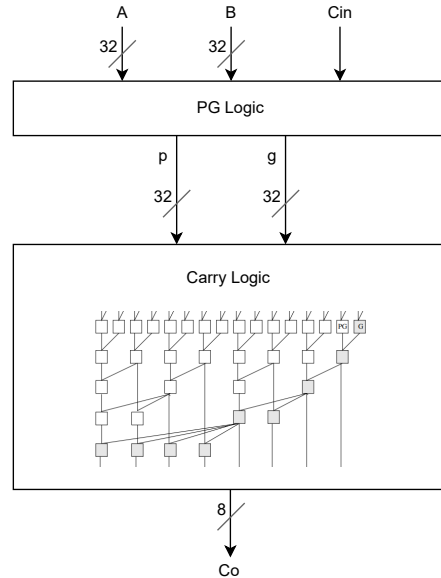


Figure 3.10: Logic diagram of the carry generator.

Carry Select Adder (`a.a-carry_select.vhd`): improves speed by precomputing 4-bit sum results for both possible carry-in values (0 or 1) and selecting the correct value as soon as the carry-in is available, through multiplexers. The Carry Select Adder itself is built using two sub-components (as shown in Figure 3.12): 4-bit **Ripple Carry Adders** and 2-1 **multiplexers**.

- **Ripple-Carry Adder (RCA)** (`a.a.a-rca_generic.vhd`): it is build using a chain of 4 **Full Adders (FA)** (as shown in Figure 3.13) and computes the sum for a 4-bit block given a specific carry-in.

Full Adder (FA) (`a.a.a-a-fa.vhd`): Implements single-bit addition with carry-in and carry-out. The outputs produced are:

- * Sum: $S = A \oplus B \oplus Ci$
- * Carry-out: $Co = A \cdot B + Ci \cdot (A \oplus B)$

The gate-level schematic of a Full Adder is shown in Figure 3.14.

- **2-1 Multiplexer** (`02.a-mux21.vhd`): selects between two 4-bit sum outputs based on the carry-in signal, allowing the Carry Select Adder to output the correct sum once the carry-in is known. A detailed description of this component is given later.

The Sum Generator arranges these components in a hierarchical structure, matching the organization of the Carry Generator. This enables efficient computation of the sum for wide adders, minimizing delay and maximizing performance. The final output is a vector of sum bits representing the result of the addition or subtraction operation.

Integration in the ALU

The Adder/Subtractor is integrated in the ALU as in Figure 3.2. The mapping between its ports and the ALU signals is the following:

- $A \rightarrow A$

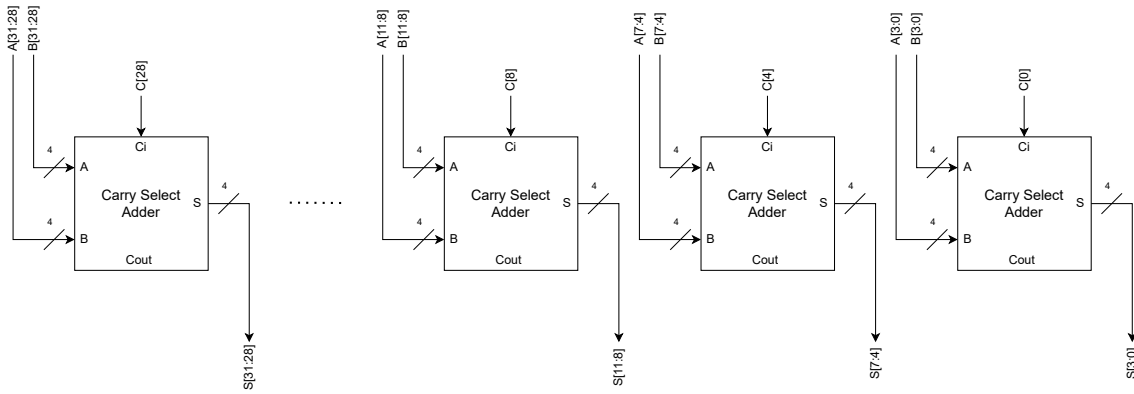


Figure 3.11: Logic diagram of the sum generator.

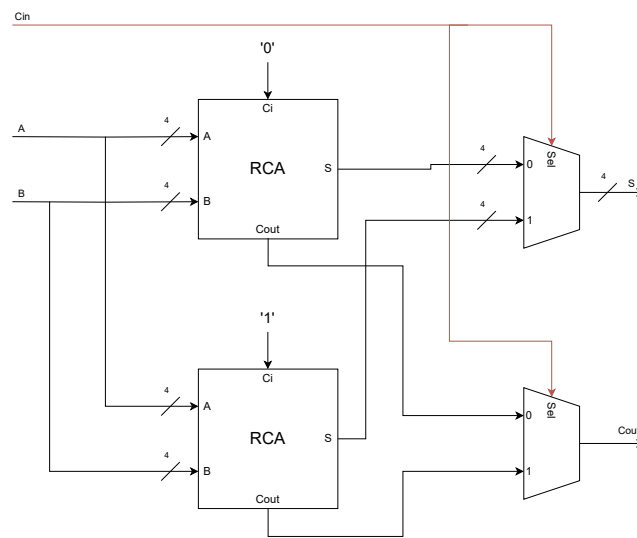


Figure 3.12: Logic diagram of a 4-bit carry select adder.

- $B \rightarrow B$
- $C_i \rightarrow \text{Cin}$ (Sel[7])
- $S \rightarrow \text{ADD_SUB_OUT}$
- $\text{Cout} \rightarrow \text{Cout}$

Its operation as an ALU component is summarized in Table 3.7.

The output sum (signal ADD.SUB_OUT) is used in three different ways:

- it is directly fed into the output multiplexer of the ALU (see Section 3.2.7).
- it is fed to the Equal 0 checker (see Section 3.2.5) to generate the Zero flag.
- its MSB is fed into the Comparator (see Section 3.2.6).

The Carry-Out (Co) is only fed to the Comparator.

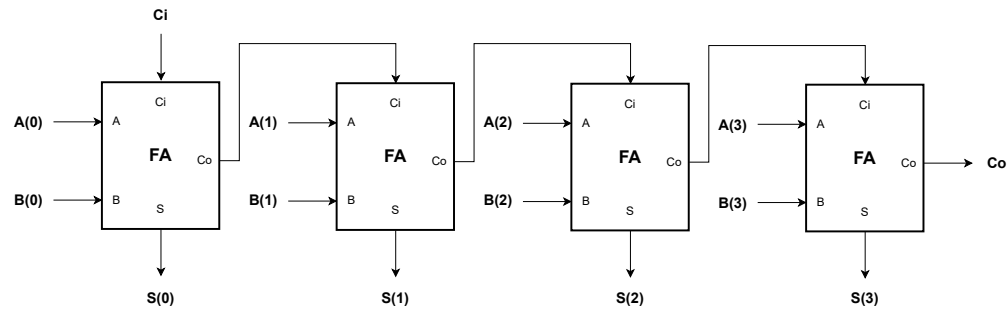


Figure 3.13: Logic diagram of a 4-bit ripple-carry adder.

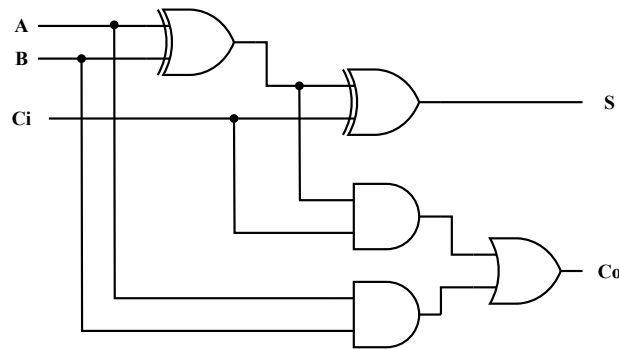


Figure 3.14: Logic diagram of a full adder.

VHDL implementation

The VHDL specification of this component can be found in `01-p4_adder.vhd`, located in the `vhdlsim/src/a.a-datapath.core/` directory. The implementation clearly follows a structural description, and its components are implemented in the folder `vhdlsim/src/a.a-datapath.core/01-p4_adder.core`.

3.2.5 Equal 0 checker

Description

This functional block checks whether a 32-bit input operand is null, returning 1 if that's the case, 0 otherwise. Its block representation can be seen in Figure 3.15.

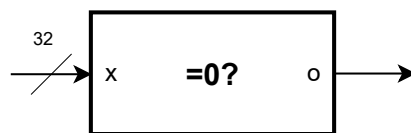


Figure 3.15: Schematic symbol of the equal 0 checker.

Input ports:

- x: operand (32-bit)

Sel[7]	Operation
0	ADD_SUB_OUT = A + B
1	ADD_SUB_OUT = A - B

Table 3.7: Truth table of the Adder/Subtractor as a component of the ALU

Output ports:

- o: output (1-bit)

Function performed:

$$o = \begin{cases} 1 & \text{if } x = \{0\} \\ 0 & \text{if } x \neq \{0\} \end{cases} \quad (3.5)$$

Integration in the ALU

The Equal 0 checker is integrated into the ALU as in Figure 3.2. The mapping between its ports and the ALU signals is the following:

- x $\rightarrow$ ADD_SUB_OUT
- o $\rightarrow$ Zero

and its output (Zero) is used in three ways:

- it is used as the zero flag by the Comparator (see Section 3.2.6).
- it is used as signal EQ (Equality)
- it is negated, producing signal NE (Not Equality)

VHDL Implementation

The VHDL specification of this component can be found in `c-equal_0.vhd`, located in the `vhdlsim/a.a-datapath.core/a-ALU.core/` directory. Since this component was designed with a behavioral style, the details of the implementation are hidden by the abstraction layer of VHDL and are only known after synthesis.

3.2.6 Comparator

Description

The **Comparator** module determines the comparison between two **signed** or **unsigned** operands by analyzing the flags produced by a subtraction operation, and produces comparison outputs.

Inputs ports:

- Cout: Carry-out flag from the subtraction operation.
- SA: Sign bit (MSB) of the first operand (A)
- SB: Sign bit (MSB) of the second operand (B).
- S: Sign flag, reflects the sign of the subtraction result.

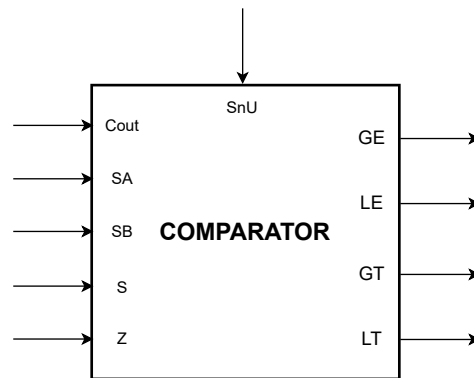


Figure 3.16: Schematic symbol of the comparator based on subtractor flags

- **Z**: Zero flag
- **SnU**: Signed/Unsigned control signal.

Outputs ports:

- **GE**: Greater or Equal
- **LE**: Less or Equal
- **GT**: Greater Than
- **LT**: Less Than

Logic function: The comparator uses the flags from the subtraction operation to determine the relationship between the two operands. The logic is as follows:

- For **unsigned** comparisons (**SnU** = 0):
 - **GE** is asserted if **Cout** is high
 - **LE** is asserted if **Cout** is low or if **Z** is high.
 - **GT** is asserted if **Cout** is high and **Z** is low.
 - **LT** is asserted if **Cout** is low
- For **signed** comparisons (**SnU** = 1):
 - **GE** is asserted if **S** and **OVF** are equal or if **Z** is high.
 - **LE** is asserted if **S** and **OVF** differ or if **Z** is high.
 - **GT** is asserted if **S** and **OVF** are equal and **Z** is low.
 - **LT** is asserted if **S** and **OVF** differ.

The overflow condition **OVF** is detected by checking if the sign bits of the operands differ and if the sign of the result differs from the sign of the first operand.

Implementation

The comparator operates as follows:

- **Unsigned Comparison:**

$$\text{GEU} = \text{Cout}$$

$$\text{LEU} = \overline{\text{Cout}} + \text{Z}$$

$$\text{GTU} = \text{Cout} \cdot \overline{\text{Z}}$$

$$\text{LTU} = \overline{\text{Cout}}$$

- **Signed Comparison:**

- The overflow flag (**OVF**) is set when the sign bits of the operands differ and the sign of the result differs from the sign of the first operand: $\text{OVF} = (\text{SA} \oplus \text{SB}) \cdot (\text{SA} \oplus \text{S})$. Then:

$$\text{GES} = \overline{(\text{S} \oplus \text{OVF})} + \text{Z}$$

$$\text{LES} = (\text{S} \oplus \text{OVF}) + \text{Z}$$

$$\text{GTS} = \overline{(\text{S} \oplus \text{OVF})} \cdot \overline{\text{Z}}$$

$$\text{LTS} = (\text{S} \oplus \text{OVF})$$

The final outputs are then selected based on the **SnU** control signal, as shown in Table 3.8.

SnU	LE	GE	LT	GT
0	LEU	GEU	LTU	GTU
1	LES	GES	LTS	GTS

Table 3.8: Comparator output selection based on the **SnU** control signal

Integration in the ALU

The Comparator is integrated into the ALU as in Figure 3.2. Its ports are mapped to the ALU signals as follows:

- **Cout** → **Co**
- **SA** → **A[31]**
- **SB** → **B[31]**
- **S** → **ADD.SUB_OUT[31]**
- **Z** → **Zero**
- **SnU** → **SnU** (**Se1[6]**)
- **GE** → **GE**
- **LE** → **LE**
- **GT** → **GT**
- **LT** → **LT**

The outputs **GE**, **LE**, **GT**, and **LT** are extended to 32 bits (with 31 leading zeros) and fed into the output multiplexer of the ALU (see Section 3.2.7).

VHDL Implementation

The VHDL specification of this component can be found in `d-comparator.vhd`, located in the `vhdsim/src/a.a-datapath.core/a-ALU.core/` directory. The implementation follows a behavioral approach, clearly describing the logic functions and the selection mechanism based on the `SnU` signal.

3.2.7 Output Multiplexer

Description

The output multiplexer is a crucial component of the ALU that selects one of several possible outputs of the previous blocks. In particular we use a **8-1 multiplexer**. Its schematic symbol can be seen in Figure 3.17.

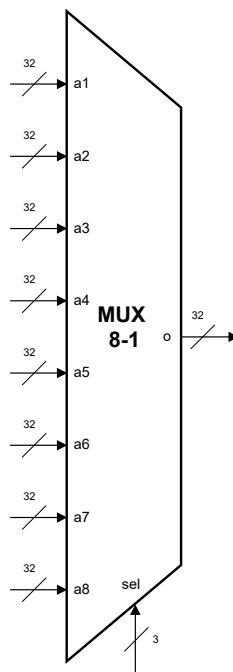


Figure 3.17: Schematic symbol of the 8-1 multiplexer

Input ports:

- **a1**: input 1 (32-bit)
- **a2**: input 2 (32-bit)
- **a3**: input 3 (32-bit)

- **a4**: input 4 (32-bit)
- **a5**: input 5 (32-bit)
- **a6**: input 6 (32-bit)
- **a7**: input 7 (32-bit)
- **a8**: input 8 (32-bit)
- **sel**: selection signals (3-bit)

Output ports:

- **o**: output (32-bit)

Function performed: the 8-1 multiplexer brings one of the 8 inputs to the output, based on the 3-bit selection signal **sel**, as detailed in Table 3.9.

sel	o
000	a1
001	a2
010	a3
011	a4
100	a5
101	a6
110	a7
111	a8

Table 3.9: Truth table of the 8-1 multiplexer

Implementation

The 8-1 multiplexer is implemented using a hierarchical approach, with smaller muxes, as shown in Figure 3.18. It is built using 4-1 multiplexers and 2-1 multiplexers as basic building blocks. The selection signals are used to control the flow of data from the inputs to the output.

4-1 multiplexer (02-mux41.vhd): selects one of four 32-bit inputs based on 2 selection signals. It is built using three 2-1 multiplexers, as shown in Figure 3.19.

2-1 multiplexer (02.a-mux21.vhd): selects one of two 32-bit inputs based on a single selection signal. It is implemented using 32 instances of 2-1 multiplexers of 1-bit. Its logic diagram is shown in Figure 3.20.

1-bit 2-1 multiplexer (02.a.a-mux21_1bit.vhd): selects one of two 1-bit inputs based on a single selection signal. It is implemented using NAND2 gates (02.a.a-a-ND2_1bit.vhd) and an inverter (02.a.a.b-IV_1bit.vhd) and its logical expression is the following:

$$Y = \overline{Y1} \cdot \overline{Y2} = \overline{(A \cdot \overline{S}) \cdot (\overline{B} \cdot S)} = \overline{\overline{A \cdot \overline{S}} + \overline{\overline{B} \cdot S}} = A \cdot \overline{S} + \overline{B} \cdot S = A \cdot \overline{S} + B \cdot S$$

This design with NAND gates is probably the best logical design of a 2-1 multiplexer in terms of area. Its gate-level netlist is shown in Figure 3.21.

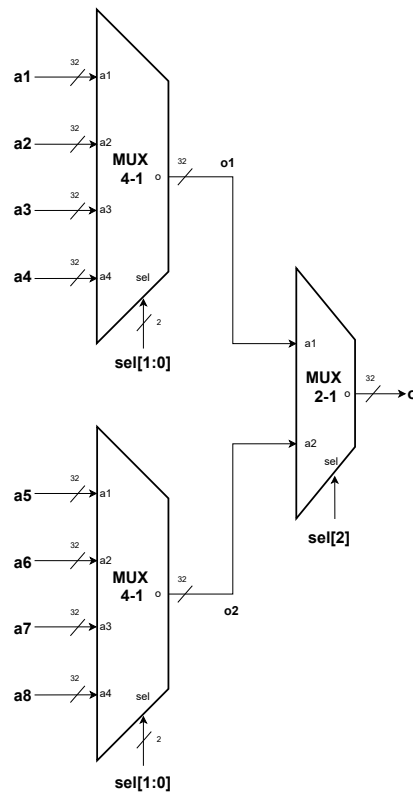


Figure 3.18: Detailed schematic representation of the 8-1 multiplexer

Integration in the ALU

The output multiplexer is integrated into the ALU as in Figure 3.2. The mapping between its ports and the ALU signals is the following:

- a1 → ADD.SUB_OUT
- a2 → GE_ext
- a3 → LE_ext
- a4 → GT_ext
- a5 → LT_ext
- a6 → EQ_NE_ext
- a7 → LOGIC_OUT
- a8 → SHIFT_OUT
- sel → S_OUT[2:0] (Sel[10:8])
- o → ALU_OUT

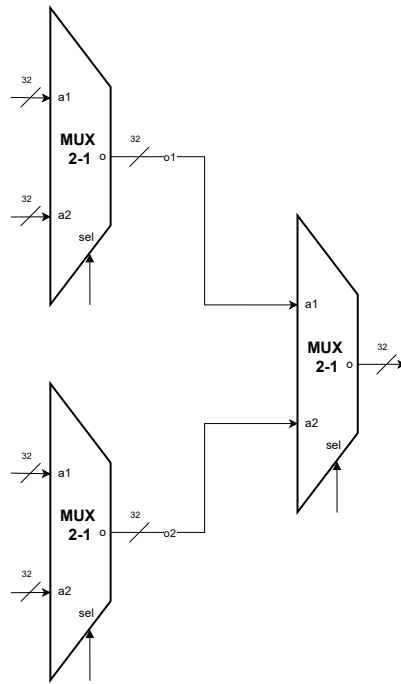


Figure 3.19: Detailed schematic representation of the 4-1 multiplexer

`EQ_NE_ext` is the 32-bit extension of `EQ_NE`, which corresponds to EQ if `EQnNE` = 1 and to NE if `EQnNE` = 0, as shown in Figure 3.2. Control signal `EQnNE` corresponds to `Sel[5]`. Table 3.10 shows the truth table of the output multiplexer as a component of the ALU.

<code>Sel[10]</code>	<code>Sel[9]</code>	<code>Sel[8]</code>	ALU_OUT
0	0	0	<code>ADD_SUB_OUT</code>
0	0	1	<code>GE_ext</code>
0	1	0	<code>LE_ext</code>
0	1	1	<code>GT_ext</code>
1	0	0	<code>LT_ext</code>
1	0	1	<code>EQ_NE_ext</code>
1	1	0	<code>LOGIC_OUT</code>
1	1	1	<code>SHIFT_OUT</code>

Table 3.10: Truth table of the output multiplexer as a component of the ALU

The output `ALU_OUT` is the final output of the ALU.

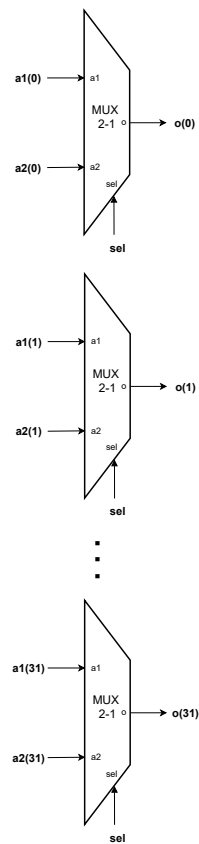


Figure 3.20: Detailed schematic representation of the 2-1 multiplexer

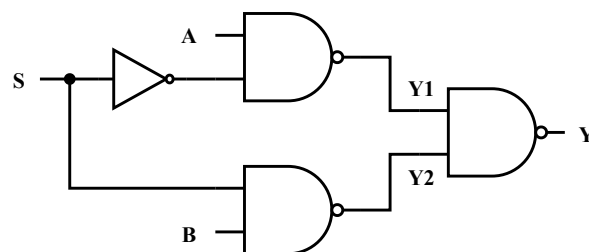


Figure 3.21: Detailed schematic representation of the 1-bit 2-1 multiplexer

3.2.8 Complete overview of ALU operations

Table (3.2.8) shows a complete overview of the operations performed by the ALU, based on the selection signals:

Sel[10:8] S_OUT[2:0]	Sel[7] Cin	Sel[6] SnU	Sel[5] EQnNe	Sel[4:2] S_LOGIC[0:2]	Sel[1] LEFT_RIGHT	Sel[0] LOGIC_ARITH	Operation
000	0	—	—	—	—	—	ADD
000	1	—	—	—	—	—	SUB
001	1	0	—	—	—	—	GE (U)
001	1	1	—	—	—	—	GE (S)
010	1	0	—	—	—	—	LE (U)
010	1	1	—	—	—	—	LE (S)
011	1	0	—	—	—	—	GT (U)
011	1	1	—	—	—	—	GT (S)
100	1	0	—	—	—	—	LT (U)
100	1	1	—	—	—	—	LT (S)
101	1	—	0	—	—	—	NE
101	1	—	1	—	—	—	EQ
110	—	—	—	001	—	—	AND
110	—	—	—	110	—	—	XOR
110	—	—	—	111	—	—	OR
111	—	—	—	—	0	0	SRL
111	—	—	—	—	0	1	SRA
111	—	—	—	—	1	—	SLL

Each of these ALU operations is involved in the execution of a subset of instructions in the instruction set:

- ADD → add, addu, addi, addui, beqz, bnez, lb, lbu, lh, lhu, lw, sb, sh, sw
- SUB → sub, subu, subi, subui, sgeu, sgeui, sge, sgei, sltu, sltui, slt, slti, sleu, sleui, sle, slei, sgtu, sgtui, sgt, sgti, sne, sneui, seq, seqi
- GE (U) → sgeu, sgeui
- GE (S) → sge, sgei
- LE (U) → sleu, sleui
- LE (S) → sle, slei
- GT (U) → sgtu, sgtui
- GT (S) → sgt, sgti
- LT (U) → sltu, sltui
- LT (S) → slt, slti
- NE → sne, sneui
- EQ → seq, seqi

- AND → `and, andi`
- XOR → `xor, xori`
- OR → `or, ori`
- SRL → `srl, srli`
- SRA → `sra, srai`
- SLL → `sll, slli`

3.3 Registers

3.3.1 Overview

Registers are the basic sequential elements that hold data temporarily during processing. They are essential for the CPU's operation, as they allow to store data and use it when needed. Registers are faster to access than other forms of memory, such as RAM, making them crucial for high-performance computing.

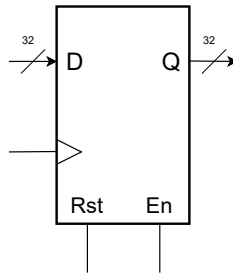


Figure 3.22: Schematic symbol of a register

Input ports:

- **D**: Data input (32-bit)
- **En**: Enable (1-bit)
- **Rst**: Reset (1-bit)
- **Clk**: Clock (1-bit)

Output ports:

- **Q**: Data output (32-bit)

Function performed:

- On the rising edge of the clock, if the enable signal (**En**) is high (1), the data present on the input port (**D**) is stored in the register and made available on the output port (**Q**).
- If the reset signal (**Rst**) is high (1), the register is cleared, and the output (**Q**) is set to zero, regardless of the clock or enable signals.

3.3.2 Implementation

Since the register is implemented behaviorally, the details of its implementation are hidden by the abstraction layer of VHDL and can only be visible after synthesis. However, as we know, a 32 bit register is usually implemented by using 32 **D Flip-Flops**, one for each bit of the data word. Each flip-flop has the same functionality of the register but for a single bit. The enable, reset and clock signals are shared among all flip-flops.

3.3.3 VHDL implementation

The VHDL specification of this component can be found inside the file `e-reg.vhd`, located in the `vhdlsrc/src/a.a-datapath.core/` directory. The design of this entity is behavioral.

3.3.4 Integration in the Datapath

The registers are widely used in the datapath of the DLX processor, as they are the essential piece of the pipeline implementation. In fact, each register can be seen as a **pipeline register**, separating two consecutive stages of the pipeline by holding the data of the instruction just arrived at the later stage and blocking data being produced in the earlier stage. For example, the IR (Instruction Register) belongs to the IF/ID pipeline registers, which separate the Instruction Fetch (IF) and Instruction Decode (ID) stages. Similarly, the ID/EX pipeline registers separate the Instruction Decode (ID) and Execute (EX) stages, and so on for the other pipeline registers (EX/MEM and MEM/WB).

3.4 Register File

3.4.1 Overview

The register file is a critical component of the CPU's datapath, serving as a small, fast storage area for holding multiple registers. It allows for quick access to data that the CPU needs to process, significantly enhancing performance. The register file typically consists of a set of registers that can be read from or written to based on control signals.

3.4.2 Implementation

Since this component was designed with a behavioral style, the details of the implementation are hidden by the abstraction layer of VHDL and are only known after synthesis. The register file in this design is configurable, allowing for a variable number of registers

$$\text{Number of registers} = 2^{\text{NBIT_ADD}}$$

and a register width of

$$\text{Register width} = \text{NBIT_DATA}.$$

In the datapath, the RF is instantiated with the default values `NBIT_ADD = 5`, resulting in 32 registers, each 32 bits wide (`NBIT_DATA = 32`).

Ports description

Input ports:

- **Clk**: clock signal for synchronizing write operations.
- **Rst**: reset signal to initialize the register file.

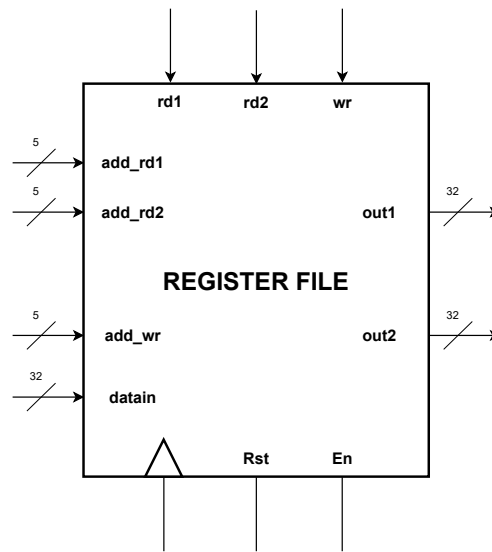


Figure 3.23: Schematic symbol of a register file

- **rd1**: control signal to enable reading from the first read port.
- **rd2**: control signal to enable reading from the second read port.
- **wr**: control signal to enable writing to the register file.
- **En**: enable signal. When low, no read or write operations occur.
- **add_wr**: write address input, specifying which register to write data to.
- **add_rd1**: read address input for the first read port, specifying which register to read data from.
- **add_rd2**: read address input for the second read port, specifying which register to read data from.
- **datain**: data input for writing to the register file.

Output ports:

- **out1**: data output for the first read port, providing the value from the specified register.
- **out2**: data output for the second read port, providing the value from the specified register.

Read and write operations

This register file supports asynchronous read and synchronous write operations.

When the read control signals (**rd1** and **rd2**) and the enable signal (**En**) are asserted, the data from the registers specified by **add_rd1** and **add_rd2** are immediately available on the outputs **out1** and **out2**, respectively.

For write operations, when the write control signal (**wr**) and the enable signal (**En**) are asserted on the falling edge of the clock (**Clk**), the data present on **datain** is written into the register specified by **add_wr**. Data is sampled on the falling edge of the clock to ensure proper timing and synchronization

within the CPU's datapath, being sure that the word to be written is ready at the input ports of the RF when the writing operation occurs.

If a register is accessed for reading and writing in the same clock cycle, the read operation will return the right value in the second half of clock period, after the falling edge triggered the write operation. This behavior is essential for the case when the RF is used in both the ID and WB stages.

Reset behavior

When the reset signal is asserted, all registers in the register file are initialized to zero. This ensures that the CPU starts with a known state, preventing any undefined behavior due to residual data in the registers. Reset is asynchronous, meaning that it has priority over all other operations.

Debugging

For debugging purposes, the RF performs the `rewrite_regfile` procedure, which writes the contents of the register file to a text file named `reg_file.mem` every time the registers' content is updated. This allows for easy monitoring and verification of the register file's state after simulation.

3.4.3 VHDL implementation

The VHDL specification of this component can be found inside the file `d-registerfile.vhd`, located in the `vhdsim/src/a.a-datapath.core/` directory. The design of this entity is behavioral so the details of its implementation are abstracted away by VHDL and can only be known after synthesis.

3.5 Sign and Zero Extenders

3.5.1 Overview

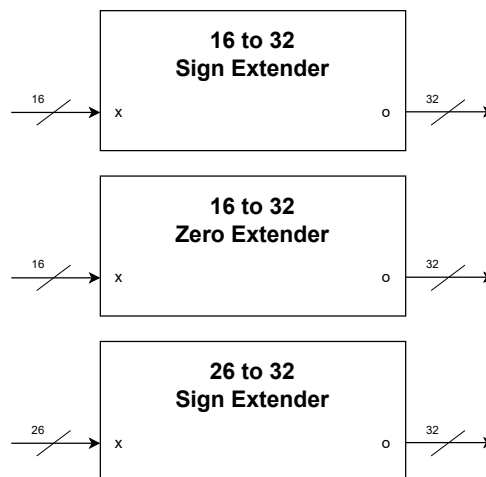


Figure 3.24: Schematic symbols of the sign and zero extenders present in the datapath

The sign and zero extenders are components used in the datapath to modify the bit-width of immediate values extracted from instructions. Immediate values inside the instructions are indeed shorter than the standard word size of the processor.

In our DLX case, immediate values have either a size of 16 bits (I-TYPE instructions) or 26 bits (J-TYPE instructions), while the processor operates on 32-bit words.

The sign extender is used to extend the immediate value by replicating the sign bit (the most significant bit) to fill the higher-order bits. The zero extender fills the higher-order bits with zeros. It is used for unsigned immediate values.

All in all, 3 of these extenders are present in the datapath:

- **Sign Extender for I-TYPE instructions:** extends 16-bit immediate values to 32 bits by replicating the sign bit.
- **Zero Extender for I-TYPE instructions:** extends 16-bit immediate values to 32 bits by filling higher-order bits with zeros.
- **Sign Extender for J-TYPE instructions:** extends 26-bit immediate values to 32 bits by replicating the sign bit.

2 multiplexers are used to select the output of only one of these 3 extenders. They are controlled by the SES and SUS control signals.

3.5.2 VHDL Implementation

The VHDL specification of this component can be found inside the files `b-sign_extender.vhd` and `g-zero_extender.vhd`, located in the `vhdlsim/src/a.a-datapath.core/` directory.

CHAPTER 4

Control Unit

4.1 Overview

The **Control Unit** (CU) is a critical component of the microprocessor architecture, responsible for directing the operation of the processor. It interprets instructions from the instruction set and generates the necessary control signals to orchestrate the activities of all the datapath components.

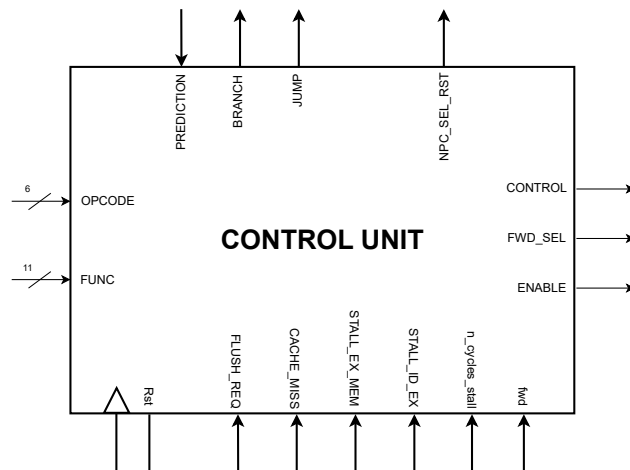


Figure 4.1: Schematic symbol of the Control Unit component.

4.2 Implementation

The Control Unit of our DLX processor is implemented as a **Hardwired** Control Unit.

This type of CU works by using a **control word memory**, that accessed with the OPCODE field of an instruction, returns the full control word of that instruction, with the control signals needed by the datapath to execute it (except for ALU signals and special control signals), for all stages of the pipeline.

ALU control signals are produced based on the OPCODE (for I-type instructions) or on FUNC (for R-type instructions). We had to separate ALU control signals from the rest of the control signals

because the control word memory is accessed with the OPCODE, which uniquely determines the control signals of an instruction, while ALU control signals are determined by the FUNC field in the case of R-type instructions.

This CU is also responsible for providing extra control signals to handle **control hazards** (branch prediction and jumps), **data hazards** (forwarding and stalling) and **cache misses**.

4.2.1 Ports description

Input ports:

- **Clk**: clock signal used to synchronize write operations. (1-bit)
- **Rst**: reset signal used to initialize the memory to an initial state when asserted. (1-bit)
- **OPCODE**: identifies the instruction to be executed. (6-bit)
- **FUNC**: represents the function code for R-type instructions. (11-bit)
- **FLUSH_REQ**: input signal indicating a flush request to correct branch misprediction. (1-bit)
- **CACHE_MISS**: input signal indicating a cache miss has occurred. (1-bit)
- **STALL_EX_MEM**: input signal requesting a stall up to the EX/MEM pipeline stage. (1-bit)
- **STALL_ID_EX**: input signal requesting a stall up to the ID/EX pipeline stage. (1-bit)
- **n_cycles_stall**: integer input signal indicating the number of cycles to stall. (integer)
- **fwd**: concatenation of all forwarding detection signals (12-bit)
- **PREDICTION**: indicates the result of the branch prediction. (1-bit)

Output ports:

- **CONTROL**: concatenation of all regular control signals for the datapath components and ALU control signals (31-bit)
- **ENABLE**: concatenation of all pipeline registers enable signals (5-bit)
 - **WBIFEN**: enable signal for the WB/IF pipeline registers (PC)
 - **IFIDEN**: enable signal for the IF/ID pipeline registers
 - **IDEXEN**: enable signal for the ID/EX pipeline registers
 - **EXMEMEN**: enable signal for the EX/MEM pipeline registers
 - **MEMWBEN**: enable signal for the MEM/WB pipeline registers
- **BRANCH**: output signal telling whether a branch has to be taken in ID (1-bit)
- **JUMP**: output signal indicating to jump (1-bit)
- **NPC_SEL_RST**: output signal forcing NPC_SEL to 0 when asserted.
- **FWD_SEL**: concatenation of all output forwarding selection signals.
 - **FWD_A_SEL**: forwarding selection signal for ALU input A (2-bit)
 - **FWD_B_SEL**: forwarding selection signal for ALU input B (2-bit)
 - **FWD_DM_SEL**: forwarding selection signal for Data Memory input (2-bit)
 - **FWD_JR_SEL**: forwarding selection signal for Jump Register target address (2-bit)

4.2.2 Regular control signals

The **CONTROL** output port is a concatenation of all the regular control signals needed by the datapath components to execute an instruction. These signals are grouped based on the pipeline stage they belong to:

- **ID** stage signals:
 - **UIS: Upper Immediate Signal.** Selects shifting or not shifting the immediate by 16 bits leftwards (**0**: no shift, **1**: shift)
 - **SUS: Signed Unsigned Selection.** Selects whether to consider the immediate as signed or unsigned (**0**: signed, **1**: unsigned)
 - **SES: Signed Extender Selection.** Selects the correct sign-extension of the immediate depending on whether we have an I-Type or J-Type instruction (**0**: 16 $\rightarrow$ 32 [*I-Type*], **1**: 26 $\rightarrow$ 32 [*J-Type*])
 - **RF1: RF read port 1 enable.** Enables reading from register file port 1 (**0**: disable, **1**: enable).
 - **RF2: RF read port 2 enable.** Enables reading from register file port 2 (**0**: disable, **1**: enable).
 - **JUMPS: Jump select.** Select the correct target address source for jump instructions (**0**: immediate, **1**: register).
- **EX** stage signals:
 - **BS: B Selection.** Selects the correct source for input B of the ALU (**0**: RF out 2, **1**: immediate).
 - **AS: A Selection.** Selects the correct source for input A of the ALU (**0**: RF out 1, **1**: PC+4).
 - **CNDS[1:0]: COND select.** Selects the condition that determines program flow (**00**: normal instruction, **01**: branch if equal 0, **11**: branch if not equal 0).
 - **ALUS[10:0]: ALU Selection.** Select signals for ALU operation. Details are explained later.
- **MEM** stage signals:
 - **DMS: Data Memory Signed.** Selects whether the data memory access is signed or unsigned (**0**: unsigned, **1**: signed).
 - **DMB[1:0]: Data Memory Bytes.** Selects the number of bytes to be written to/read from data memory (**00**: byte, **01**: half-word, **10**: word).
 - **RnW: Read not Write (Data Memory).** Selects whether to read from or write to data memory (**0**: write, **1**: read).
 - **DM_EN: Data Memory Enable.** Enables data memory access (**0**: disable, **1**: enable).
- **WB** stage signals:
 - **WBS[1:0]: Write Back Selection.** Selects the source of data to be written back to the register file (**00**: Data Memory Output, **01**: ALU Output, **10**: PC+4).
 - **WRF: Write Register File.** Enables writing to the register file (**0**: disable, **1**: enable).
 - **RDS[1:0]: Register Destination Selection.** Selects the destination register for the write back operation (**00**: RS2 [*I-Type ALU*], **01**: RD [*R-Type*], **10**: R31 [*Jump and Link*]).

Control Word Memory

The regular control signals of each instruction are stored in the **Control Word Memory**. The Control Word Memory is a LUT, implemented as a simple array of **control words**, indexed by the OPCODE. When an instruction is fetched, its OPCODE is used to look up the corresponding control word in the Control Word Memory, which is then used to generate the necessary control signals for the datapath components at the different pipeline stages.

Listing 4.1: VHDL code snippet the Control Word Memory

```

signal cw_mem : mem_array := (
    0      => "00011000000001001101", -- 0. R-type
    1      => "00000000000000000000", -- 1. //
    2      => "00100011000001000000", -- 2. JTYPE_J,
    3      => "00100011000001010110", -- 3. JTYPE_JAL,
    4      => "00010011010001000000", -- 4. ITYPE_BEQZ
    5      => "00010011110001000000", -- 5. ITYPE_BNEZ
    6      => "00000000000000000000", -- 6. //
    7      => "00000000000000000000", -- 7. //

    ...

    44 to 57 => "00000000000000000000", -- //
    58      => "01010010000001001100", -- 58. ITYPE_SLTUI
    59      => "01010010000001001100", -- 59. ITYPE_SLTUI
    60      => "01010010000001001100", -- 60. ITYPE_SLEUI
    61      => "01010010000001001100"  -- 61. ITYPE_SGEUI
);

```

4.2.3 ALU control signals

ALUS is an 11-bit signal that controls the operation of the ALU. The bits are defined as follows:

- ALUS[1:0]: control the shifter.
- ALUS[4:2]: control the logicals.
- ALUS[5]: controls whether an internal ALU signal indicates equality or inequality.
- ALUS[6]: controls the behaviour of the flags, specifying whether the ALU operation is signed or unsigned.
- ALUS[7]: controls the Cin signal for the adder.
- ALUS[10:8]: selection signals for the 8-1 output MUX.

The value of this signal is determined by the OPCODE field (for I-Type instructions) or on the FUNC field (for R-Type instructions).

4.2.4 Delaying the control signals

To ensure that the control signals are properly synchronized with the pipeline stages of the datapath, the CU implements a series registers containing different subportions of the control word, to delay

the control signals as they propagate through the pipeline stages. This ensures that each stage of the datapath receives the correct control signals at the appropriate clock cycle. The `CW_PIPE` process in the CU implements the control word pipelining.

Listing 4.2: VHDL code snippet the Control Word pipelining process

```

cw <= cw_mem(conv_integer(unsigned(OPCODE)));

-- process to pipeline control word
CW_PIPE : process (Clk, Rst)
begin
    if Rst = '1' then
        cw2      <= (others => '0');
        cw3      <= (others => '0');
        cw4      <= (others => '0');
        aluOpcode1 <= (others => '0');
    elsif rising_edge(Clk) then -- rising clock edge
        if (IDEXEN_s = '1') then
            cw2      <= cw(CW_SIZE - 1 - 6 downto 0);
            aluOpcode1 <= aluOpcode_i;
        end if;
        if (EXMEMEN_s = '1') then
            cw3 <= cw2(CW_SIZE - 1 - 6 - 4 downto 0);
        end if;
        if (MEMWBEN_s = '1') then
            cw4 <= cw3(CW_SIZE - 1 - 6 - 4 - 5 downto 0);
        end if;
    end if;
end process CW_PIPE;

```

ID stage control signals are taken from the `cw` signal (immediately as the instruction enters ID), EX stage control signals are taken from `cw2`, MEM stage control signals are taken from `cw3` and WB stage control signals are taken from `cw4`. It can be noticed that, if a specific pipeline register is disabled, the corresponding control signals are not updated and remain the same as in the previous cycle. This allows to implement stalls in the pipeline when needed.

4.2.5 Flush and Stall handling

As we will see later, sometimes the pipeline needs to be flushed or stalled to ensure correct execution of instructions. The CU is also responsible for handling flushes and stalls in the pipeline. **Flushes** are requested when wrong instructions are fetched; the CU then generates the necessary control signals to flush the incorrect instructions from the pipeline. **Stalls** are requested when a data hazard that can't be handled with forwarding is detected, or when a cache miss occurs; the CU then generates the necessary control signals to stall the pipeline for the required number of cycles.

The `FLUSH/STALL` control logic is implemented by a High Level **Mealy Finite State Machine** (FSM), that is shown in Figure 4.2. A Mealy representation is chosen because flush and stall requests are asynchronous with respect to the clock signal, and require an immediate response that can't wait for the next clock cycle. This can be achieved by a Mealy FSM, where outputs depend on both the current state and the current inputs, allowing for immediate reactions to input changes.

The FSM has the following states:

- WaitForReq: normal behavior, **waiting** for a flush or stall request event
- CacheMiss: handling a **cache miss** by stalling the pipeline as long as CACHE_MISS is asserted.
- BranchPred: handling **predicted taken** branches.
- FixBranchPred: correcting a **mispredicted branch** by flushing the pipeline.
- JumpState: handling **jump** instructions.
- IDEXStall: **stalling** the pipeline up to the ID/EX stage.
- EXMEMStall: **stalling** the pipeline up to the EX/MEM stage.

More details of how this FSM works are discussed in Chapter 5.3 and Chapter 6.

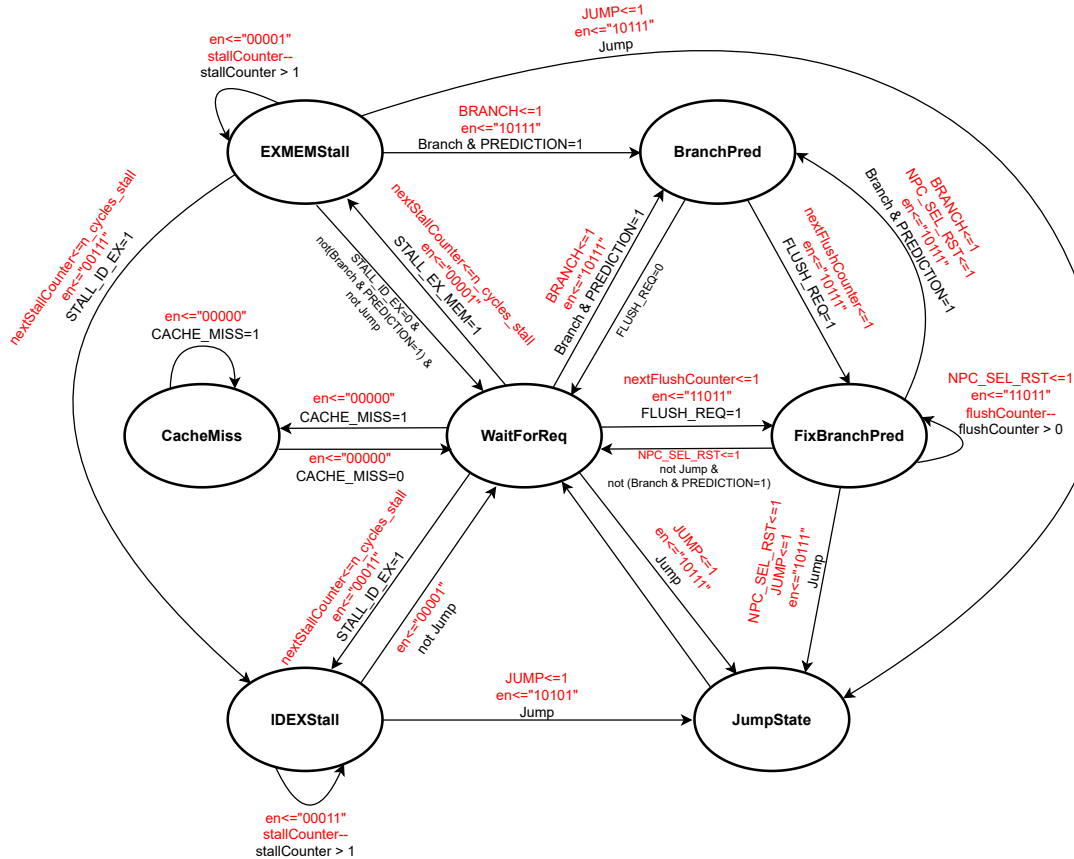


Figure 4.2: Mealy FSM for flush and stall logic. In black are indicated the conditions for state transitions and in red the outputs of the FSM. The state transition occurs at the clock rising edge.

4.2.6 VHDL Implementation

The VHDL specification of the control unit can be found inside the file `a.b-CU.vhd`, located in the `vhdsim/src/` directory. The design of this entity follows a behavioral style, so the details of its implementation are abstracted away by VHDL and can only be known after synthesis.

CHAPTER 5

Memories

In this chapter, we will describe the memory components used in our DLX microprocessor implementation.

5.1 Data Memory

5.1.1 Overview

The Data Memory is the component that allows the CPU to read and write big portions of data. It is slower but much bigger than the register file. It contains data specified in the `.data` section of an assembly file, which is used to provide initial values so that the program can read them. Its schematic symbol is shown in Figure 5.1.

More than a clock cycle is needed to perform read and write operations on data memory. However, an intermediate layer of cache allows concealing this behaviour to the CPU, which is deceived into believing that a single clock cycle is enough.

The Data Memory component is not part of the DLX processor, but it is an external module connected to the datapath through the Data Cache (described in Section 5.3). With the cache, there is no direct connection between the CPU's datapath and data memory, and the latter is involved only during cache misses (the data cache uses a write-back policy).

5.1.2 Implementation

Port description

Input ports:

- **Clk**: clock signal used to synchronize write operations.
- **Rst**: reset signal used to initialize the memory to an initial state when asserted.
- **ADDR**: address signal used to identify the memory byte to be targeted by read/write operations.
- **ENABLE**: enables memory operations.
- **READNOTWRITE**: specifies whether a read (`READNOTWRITE=1`) or a write (`READNOTWRITE=0`) has to be performed.

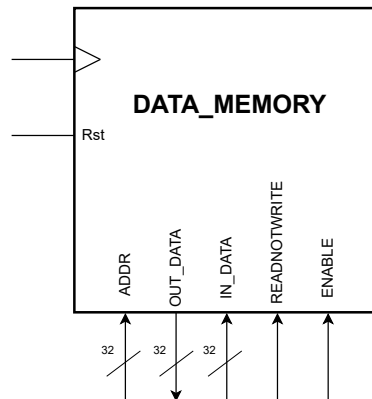


Figure 5.1: Schematic symbol of the data memory component.

- **DATA_READY**: signal asserted when data is ready at the **OUT_DATA** port. In this project's implementation, this signal is not used as the cache already knows in advance the number of clock cycles needed to access data memory.
- **IN_DATA**: input data coming from the cache during cache misses.

Output ports:

- **OUT_DATA**: data read from memory .

Reset behaviour

Reset is **asynchronous** and, when the **Rst** signal is set, it initializes the memory to an initial state using a non synthesizable VHDL procedure. It reads a text file (**data_mem_init.mem**) line by line and fills memory with its content. Resetting also sets the value of the internal counter to 0. Its purpose will be explained in the following paragraph.

Read and write operations

To model read and write operations taking more than 1 clock cycle, a high level FSM model is used, featuring a **counter**. This counter is incremented by one at every clock rising edge.

The counter threshold at which the read/write operation is effectively performed is configurable directly in the datapath: the **DATA_MEMORY** and **DATA_CACHE** components have, respectively, the generics **DATA_DELAY** and **MEMORY_ACCESS_CYCLES** to specify the delay.

When the counter reaches this threshold, it is reset to 0 and data is present at the output port (for read operations) or written into memory at the specified address (for write operations). The output signal **DATA_READY** is asserted in this specific instant, allowing for a possible asynchronous management of the memory. In particular, **DATA_READY** is obtained by the **or** of **int_data_ready**, signalling the end of a read, and **mem_ready**, flagging the end of a write.

While memory is busy retrieving reading or writing data, it expects address and **data_in** to be stable during all the cycles. This behaviour is enforced by the **DATA_CACHE**. In this project, as previously specified, data cache and data memory know in advance how many clock cycles are needed

for reading and writing operations into memory to take place, so the `DATA_READY` signal is not used. Write operations are synchronized with the **falling edge** of the clock cycle, so that they occur far from possible transitions.

Debugging

The `rewrite_content` procedure logs into a text file (`data_mem.mem`) the content of `DATA_MEMORY`. This is extremely useful for debugging purposes, as the correct execution of a program can be checked directly in the memory debug logs, where results of computations may be stored.

5.1.3 VHDL Implementation

The VHDL specification of this component can be found inside the file `c-DRAM.vhd`, located in the `vhdlsrc/src/` directory. The design of this entity is behavioral so the details of its implementation are abstracted away by VHDL and can only be known after synthesis.

5.2 Instruction Memory

5.2.1 Overview

Our DLX processor follows an **Harvard architecture**, as instruction and data memories are separate entities and store different content. The instruction memory is a **read-only** memory, which feeds the datapath with instructions. Its schematic symbol is shown in Figure 5.2.

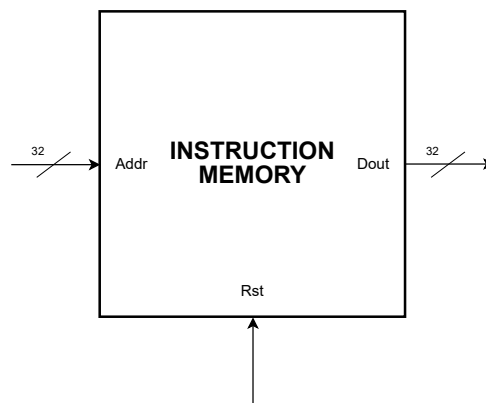


Figure 5.2: Schematic symbol of the instruction memory component.

5.2.2 Implementation

There is no instruction cache in this project and, since it would be extremely annoying to take more than a clock cycle to fetch a new instruction word, we made the assumption that this memory performs read operations asynchronously.

Port description

- **Rst**: Reset signal used to initialize the memory to a known state when asserted.

- **Addr:** Address signal used to identify the memory word to be read.
- **Dout:** Output instruction going to the datapath

Read operations

Reading is modeled as an asynchronous operation happening instantly, as it is for data cache.

Reset behaviour

As for the data memory, the content of instruction memory is initialized at reset by reading the content of a file (`instr_mem_init.mem`). This procedure increases very much the efficiency of the simulation process, as there is no need to load instructions one by one into memory before starting the simulation (with our Instruction Memory it couldn't be done anyway, as it is read-only).

5.2.3 VHDL Implementation

The VHDL specification of this component can be found inside the file `b-IRAM.vhd`, located in the `vhdsim/src/` directory.

5.3 Data Cache

5.3.1 Overview

The **Data Cache** is a crucial component in a processor, as it allows to deceive the CPU into believing that a memory access takes 1 clock cycle to execute. Of course, this is not always true as **cache misses** can occur.

The data cache acts as an intermediary between the datapath and the data memory. Data is transferred between the cache and the memory only during cache misses. The schematic symbol of our Data Cache is shown in Figure 5.3.

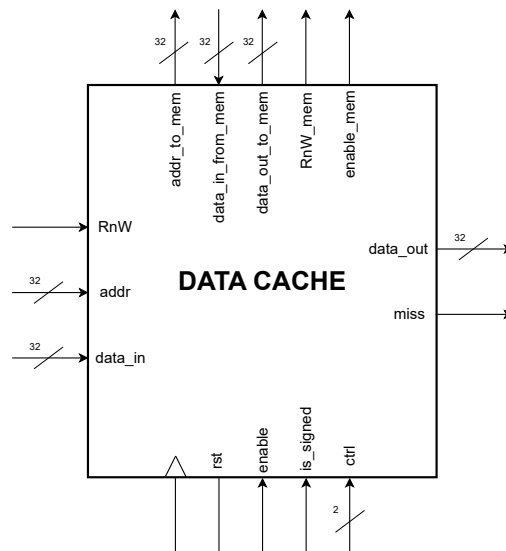


Figure 5.3: Schematic symbol of the data cache component.

5.3.2 Implementation

This project's Data Cache is a **byte-addressable, aligned, direct-mapped** cache with a **write-back** policy. Being it direct mapped, it does not require a replacement policy.

Cache parameters

Cache parameters are defined by the VHDL generics:

- **FILE_PATH**: name of the file where cache content is dumped at the end of the simulation.
- **MEMORY_ACCESS_CYCLES**: number of clock cycles needed for data to be read/written from/to memory.
- **K**: number of bits necessary to identify a word in memory ($\log_2$ MAINSIZE).
- **R**: number of bits necessary to identify a row in the cache ($\log_2$ NUMROWS).
- **W**: number of bits necessary to identify a word in the cache line ($\log_2$ WORD_OFFSET).

The cache is composed of 2^R lines of 2^W words. Each cache line includes also a **TAG** (as many memory lines can be mapped to the same cache line), a **dirty bit** (for the write-back policy). and a **reset bit** (used to invalidate the cache initial content and force the reading from memory. Indeed, the cache is empty at initialization, while data memory can store some initial content).

The address is decomposed into 3 fields:

- **TAG** : $address[K : W + R]$
- **LINE** : $address[W + R - 1 : W]$
- **WORD_OFFSET** : $address[W - 1 : 0]$

Let's now go through all the different scenarios to understand how the cache behaves.

Normal operation

The normal behaviour is the one where the address of the word to read/write is already included in the cache. This condition holds when the TAG field of the address in input is equal to the TAG of a specific cache line. Two are the possible scenarios:

- **Cache hit for reading operation**: the cache is accessed at word WORD_OFFSET of line LINE. Some extra logic is included, to read an half-word or a single byte.
- **Cache hit for writing operation**: the cache is accessed at word WORD_OFFSET of line LINE. The content of that word is overwritten by **datain** value. The dirty bit is set to 1, which signals that in the occurrence of a cache miss the content of this line will have to be transferred to the data memory. Also for writing operations some extra logic is present to allow writing half words or single bytes.

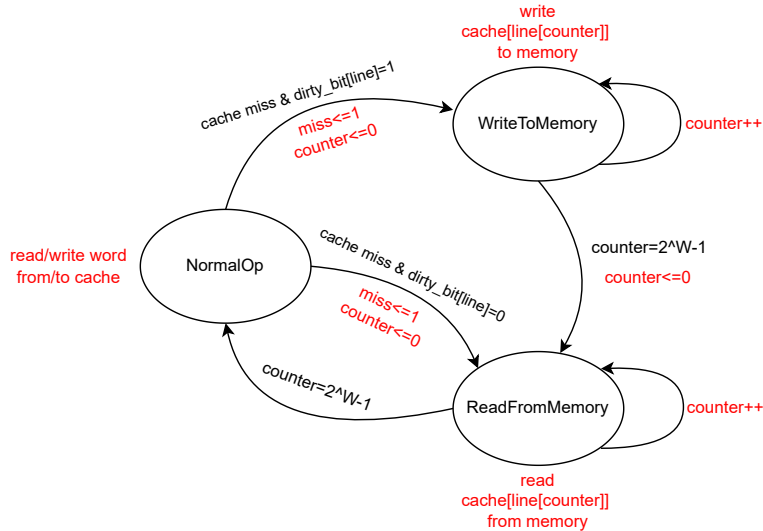


Figure 5.4: Mealy FSM for data cache operation. In black are indicated the conditions for state transitions and in red the outputs of the FSM. The state transition occurs at the clock rising edge.

Cache miss

As the TAG field of the input address does not match with the TAG field of the cache line LINE, the word to be accessed is not included in the cache. The cache will then replace the current wrong line with the correct line imported from memory. This process will last for $\text{MEMORY_ACCESS_CYCLES} * 2^W$.

If the dirty bit is set, the content of the cache line was modified and does not reflect the corresponding line in memory. In this scenario, in order not to lose data, the modified cache line will be written in memory and the dirty bit reset. This process too will last for $\text{MEMORY_ACCESS_CYCLES} * 2^W$.

The line to be accessed is now present in the cache, so a normal behaviour will follow.

Reading/Writing half words and single bytes

The DLX ISA includes load-store operations where half words and bytes are read/written. This data cache, as already mentioned, is byte-addressable and some extra logic is present to allow reading and writing operations not only regarding full words.

Example of cache operation

```

addi r1, r0, #1
sw 0(r0), r1
addi r2, r0, #0x1234
sw 4(r0), r2
addi r3, r0, #0x5678
sh 19(r0), r3
lw r4, values(r0)
sw 0(r0), r1

```

This example is the one described in the example program `asm/programs/cache_test_3.asm`, in the `vhdlsim/` folder. The cache is composed of 2 lines ($R=1$) of 4 words ($W=2$). Let's go through the program line by line to understand how the cache behaves.

Lines 0-1

```
addi r1, r0, #1
sw 0(r0), r1
```

The value 1 is stored at the address $0(r0) = 0x00000000$.

- TAG = "00000000";
- LINE = "00" $\rightarrow$ 0;
- WORD_OFFSET = "00" $\rightarrow$ 0;

Cache line 0 has the reset bit set, so a **CACHE_MISS** is issued and the line will be filled with 4 words coming from memory. Later, the word at line 0, offset 0 is filled with the value 1. The dirty bit of line 0 is set.

Lines 2-3

```
addi r2, r0, #0x1234
sw 4(r0), r2
```

The value 0x1234 is stored at the address $0(r0) = 0x00000004$.

- TAG = "00000000";
- LINE = "00" $\rightarrow$ 0;
- WORD_OFFSET = "01" $\rightarrow$ 1;

The word at line 1, offset 0 is filled with the value 0x1234.

Lines 4-5

```
addi r3, r0, #0x5678
sh 19(r0), r3
```

The value 0x5678 is stored at the address $19(r0) = 0x00000013$.

- TAG = "00000000";
- LINE = "01" $\rightarrow$ 1;
- WORD_OFFSET = "00" $\rightarrow$ 0;

Cache line 1 has the reset bit set, so a **CACHE_MISS** is issued and the line will be filled with 4 words coming from memory and, later, the writing operation can start.

Writing a half word at address 19 is not possible, as it would involve a misaligned access. A masking is applied (the LSB is set to 0) and the address is converted to the aligned address 18.

The target word is composed of 4 bytes at addresses: [19, 18, 17, 16]. The half word is placed at the bytes [19, 18].

All in all, the word at line 1, offset 0 will be 0x56780000.

Lines 6-7

```
lw r4, values(r0)
sw 0(r0), r1
```

The word to be loaded is at address `values(r0) = 256 = 0x00001000`.

- TAG = "00010000";
- LINE = "00" $\rightarrow$ 0;
- WORD_OFFSET = "00" $\rightarrow$ 0;

The TAG field of the address is different from the TAG field of cache line 0. Moreover, that cache line has the dirty bit set. As a consequence, the content of the line will be stored in memory and later the cache line will be filled with the content coming from the correct memory line. The dirty bit will then be reset.

5.3.3 VHDL Implementation

The VHDL specification of this component can be found inside the file `h-data_cache.vhd`, located in the `vhdl_sim/src/a.a-datapath.core/` directory. The design of this entity is behavioral so the details of its implementation are abstracted away by VHDL and can only be known after synthesis.

CHAPTER 6

Pipeline Hazards

6.1 Introduction

In this chapter, we will delve into the various types of hazards that we encountered in our pipeline processor and the strategies employed to handle them effectively. We will categorize hazards into two main types: **data hazards**, **control hazards**. The third known type, that would be **structural hazards**, does not occur in our architecture as there are no multicycle operations. For each of the two types, we will explain the nature of the hazard, provide examples of how they can affect instruction execution, and discuss the techniques used to handle them without affecting the overall performance of the pipeline too much. The theory behind this chapter is mostly taken from *Computer Architecture, A Quantitative Approach* [1], along with the inspiration for some drawings.

6.2 Data Hazards

Data Hazards occur when an instruction depends on the result of a previous instruction that has not yet completed its execution in the pipeline. They intrinsically arise because of the overlapping execution of instructions in the pipeline, and if we had no pipelining, there would be no data hazards at all. There are three types of data hazards: **Read After Write (RAW)**, Write After Read (WAR) and Write After Write (WAW). However, in a classic 5-stage pipeline, WAR and WAW hazards do not occur, therefore in our design, only RAW hazards had to be dealt with.

6.2.1 Read-After-Write (RAW) Hazards

RAW hazards occur when an instruction i needs to read a register that is written by a previous instruction that has not yet completed. The correct behavior should ensure that instruction i *Read* the register *After* it has been *Written* by the previous instruction, hence the name Read-After-Write. For example, consider the following sequence of instructions:

```
add R1, R2, R3
sub R4, R1, R5
```

In this case, one of the source operands of the second instruction (**sub**) is also the destination operand of the first instruction (**add**), namely R1. But when the **sub** instruction is in the ID stage, which is where source registers are read from the register file, the **add** instruction is still in the EX stage, and the new value of R1 is not yet available in the register file as it won't be written until **add** reaches the WB stage. This example is depicted in Figure 6.1, where the horizontal axis represents time (in clock

cycles) and the vertical axis represents the program order (in instructions); each stage is represented by its main Datapath unit: IF $\rightarrow$ IM (Instruction Memory), ID $\rightarrow$ RF (Register File), EX $\rightarrow$ ALU, MEM $\rightarrow$ DM (Data Memory), WB $\rightarrow$ RF. So if the **sub** instruction were to read R1 at this moment, it would get the old value of R1, leading to incorrect results.

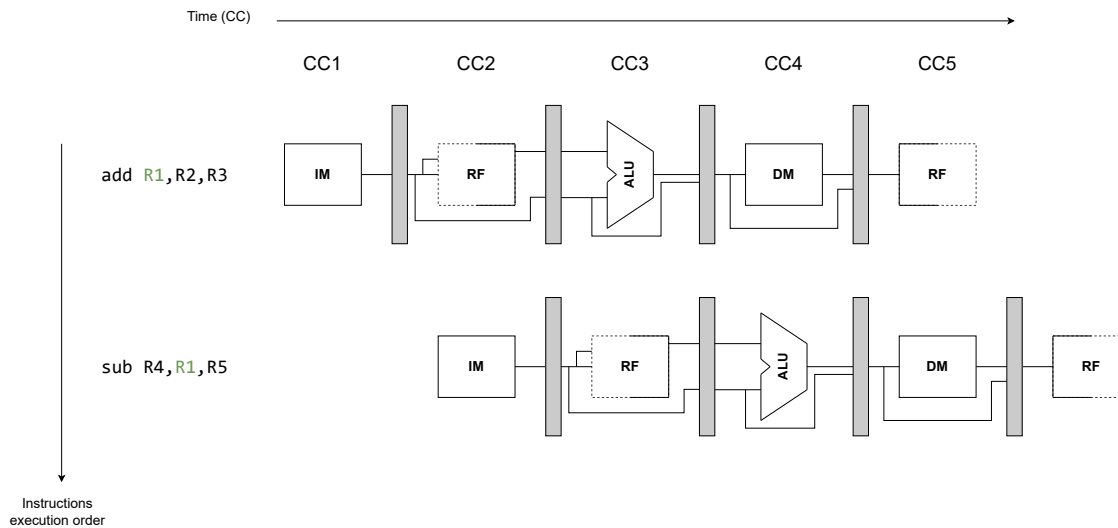


Figure 6.1: Example of a RAW hazard between two instructions

6.2.2 Handling RAW Hazards

Pipeline Stalls

The most immediate and simple way to handle RAW hazards is to introduce **stalls** in the pipeline. To stall the pipeline means to **freeze** all stages of the pipeline up to the one requiring the stall. In our design, this is done by disabling the corresponding pipeline registers, so that instructions in such stages effectively cannot advance. In this example, we effectively need to keep the **sub** instruction in the ID stage until the **add** instruction reaches the WB stage and writes the new value of R1 to the register file.

In order to do this, we have to disable the ID/EX pipeline register, to prevent the **sub** instruction from proceeding to the EX stage. But in doing so, we also have to disable IF/ID, otherwise a new instruction would enter the ID stage and the **sub** instruction would be overwritten. In the same way we also have to disable the PC so that it won't fetch new instructions. As a result, we freeze the whole pipeline up to the ID stage, coherently to the initial definition of stall. This situation will last 2 clock cycles, after which the **add** instruction reaches the WB stage and updates the value of R1 in the register file, allowing **sub** to read the correct value. Notice that if it wasn't possible to write in the RF and read the written value in the same cycle, we would need to stall for one more cycle.

At this point, the pipeline registers are re-enabled and the pipeline resumes its normal operation. This approach is simple to implement but can lead to significant **performance degradation**, especially for programs with high data dependency.

A much more efficient - although more expensive - approach is to use **data forwarding** (also known as data bypassing), which is the one we implemented in our design.

Data Forwarding

Data forwarding is a technique that allows the pipeline to directly use the result of an instruction as soon as it is computed, rather than waiting for it to be written back to the register file. In our previous example, implementing forwarding means that instead of stalling the `sub` instruction, we can directly forward the result of the `add` instruction from the **EX/MEM** pipeline register to the input of the ALU as soon as the `sub` instruction reaches the EX stage, without having to stall. This is depicted in Figure 6.2. This technique requires additional hardware, namely multiplexers, detection logic and more complex control logic, but allows us to avoid stalling the pipeline (in most cases), thus significantly improving performance.

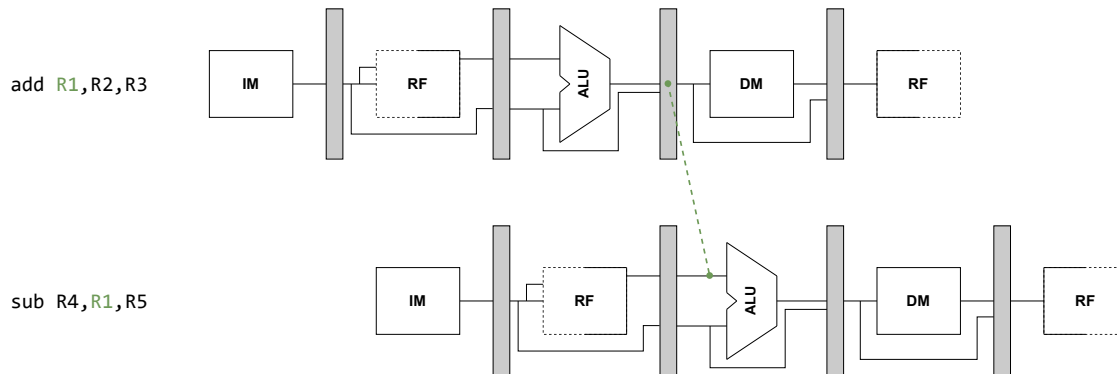


Figure 6.2: Example of data forwarding between two instructions

Implementation

In this design, forwarding was used whenever possible to solve RAW hazards. In some cases, as we'll see, stalls were still necessary. In some cases both stalling and forwarding are needed to solve the hazard. The **FORWARDING STALLING LOGIC**, in the Datapath, is the unit responsible for detecting hazards during the pipeline execution and generating the appropriate signals to send to the **Control Unit** which, starting from those, will generate the necessary control signals to resolve the hazard. Together, the FORWARDING STALLING LOGIC and the Control Unit guarantee the correct execution of the program even in presence of data hazards. To implement the FORWARDING STALLING LOGIC in VHDL, it was first necessary to list all possible scenarios of RAW hazards and formalize them in a way that allowed to implement their detection in hardware. As forwarding and stalling are orthogonal solutions, it was possible to treat them separately.

Forwarding Let us bring back our previous simple example of data dependency between two instructions:

```
add R1, R2, R3
sub R4, R1, R5
```

This case can be formalized as follows (refer to Figure 6.2):

- The instruction in the MEM stage (`add`) is R-Type $\rightarrow$ `EX/MEM.IR[OPCODE]=R-Type`
- The instruction in the EX stage (`sub`) is R-Type $\rightarrow$ `ID/EX.IR[OPCODE]=R-Type`
- The destination register of the instruction in MEM (R1) is the same as the first source register of the instruction in EX (R1) $\rightarrow$ `EX/MEM.IR[RD]=ID/EX.IR[RS1]`

Every time we have this situation, we need to forward the result of the `add` instruction from the EX/MEM pipeline register to the first input of the ALU. Notice that the same situation would verify if the instruction following the `add` were any I-Type instruction using the loaded data as source operand (e.g. `addi R4,R1,#2`, or `lw R4,0(R1)`, or `sw 0(R1),R4`, or `beqz R1,label`, etc..), therefore we can generalize the condition of the type of instruction in the EX stage to be `ID/EX.IR[OPCODE]=R-Type` or `ID/EX.IR[OPCODE]=I-Type`. We will refer to this specific forwarding case as **F1**.

In our architecture, what happens is the following:

- the FORWARDING STALLING LOGIC detects this situation and raises the corresponding detection signal, which is sent to the Control Unit
- the Control Unit, upon receiving this signal, generates the appropriate control signal that acts as selection signal for the multiplexer at the first ALU input
- the multiplexer at the first ALU input, controlled by the signal coming from the Control Unit, forwards the data from the EX/MEM pipeline register to the ALU input A

Listing 6.1: VHDL code snippet of the FORWARDING STALLING LOGIC for the F1 forwarding case

```

if (
  — F1.
  (EX.MEM.OPCODE = RTYPE and (ID.EX.OPCODE = RTYPE or is_itype(
    ID.EX.OPCODE)) and EX.MEM.RD = ID.EX.RS1) or ...
) then

  FWD.ALU.to_A <= '1';

end if;

```

Listing 6.2: VHDL code snippet of the Control Unit handling F1 forwarding case

```

FWD.LOGIC : process (fwd)
begin
  ...
  — forward to A
  if (fwd(0) = '1') then
    FWD.A_SEL <= "10"; — A <= ALU_OUT_Q
  elsif (fwd(1) = '1') then
    ...
  end if;

  ...

```

`fwd` is the vector of signals generated by the FORWARDING STALLING LOGIC, and `fwd(0)` corresponds to `FWD.ALU.to_A`, which is the one telling the Control Unit: "Hey, we need to forward the output of the register EX/MEM.ALU_OUT to the ALU input A". The Control Unit, upon receiving this signal, generates the appropriate control signal `FWD.A_SEL` to select the correct forwarded data.

Figure 6.3 shows the 4-1 multiplexer at the ALU input A, which selects between 4 possible data

sources (the dashed lines are forwarding paths). It's easy to see that if `FWD_A_SEL="10"`, then the ALU input A will be assigned the output of `EX/MEM.ALU_OUT` register, namely `ALU_OUT_Q`. Other components and connections are omitted in the figure for better clarity.

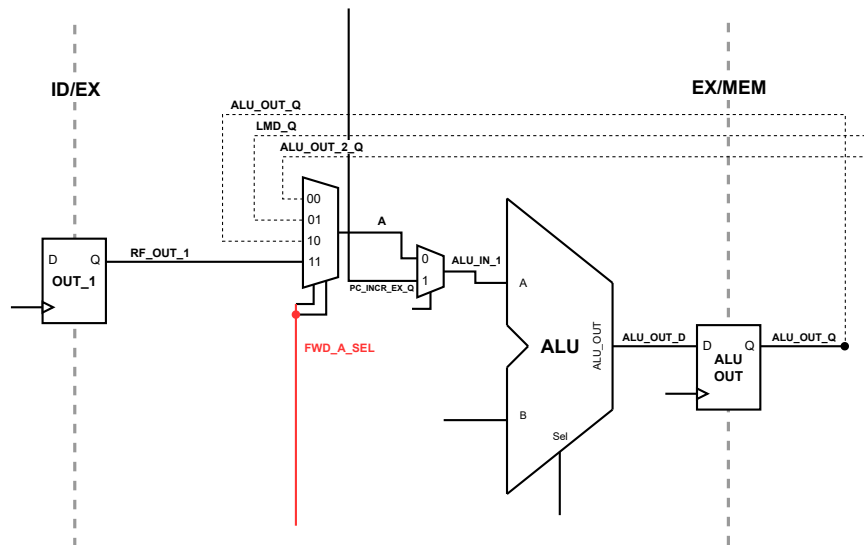


Figure 6.3: Multiplexer at ALU input A for data forwarding

The same mechanism applies to all other forwarding cases, that are thoroughly listed in Table A.1 in Appendix A. Each record of that table translates to a conditional statement in the FORWARDING STALLING LOGIC like the one shown in Listing 6.1, and a corresponding handling in the Control Unit like the one shown in Listing 6.2.

From Table A.1 it is clear that there are 4 possible forwarding destinations in our architecture:

- ALU input A
- ALU input B
- Data Memory Input Data
- Jump Register Target Address

The first one was already shown in Figure 6.3 and receives forwarded data from `EX/MEM.ALU_OUT`, `MEM/WB.ALU_OUT_2` (`MEM/WB` pipeline register storing the output of the ALU) and `MEM/WB.LMD` (`MEM/WB` pipeline register storing the loaded data from memory). The second one is similar, but refers to the multiplexer at ALU input B; it receives the same forwarded data as ALU input A. The third one refers to the multiplexer that selects the data to be written to Data Memory in case of store instructions, and it receives forwarded data from `MEM/WB.ALU_OUT_2`, `MEM/WB.LMD_Q` and `MEM/WB.PC_INCR_WB` (i.e `PC+4` of the current instruction in `WB`, needed when storing the Link Register into memory right after a `JAL` instruction, as it happens with subroutine calls). The fourth one refers to the multiplexer that selects the target address for Jump Register instructions and will be described in detail in the Control Hazards section. All of this is clearly depicted in the DLX schematic in Figure 2.4.

Stalling There are some cases where forwarding is not sufficient to resolve a RAW hazard, and stalling is still necessary. One such case occurs with load instructions followed by an instruction that uses the loaded data. For example:

```
lw R1, 0(R2)
add R3, R1, R4
```

This example is depicted in Figure 6.4. In this case, when the **add** instruction gets to the EX stage, the data from the **lw** instruction is still not available as it is being loaded from memory, so it can't be forwarded from anywhere.

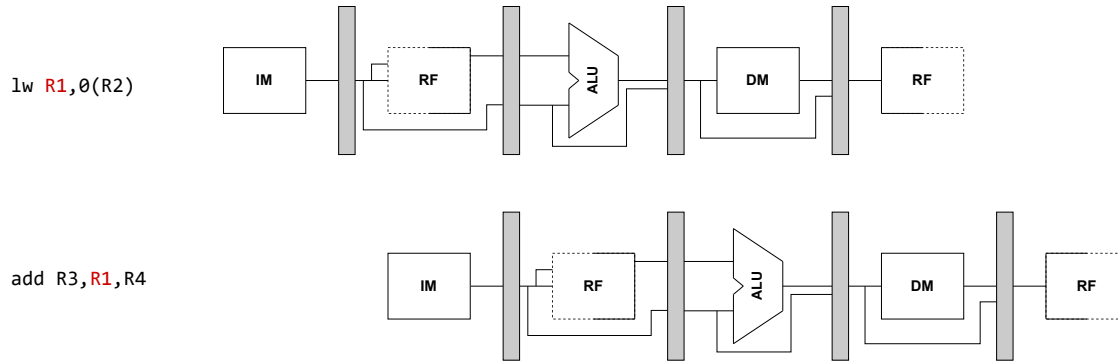


Figure 6.4: Example of a RAW hazard between two instructions

Therefore we need to stall the **add** instruction in the EX stage for 1 clock cycle, after which the **lw** instruction reaches the WB stage and the loaded data is available to be forwarded from MEM/WB.LMD register. The instruction flow in this situation is represented in Table 6.1, where other instructions are added to illustrate what happens with the Stall. The example is also visually depicted in Figure 6.5, where the red cloud indicates the stalled instruction. Notice that the same situation would verify if the instruction following the **lw** were any I-Type instruction using the loaded data as source operand (e.g. **addi** R3,R1,#2, or **lw** R3,0(R1), or **sw** 0(R1),R3, or **beqz** R1,label, etc..).

Instruction	1	2	3	4	5	6	7	8	9
lw R1,0(R2)	IF	ID	EX	MEM	WB				
add R3,R1,R4		IF	ID	EX	Stall	MEM	WB		
andi R10,R9,#2			IF	ID	Stall	EX	MEM		
sw 0(R11),R12				IF	Stall	ID	EX	MEM	WB
xori R14,R13,#5					Stall	IF	ID	EX	MEM

Table 6.1: Representation of instruction execution with stalling to resolve a RAW hazard

What happens behind the scenes is the following:

- the FORWARDING STALLING LOGIC detects this situation, raises a stall request signal and sends it to the Control Unit along with the number of cycles to stall
- the Control Unit, internally, has a *Mealy FSM* dedicated to Flush/Stall Control, as mentioned in Chapter 4, that upon receiving this stall request signal, disables the appropriate pipeline registers for the required number of cycles. In this case, it disables EX/MEM, ID/EX, IF/ID and WB/IF (i.e the PC) for 1 cycle, effectively stalling the **add** instruction in the EX stage for 1 cycle, as desired.

After the stall has terminated, in this case, the FORWARDING STALLING LOGIC will detect a situation requiring forwarding and the mechanism described in the previous paragraph will take place,

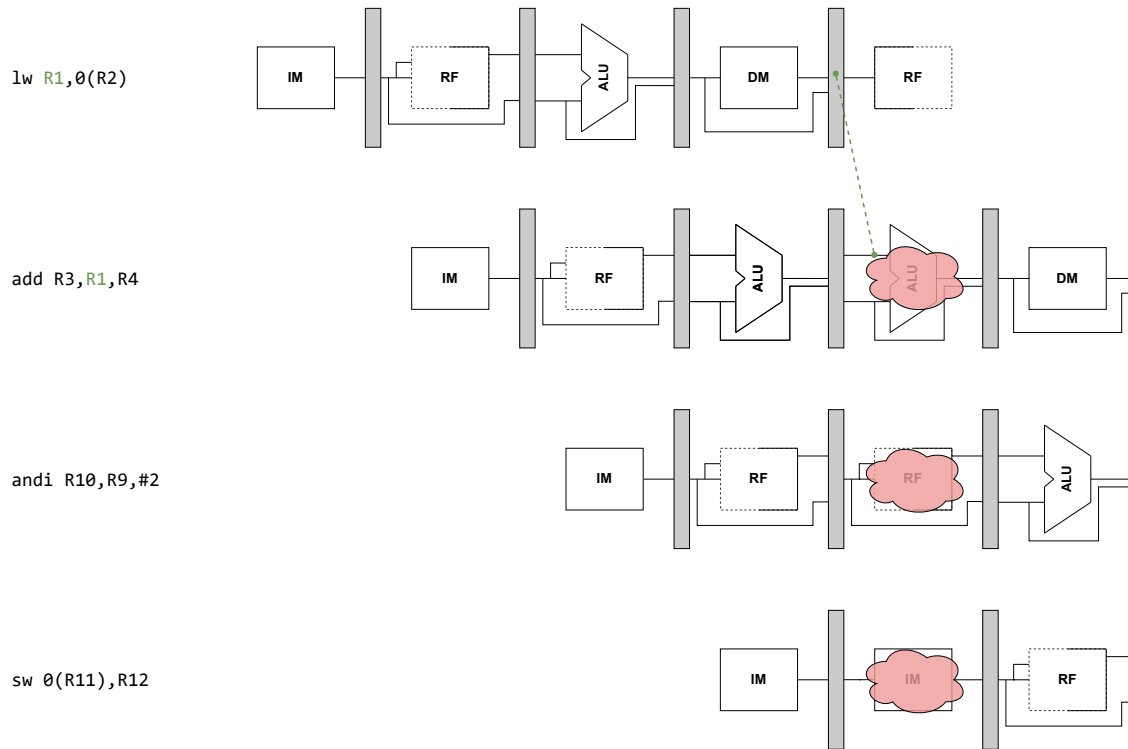


Figure 6.5: Graphical example of stalling to resolve a RAW hazard between a load and an ALU instruction

allowing the add instruction to use the correct value of R1. The detection mechanism is implemented in a totally similar way to the forwarding detection. This specific case (which we will refer to as **S1**) can be formalized as follows:

- The instruction in the MEM stage (lw) is Load $\rightarrow$ EX/MEM.IR[OPCODE]=Load
- The instruction in the EX stage (add) is R-Type or I-Type $\rightarrow$ ID/EX.IR[OPCODE]=R-Type or ID/EX.IR[OPCODE]=I-Type
- The destination register of the instruction in MEM (R1) is the same as the first source register of the instruction in EX (R1) $\rightarrow$ EX/MEM.IR[RS2]=ID/EX.IR[RS1]

In the code, this translates to:

Listing 6.3: VHDL code snippet of the FORWARDING STALLING LOGIC for the S1 stalling case

```

if (
  — S1
  (is_load(EX_MEM_OPCODE) and (ID_EX_OPCODE = RTYPE or is_itype(
    ID_EX_OPCODE)) and (EX_MEM_RS2 = ID_EX_RS1)) or ...
) then
  STALL_EX_MEM <= '1';
  n_cycles_stall <= 1;
elsif ...

```

Listing 6.4: VHDL code snippet of the Control Unit handling S1 stall case

```

FLUSH_STALL_STATE : process (Clk, Rst)
begin
    if (Rst = '1') then
        ...
        stallCounter <= 0;
        state <= WaitForReq;
    elsif (rising_edge(Clk)) then
        ...
        stallCounter <= nextStallCounter;
        state <= nextState;
    end if;
end process;

FLUSH_STALL_COMB_LOGIC : process (OPCODE,
    ...
    STALL_EX_MEM,
    n_cycles_stall,
    state,
    stallCounter,
    ...
)
begin

    — default behaviour
    enableSignals <= "11111"; — all pipeline registers enabled
    ...

    case state is

        when WaitForReq =>
            if (...) then
                ...
            elsif (STALL_EX_MEM = '1') then
                enableSignals <= "00001"; — all pipeline
                    registers up to EX/MEM are disabled
                nextStallCounter <= n_cycles_stall;
                nextState <= EXMEMStall;
            end if;
            ...

        ...

        when EXMEMStall =>
            if (stallCounter > 1) then
                nextStallCounter <= stallCounter - 1;
                enableSignals <= "00001"; — all pipeline
                    registers up to EX/MEM are disabled
            end if;
    end case;
end process;

```

```

        nextState      <= EXMEMStall;
    else
        nextState <= WaitForReq;
        ...
    end if;
end case;
end process;

```

In the above code snippet of the Control Unit, only the logic pertaining to stalling EX/MEM is shown for brevity, but the same kind of logic applies to stalling ID/EX. The rest of the logic of the FSM is to handle Data Cache misses and flushes to handle Control Hazards, as described later in this chapter.

The same mechanism applies to all other forwarding cases, that are thoroughly listed in Table A.2 in Appendix A. From this table it is clear that there are 2 categories of stall in our architecture:

- Stalls up to EX/MEM (i.e stalling EX, ID and IF stages)
- Stalls up to ID/EX (i.e stalling ID and IF stages)

Now we can fully describe the FORWARDING STALLING LOGIC as a component of the Datapath and the logic of the Control Unit dedicated to Data Hazard handling.

FORWARDING STALLING LOGIC

Description As previously mentioned, the FORWARDING STALLING LOGIC is a combinational logic block that continuously monitors the pipeline registers to detect data hazards. It checks for all possible scenarios of RAW hazards, and generates the appropriate signals that are sent to the Control Unit. These signals include forwarding control signals (to indicate which data needs to be forwarded and to where) and stall request signals (to indicate when a stall is necessary and for how many cycles). The FORWARDING STALLING LOGIC is crucial for ensuring that data hazards are detected and handled correctly, allowing the pipeline to operate efficiently without sacrificing correctness. Its schematic symbol is shown in Figure 6.6, in which all forwarding outputs are grouped in a single vector FWD for simplicity.

Input ports:

- IF_ID_IR : Instruction Register of IF/ID pipeline register (32-bit)
- ID_EX_IR : Instruction Register of ID/EX pipeline register (32-bit)
- EX_MEM_IR : Instruction Register of EX/MEM pipeline register (32-bit)
- MEM_WB_IR : Instruction Register of MEM/WB pipeline register (32-bit)

Output ports:

- FWD : Vector of forwarding signals (12-bit)
 - FWD_ALU_to_A : Forward from EX/MEM.ALU_OUT to ALU input A (1-bit)
 - FWD_ALU2_to_A : Forward from MEM/WB.ALU_OUT_2 to ALU input A (1-bit)
 - FWD_LMD_to_A : Forward from MEM/WB.LMD to ALU input A (1-bit)
 - FWD_ALU_to_B : Forward from EX/MEM.ALU_OUT to ALU input B (1-bit)
 - FWD_ALU2_to_B : Forward from MEM/WB.ALU_OUT_2 to ALU input B (1-bit)

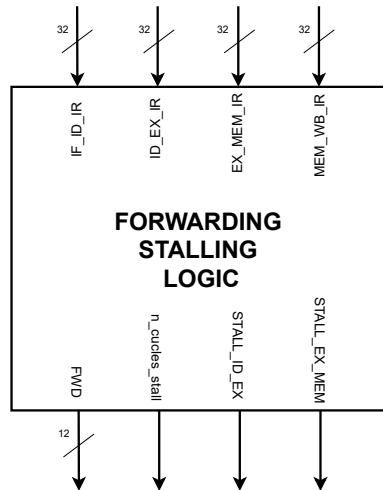


Figure 6.6: Schematic symbol of the FORWARDING STALLING LOGIC unit

- FWD_LMD_to_B : Forward from MEM/WB.LMD to ALU input B (1-bit)
- FWD_ALU2_to_DM : Forward from MEM/WB.ALU_OUT_2 to Data Memory Input Data (1-bit)
- FWD_LMD_to_DM : Forward from MEM/WB.LMD to Data Memory Input Data (1-bit)
- FWD_PC_INCR_WB_to_DM : Forward from MEM/WB.PC_INCR_WB to Data Memory Input Data (1-bit)
- FWD_ALU_to_JR : Forward from EX/MEM.ALU_OUT to Jump Register Target Address (1-bit)
- FWD_PC_INCR_EX_to_JR : Forward from ID/EX.PC_INCR_EX to Jump Register Target Address (1-bit)
- FWD_PC_INCR_MEM_to_JR : Forward from EX/MEM.PC_INCR_MEM to Jump Register Target Address (1-bit)
- STALL_EX_MEM : Stall request for EX/MEM stage (1-bit)
- STALL_ID_EX : Stall request for ID/EX stage (1-bit)
- n_cycles_stall : Number of cycles to stall (integer)

Integration in the Datapath The FORWARDING STALLING LOGIC is integrated into the Datapath as shown in Figure 2.4. The mapping between its ports and the Datapath signals is as follows:

- IF_ID_IR → IR_Q
- ID_EX_IR → ID_EX_IR_Q
- EX_MEM_IR → EX_MEM_IR_Q
- MEM_WB_IR → MEM_WB_IR_Q
- FWD_ALU_to_A → fwd(0)

- FWD_ALU2_to_A $\rightarrow$ fwd(1)
- FWD_LMD_to_A $\rightarrow$ fwd(2)
- FWD_ALU_to_B $\rightarrow$ fwd(3)
- FWD_ALU2_to_B $\rightarrow$ fwd(4)
- FWD_LMD_to_B $\rightarrow$ fwd(5)
- FWD_ALU2_to_DM $\rightarrow$ fwd(6)
- FWD_LMD_to_DM $\rightarrow$ fwd(7)
- FWD_PC_INCR_WB_to_DM $\rightarrow$ fwd(8)
- FWD_ALU_to_JR $\rightarrow$ fwd(9)
- FWD_PC_INCR_EX_to_JR $\rightarrow$ fwd(10)
- FWD_PC_INCR_MEM_to_JR $\rightarrow$ fwd(11)
- STALL_EX_MEM $\rightarrow$ STALL_EX_MEM
- STALL_ID_EX $\rightarrow$ STALL_ID_EX
- n_cycles_stall $\rightarrow$ n_cycles_stall

VHDL implementation The VHDL specification of this component can be found in `f-forwarding_stalling_logic.vhd`, located in the `vhdlsrc/a.a-datapath.core/` directory. The implementation follows a behavioral approach, clearly describing the logic behind the detection of data hazards and implementing each case listed in Table A.1 and Table A.2.

Control Unit

Description The Control Unit, as already mentioned, receives the signals generated by the FORWARDING STALLING LOGIC and, through dedicated processes, generates the necessary control signals to handle data hazards. Forwarding and stalling are treated separately:

- **Forwarding** signals are processed in the FWD.LOGIC process, which generates the appropriate control signals going to multiplexers in the Datapath to select the correct forwarded data (or the non-forwarded data) for the ALU inputs, Data Memory input data and Jump Register target address.

Listing 6.5: VHDL code snippet of the FWD.LOGIC process in the Control Unit

```
FWDLOGIC : process (fwd)
begin
    -- forward to A
    if (fwd(0) = '1') then
        FWD.A_SEL <= "10"; -- A <= ALU_OUT_Q
    elsif (fwd(1) = '1') then
        FWD.A_SEL <= "00"; -- A <= ALU_OUT_2_Q
    elsif (fwd(2) = '1') then
        FWD.A_SEL <= "01"; -- A <= LMD_Q
    else
```

```

        FWD.A_SEL <= "11"; -- A <= RF_OUT_1_Q
    end if;

    -- forward to B
    if (fwd(3) = '1') then
        FWD.B_SEL <= "11"; -- B <= ALU_OUT_Q
    elsif (fwd(4) = '1') then
        FWD.B_SEL <= "10"; -- B <= ALU_OUT_2_Q
    elsif (fwd(5) = '1') then
        FWD.B_SEL <= "01"; -- B <= LMD_Q
    else
        FWD.B_SEL <= "00"; -- B <= RF_OUT_2_Q
    end if;

    -- forward to DM_IN
    if (fwd(6) = '1') then
        FWD.DM_SEL <= "01"; -- DM_IN <= ALU_OUT_2_Q
    elsif (fwd(7) = '1') then
        FWD.DM_SEL <= "10"; -- DM_IN <= LMD_Q
    elsif (fwd(8) = '1') then
        FWD.DM_SEL <= "11"; -- DM_IN <= PC_INCR_WB_Q
    else
        FWD.DM_SEL <= "00"; -- DM_IN <= IN_DATA_Q
    end if;

    -- forward to JR address input
    if (fwd(10) = '1') then
        FWD.JR_SEL <= "11"; -- JR address input <= PC_INCR_EX_Q
    elsif (fwd(11) = '1') then
        FWD.JR_SEL <= "10"; -- JR address input <= PC_INCR_MEM_Q
    elsif (fwd(9) = '1') then
        FWD.JR_SEL <= "01"; -- JR address input <= ALU_OUT_Q
    else
        FWD.JR_SEL <= "00"; -- JR address input <= RF_OUT_1
    end if;
end process FWDLOGIC;

```

The order of the **if-elsif-else** statements is important here, as it establishes a priority for forwarding sources. For example, if both **FWD_ALU_to_A** and **FWD_ALU2_to_A** are asserted, the former will take precedence, as it accounts for the most recent data dependency.

- **Stall** signals are processed in the **FLUSH_STALL_STATE** and **FLUSH_STALL_COMB_LOGIC** processes, that together implement a High-level Mealy FSM to control the overall stalling and flushing of the pipeline. Upon receiving a stall request signal from the **FORWARDING STALLING LOGIC**, the FSM disables the appropriate pipeline registers for the required number of cycles, effectively stalling the necessary stages of the pipeline.

Refer to **a.b-CU.vhd** in the **vhdlsim/src/** directory for the full VHDL implementation of the Control Unit, including the processes described above.

6.3 Control Hazards

Control hazards occur when the processor doesn't yet know which instruction should be fetched next, for the current instruction is a branch (conditional or unconditional) whose outcome isn't known immediately. This happens because in a pipeline, instructions are fetched before previous ones have completed, so when a branch is encountered, the processor doesn't know whether to continue fetching the next sequential instruction or to jump to a different address. That uncertainty is the control hazard.

6.3.1 Handling Control Hazards

In our design, we made a progressive improvement of the control hazard handling mechanism, starting from a simple approach and then refining it to achieve better performance. We will quickly describe the initial approaches, then thoroughly explain the final one, highlighting the improvements achieved, and we'll explain how is the pipeline **Flushed** to discard incorrect instructions fetched after a conditional or unconditional branch (we will refer to the first type as **branches** and to the second type as **jumps** for simplicity).

Initial Approach Initially, branches were dealt with in the following way:

- the condition evaluation was done in the EX stage
- the address calculation was done in the EX stage
- both condition evaluation result and address were used in the MEM stage, which meant that if a branch was taken, the correct instruction would be fetched only when the branch instruction reached WB stage
- since the model used for prediction was *predict untaken*, we had:
 - a penalty of 0 clock cycles for untaken branches (Table 6.2)
 - a penalty of 3 clock cycles for taken branches (Table 6.3)

Instruction	1	2	3	4	5	6	7	8	9
untaken branch instruction i	IF	ID	EX	MEM	WB				
instruction i+1		IF	ID	EX	MEM	WB			
instruction i+2			IF	ID	EX	MEM	WB		
instruction i+3				IF	ID	EX	MEM	WB	

Table 6.2: Initial approach: untaken branch (no penalty)

Instruction	1	2	3	4	5	6	7	8	9
taken branch instruction i	IF	ID	EX	MEM	WB				
instruction i+1		IF	ID	EX	Stall				
instruction i+2			IF	ID	EX	Stall			
instruction i+3				IF	ID	EX	Stall		
target instruction					IF	ID	EX	MEM	WB

Table 6.3: Initial approach: taken branch (3-cycle penalty)

In this case the **Stall** is implemented differently than for data hazards and it effectively implements a **Flush**: as it can be seen in Table 6.3, when instruction $i+1$ reaches the EX stage, it is blocked there and doesn't proceed to MEM stage; however, at the same time, the previous stages are NOT stalled (all other pipeline registers are enabled), meaning that subsequent instructions continue to progress in the pipeline and to enter the EX stage, overwriting the previous instruction, thus erasing it completely. This goes on for 3 cycles, after which the target instruction (correct instruction to be executed after the branch) reaches the EX stage (overwriting instruction $i+3$). At this point the EX/MEM pipeline register is re-enabled, allowing the target instruction to proceed through the pipeline. As a result, all the incorrect instructions that were fetched after the branch instructions are effectively discarded, as they never reach the MEM stage or WB stage, so they can't change the state of the processor. This mechanism is called **flushing** the pipeline. **Jump** instructions were handled in a similar way: the target address was computed in the EX stage, and used in the MEM stage, so the next three instructions needed to be flushed as well, as shown in Table 6.4.

Instruction	1	2	3	4	5	6	7	8	9
jump instruction i	IF	ID	EX	MEM	WB				
instruction $i+1$		IF	ID	EX	Stall				
instruction $i+2$			IF	ID	EX	Stall			
instruction $i+3$				IF	ID	EX	Stall		
target instruction					IF	ID	EX	MEM	WB

Table 6.4: Initial approach: jump (3-cycle penalty)

This approach was simple to implement but had very poor performance results, for all taken branches and jumps caused a penalty of 3 clock cycles, which becomes quite significant for programs with many branches, especially for those containing loops with many iterations. To achieve better performance, the following improvements were made progressively:

1. use branch evaluation outcome and branch target address as soon as they are computed in the EX stage, which results in a penalty of 2 clock cycles for taken branches and jumps
2. move the address calculation in the ID stage, by inserting an adder that computes the branch target address as $PC + 4 + \text{immediate}$; this results in a penalty of only 1 clock cycle for jumps, but no improvement for taken branches as they are still evaluated in the EX stage
3. implement a dynamic branch prediction mechanism, using a **Branch History Table (BHT)**

With these improvements, the final handling of control hazards in our architecture is the following.

Implementation

Jumps Jumps are always taken, so the target address is computed in the ID stage and used immediately to fetch the correct instruction in the next cycle. However, since the instruction after the jump is already fetched in the IF stage when the jump is in ID stage, that instruction needs to be flushed. This results in a penalty of 1 clock cycle for jumps, as shown in Table 6.5. When a jump

Instruction	1	2	3	4	5	6	7
jump instruction i	IF	ID	EX	MEM	WB		
instruction $i+1$		IF	Stall				
target instruction			IF	ID	EX	MEM	WB

Table 6.5: Final approach: jump (1-cycle penalty)

enters the ID stage, the following actions occur (refer to Figure 2.4 for signal names and paths):

- control signal **SES** (Sign Extend Select) from the Control Unit is set to 1, therefore selecting **EXT_IMM_J** - the sign extended immediate field **IMM_J** (**IR[25:0]**) of the jump instruction - to become signal **EXT_IMM**
- an adder sums up **PC+4** (from **IF/ID.PC_INCR_ID**) with (**EXT_IMM**) to compute the target address
- the **JUMPS** signal from the Control Unit is used to select the jump target address between the one just computed (**JUMPS=0**, for normal jumps) and the one coming from the Register File or forwarded from elsewhere (**JUMPS=1**, for Jump Register instructions). The resulting signal is **TARGET_ADDR**.
- the **FLUSH/STALL** FSM of the Control Unit detects that the current instruction in ID is a jump, so it disables the IF/ID pipeline register (to flush the incorrect instruction fetched after the jump), raises the **JUMP** control signal and sets its next state to **JumpState**
- signal **TARGET_ADDR** directly feeds the multiplexer that chooses between this signal and **PC + 4** (**PC_INCR**) to become the NPC. The multiplexer is controlled by signal **JB**, obtained as **JB = JUMP or BRANCH**, so in this case **JB=1** and the selected signal is **TARGET_ADDR**, outputted from the mux as signal **INSTR_ADDR**
- if no branch correction is taking place in this cycle, **INSTR_ADDR** becomes the NPC (Next Program Counter), the signal that directly feeds the input of the PC register

In the next clock cycle:

- the target instruction enters the IF stage, thereby overwriting the incorrect instruction that was fetched after the jump (flushing it)
- the jump instruction moves to the EX stage and continues its normal execution through the pipeline; however, it is duplicated in the ID stage (as the next one was flushed), so the same sequence of actions could be repeated, leading to incorrect behavior. This doesn't happen because right now the **FLUSH/STALL** FSM is in the **JumpState**, so it ignores the current jump instruction in ID. The next state is set back to **WaitForReq**, so that normal operation resumes in the following cycle.

Conditional Branches For conditional branches, the story is much more complicated. The complexity lies in the implementation of the dynamic branch prediction mechanism, which requires handling both the predictions and the correction of mispredictions, without messing up the mechanisms already in place for the other undeterministic situations (data hazards, cache misses, jumps).

The dynamic branch prediction mechanism is based on a 16-entry **Branch History Table (BHT)**, which is a small memory that stores the history of branch outcomes to predict future behavior. The BHT is addressed with the 4 LSBs of branch instructions (excluding the 2 LSBs of alignment) and each entry in the BHT contains a 2-bit saturating counter that indicates the prediction of the corresponding branch being taken or not taken, following the scheme shown in Figure 6.7. Its symbol representation is shown in Figure 6.8.

Input ports:

- **Clk** : clock signal (1-bit)
- **Rst** : reset signal (1-bit)

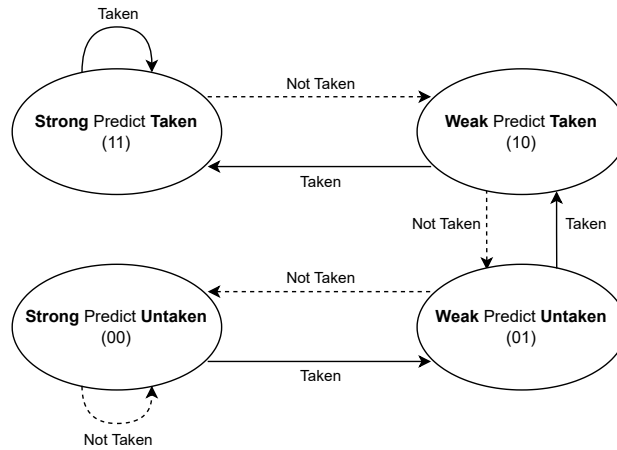


Figure 6.7: States of the 2-bit saturating counter used in the Branch History Table (BHT)

- **PC_1** : program counter 1 (32-bit)
- **PC_2** : program counter 2 (32-bit)
- **Taken** : branch taken (1-bit)
- **Update**: update signal for the BHT (1-bit)

Output ports:

- **Prediction_1** : branch prediction 1 (1-bit)
- **Prediction_2** : branch prediction 2 (1-bit)

Function performed:

- When **Rst** is asserted, the BHT is initialized to a default state (all counters set to '01' - weak predict untaken)
- the 4 LSBs (excluding the 2 LSBs of alignment) of **PC_1** and **PC_2** are used to index the BHT and retrieve the corresponding 2-bit saturating counters, which provide the predictions **Prediction_1** and **Prediction_2**. A prediction of '1' indicates that the branch is predicted to be taken, while '0' indicates not taken.
- when the **Update** signal is asserted, the BHT entry corresponding to **PC_2** is updated (on the **Clk** rising edge) based on the actual outcome of the branch (**Taken** signal)

The VHDL implementation of this component can be found in `c-bht.vhd`, located in the `vhdlsrc/src/a.a-datapath.core/` directory. The implementation follows a behavioral approach, clearly describing the logic behind the BHT operations.

The BHT is integrated into the Datapath as shown in Figure 2.4. The mapping between its ports and the Datapath signals is as follows:

- **Clk** → **Clk**
- **Rst** → **Rst**

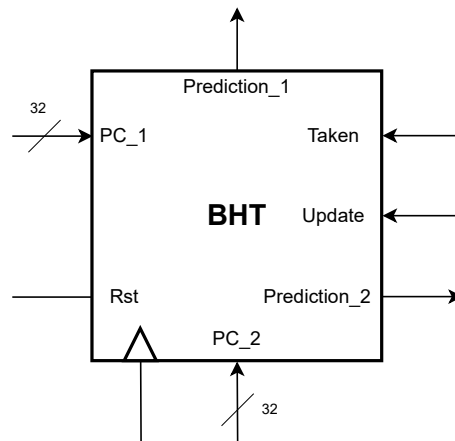


Figure 6.8: Symbolic representation of the Branch History Table (BHT)

- $PC_1 \rightarrow PC_ID_Q$
- $PC_2 \rightarrow PC_EX_Q$
- $Taken \rightarrow COND$
- $Update \rightarrow CNDS[0] \text{ and } not(NPC_SEL_RST)$
- $Prediction_1 \rightarrow PREDICTION_1$
- $Prediction_2 \rightarrow PREDICTION_2$

When a branch enters the **ID** stage, the following actions occur (refer to Figure 2.4 for signal names and paths):

- control signal **SES** (Sign Extend Select) from the Control Unit is set to 0, therefore selecting **EXT_IMM_I** - that is the sign extended immediate field **IMM_I** (**IR[15:0]**) of the branch instruction - to become signal **EXT_IMM**
- an adder sums up **PC+4** (from **IF/ID.PC_INCR_ID**) with (**EXT_IMM**) to compute the branch target address
- the BHT is accessed with **PC_ID_Q** (the PC of the branch instruction in ID stage) to retrieve **PREDICTION_1**, which indicates whether the branch is predicted taken or not taken.
- **PREDICTION_1** is fetched to the Control Unit which, in the **FLUSH/STALL** FSM, also checks for the presence of branch instructions in ID and for the prediction made by the BHT. If the instruction is not a branch or **PREDICTION_1=0** (predicted not taken), no action is taken and normal operation continues. If instead the instruction is a branch and **PREDICTION=1** (predicted taken):
 - the FSM disables the IF/ID pipeline registers just as it does with jumps (to flush the incorrect instruction fetched after the branch), raises the **BRANCH** control signal and sets its next state to **BranchState**.

- **BRANCH=1** drives JB high, which selects the computed target address (**TARGET_ADDR**) as the **INSTR_ADDR**, which in turn becomes the NPC if no branch correction is taking place in this cycle
 - in the **next clock cycle**, the assumed-to-be target instruction enters the IF stage, thereby overwriting the incorrect instruction that was fetched after the branch (flushing it)
 - the branch instruction moves to the EX stage and continues its normal execution through the pipeline; however, it is duplicated in the ID stage (as the next one was flushed), so the same sequence of actions could be repeated, leading to incorrect behavior. This doesn't happen because right now the **FLUSH/STALL** FSM is in the **BranchState**, so it ignores the current branch instruction in ID. The next state is set back to **WaitForReq**, so that normal operation resumes in the following cycle.
- in the **EX** stage, the branch condition is evaluated and the actual outcome (taken or not taken) is determined and carried by signal **COND** (**COND=0** if not taken, **COND=1** if taken).
 - the BHT will be updated based on the actual outcome (**COND**)
 - The BHT is accessed again with **PC_EX_Q** (the PC of the branch instruction in EX stage) to retrieve **PREDICTION_2**, which is the same as **PREDICTION_1** was 1 cycle before (two ports are used because another branch instruction could enter ID stage while the previous one is still in EX stage). If the actual outcome (**COND**) and the prediction (**PREDICTION_2**) don't match, it means a misprediction has occurred (**DIFF** is set to 1) and a branch correction is necessary. To be precise, **DIFF** only has meaning if the current instruction in EX is a branch; that is if **CNDS[0]=1**. Therefore, the actual signal used to detect mispredictions is **MISS = DIFF and CNDS[0]**.
 - if a misprediction has not occurred, **MISS=0** and normal operation continues.
 - if a misprediction has occurred:
 - the two instructions fetched after the branch are incorrect and need to be flushed. The **FLUSH/STALL** FSM detects the misprediction through signal **FLUSH_REQ** (**NPC_SEL** in the Datapath, that is equivalent to **MISS and not(NPC_SEL_RST)**), immediately disables the ID/EX pipeline register and sets the next state to **FixBranchPred**
 - at the same time, **NPC_SEL** selects **CORRECTED_INSTR_ADDR** as the NPC, which is the correct next instruction address, selected by signal **COND**. That is either **PC + 4** (**PC_INCR_EX_Q**) if the branch was actually not taken, or the computed target address (**ALU_OUT_D**) if it was taken.
 - in the **next clock cycle**, the correct instruction enters the IF stage.
 - meanwhile, the second wrong instruction enters ID stage, so ID/EX must be still disabled for another cycle to flush it as well. At the same time, signal **NPC_SEL_RST** is raised by the control unit to prevent any further branch corrections from interfering while the current one is being resolved and to prevent from erroneously updating the BHT again.
 - the FSM sets the next state to **FixBranchPred** again
 - in the **next clock cycle**, the correct instruction moves to ID stage, so now ID/EX must be re-enabled to allow normal operation to resume. **NPC_SEL_RST** is still high for the same reasons as before. The next state is set back to **WaitForReq**.
 - finally, in the following clock cycle the correct instruction moves to EX stage and normal operation is fully restored.

Performance Results The results achieved with this approach are the following:

- prediction: **not taken**, outcome: **not taken** → 0 cc penalty (Table 6.6)
- prediction: **taken**, outcome: **taken** → 1 cc penalty (Table 6.7)
- prediction: **not taken**, outcome: **taken** → 2 cc penalty (Table 6.8)
- prediction: **taken**, outcome: **not taken** → 2 cc penalty (Table 6.9)

Instruction	1	2	3	4	5	6	7	8
branch instruction i	IF	ID	EX	MEM	WB			
instruction i+1		IF	ID	EX	MEM	WB		
instruction i+2			IF	ID	EX	MEM	WB	
instruction i+3				IF	ID	EX	MEM	WB

Table 6.6: Prediction: not taken, Outcome: not taken (0-cycle penalty)

Instruction	1	2	3	4	5	6	7	8
branch instruction i	IF	ID	EX	MEM	WB			
instruction i+1		IF	Stall					
branch target instruction			IF	ID	EX	MEM	WB	

Table 6.7: Prediction: taken, Outcome: taken (1-cycle penalty)

Instruction	1	2	3	4	5	6	7	8
branch instruction i	IF	ID	EX	MEM	WB			
instruction i+1		IF	ID	Stall				
instruction i+2			IF	ID	Stall			
branch target instruction				IF	ID	EX	MEM	WB

Table 6.8: Prediction: not taken, Outcome: taken (2-cycle penalty)

Instruction	1	2	3	4	5	6	7	8
branch instruction i	IF	ID	EX	MEM	WB			
instruction i+1		IF	Stall	IF	ID	EX	MEM	WB
instruction i+2					IF	ID	EX	MEM
instruction i+3						IF	ID	EX
branch target instruction			IF	Stall				

Table 6.9: Prediction: taken, Outcome: not taken (2-cycle penalty)

To appreciate the improvement achieved with this final approach with respect to the initial one, consider a program with 20% branches, of which 60% are taken. With the initial approach (3-cycle penalty for all taken branches), the average CPI would be:

$$CPI_{initial} = 1 + 0.2 \cdot 0.6 \cdot 3 = 1.36$$

With the final approach (2-cycle penalty for mispredicted branches, 0-cycle penalty for correctly predicted untaken branches, 1-cycle penalty for correctly predicted taken branches), assuming a prediction

accuracy of 80%, the average CPI would be:

$$CPI_{final} = 1 + 0.2 \cdot [0.6 \cdot (0.2 \cdot 2 + 0.8 \cdot 1) + 0.4 \cdot 0.2 \cdot 2] = 1.18$$

In a program with 1 million instructions, this results in a reduction of the total number of clock cycles from 1,360,000 to 1,180,000, that with a clock period of 2 ns translates to a time saving of 0.36 ms.

CHAPTER 7

Simulation and Verification

7.1 Overview

In this chapter, we will present the results of our DLX microprocessor simulation and the verification process we undertook to ensure its correctness and performance. We will discuss the various test cases used to verify the functionality of the processor, including both unit tests for individual components and integration tests for the entire system.

7.2 Unit tests

To validate the functionality of individual components of the DLX microprocessor, we developed a series of unit tests. Each component was tested in isolation to ensure that it behaved as expected according to its specifications. The unit tests are present in the `vhdlsim/testbench/` directory of the project, containing testbenches for every single component of the microprocessor. Notice that some testbenches may be obsolete, as they were used during the initial development phase and later replaced by more comprehensive integration tests.

7.3 Integration tests

After validating individual components, we proceeded to integration testing, where we tested progressively complex blocks up until the entire microprocessor system. The fully system test was performed in three main phases:

- First, by directly inserting instructions into the datapath with the testbench driving the IR input port and making use of **debug ports**. This phase was focused on observing the most significant waveforms step by step to ensure that the control signals generated by the Control Unit were correct and that the datapath components interacted as expected. Branches and jumps couldn't be properly tested, as the PC was not used.
- Next, by loading complete programs into the instruction memory and allowing the microprocessor to fetch and execute them autonomously. This phase allowed us to test the full functionality of the microprocessor, observing the waveforms of the problematic signals when needed.
- Finally, using a non-transparent DLX component with no debug ports exposed. The only way to interact with the simulation was to load programs into the instruction memory and read the contents of the data memory, data cache, register file and BHT after execution. This phase

was crucial for validating the microprocessor's behavior in a more realistic scenario, as it closely resembled how the processor would operate in a real-world application. This final step allowed us to perform rapid tests without the overhead of waveform analysis, enabling us to focus on the correctness of the final results. A set of assembly test programs was developed to cover a wide range of scenarios, trying to account for all possible edge cases. The following section describes these programs in detail.

The testbench used for testing the full microprocessor system is `TB_dlx.vhd`, located in the `vhdlsim/testbench/` directory. This testbench instantiates the DLX microprocessor, the Instruction Memory and Data Memory, connecting them together, provides the necessary clock and reset signals, manages the loading of code into memories before starting the simulation and logging of memory contents into output files during simulation. The schematic of this testbench is shown in Figure 7.1.

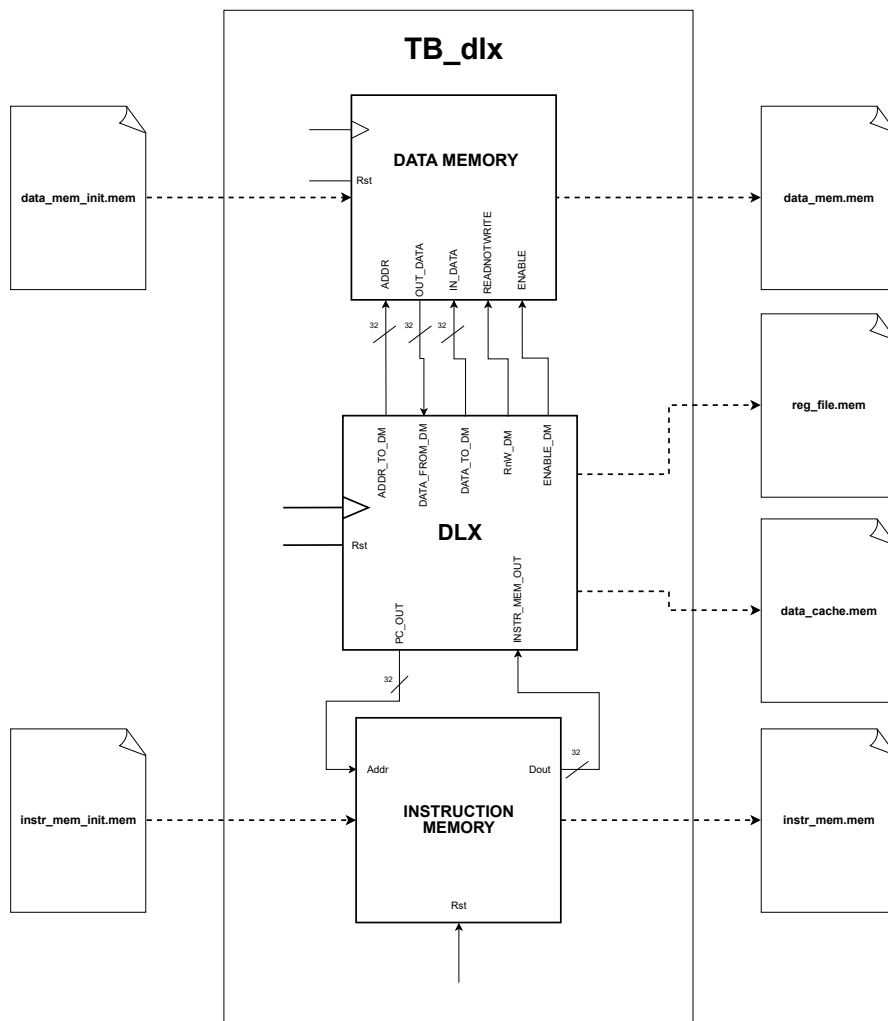


Figure 7.1: Schematic of the TB_dlx testbench used for full microprocessor system testing.

7.4 Testing using assembly programs

Inside the `vhdlsim/asm` directory of the project, one can find:

- The `perl` assembler script `dlxasm.pl`, generating the executable binary file from the assembly source code.
- The linker bash script `build.sh`, calling the `perl` assembler and generating the instruction memory and data memory initialization files. They are written in HEX format, as the procedures inside the memories read HEX values.
- The programs folder, containing all the assembly test programs. Every file contains at the end a comment specifying the expected final values in the data memory and register file after execution.
- The output folder, containing the files generated by the previous scripts starting from the programs

The memory initialization files `instr_mem_init.mem` and `data_mem_init.mem` are directly produced into the `vhdlsim/memory_files` directory, where also the memories output files `instr_mem.mem`, `data_mem.mem`, `data_cache.mem` and `reg_file.mem` will be generated during simulation.

7.4.1 Running a simulation

To run a simulation with a specific assembly program, it is enough to move to the `vhdlsim/` directory and execute the script

```
./simulate.sh <program_name>
```

This script will in turn call the `Makefile` that executes the following steps:

- Compiles all VHDL source code (if not already done)
- Calls the `build.sh` script to assemble the specified assembly program and load executable code into memory initialization files
- runs the QuestaSim simulation with the `TB_dlx.vhd` testbench

At the end of the simulation, the memory output files can be viewed in the `vhdlsim/memory_files` directory, containing the final contents of the instruction memory, data memory, data cache and register file.

As already mentioned, a set of assembly test programs was developed to validate the microprocessor's functionality. This paragraph describes the most significant test programs used during the verification process, as well as the properties verified by each of them.

7.4.2 ALU operations test

The following programs test the ALU operations implemented in the microprocessor. They are very simple assembly programs, each one focusing on a single ALU operation. The expected results are verified by checking the contents of specific registers in the register file after execution:

- **addu.asm**: tests the addition of two unsigned integers.
- **addui.asm**: tests the addition of an unsigned integer and an unsigned immediate value.
- **subu.asm**: tests the subtraction of two unsigned integers.
- **subui.asm**: tests the subtraction of an unsigned immediate value from an unsigned integer.
- **seq.asm**: tests the equality comparison between two integers.

- **seqi.asm**: tests the equality comparison between an integer and an immediate value.
- **sgeu.asm**: tests the greater than or equal comparison between two unsigned integers.
- **sgeui.asm**: tests the greater than or equal comparison between an unsigned integer and an unsigned immediate value.
- **sgt.asm**: tests the greater than comparison between two signed integers.
- **sgti.asm**: tests the greater than comparison between a signed integer and an immediate value.
- **sleu.asm**: tests the less than or equal comparison between two unsigned integers.
- **sleui.asm**: tests the less than or equal comparison between an unsigned integer and an unsigned immediate value.
- **slt.asm**: tests the less than comparison between two signed integers.
- **slti.asm**: tests the less than comparison between a signed integer and an immediate value.
- **sltu.asm**: tests the less than comparison between two unsigned integers.
- **sltui.asm**: tests the less than comparison between an unsigned integer and an unsigned immediate value.
- **sra.asm**: tests the arithmetic right shift operation.
- **srai.asm**: tests the arithmetic right shift operation with an immediate value.
- **xor_swap.asm**: tests the XOR operation by swapping two registers using XOR.

Verified properties:

- P4 Adder correct behaviour in addition operations.
- Comparator based on adder flags correct behaviour in comparison operations.
- Logicals correct behaviour in logical operations.
- Proper handling of signed and unsigned integers in comparison operations.
- Proper handling of immediate values in ALU operations.

7.4.3 Jump instructions test

The following programs test the jump instructions implemented in the microprocessor:

- **j.asm**: tests the unconditional jump instruction.
- **jal.asm**: tests the jump and link instruction.
- **jr.asm**: tests the jump register instruction.
- **jalr.asm**: tests the jump and link register instruction.
- **procedures_test.asm**: tests the use of jal and jr instructions to call and return from procedures.

Verified properties:

- Correct calculation of target addresses for jump instructions.
- Proper updating of the PC for jump instructions.
- Correct flushing of the pipeline on jump instructions.
- Correct behaviour of procedure calls and returns.

7.4.4 Branch instructions test

The following programs test the jump instructions implemented in the microprocessor. These programs were tested before implementing branch prediction, so the focus was on verifying the correct evaluation of branch conditions and the proper updating of the PC and pipeline flushing when branches were taken or not taken.

- **branch_1.asm**: tests a simple while loop using the beqz instruction.
- **branch_2.asm**: tests a simple while loop using the bnez instruction.
- **branch_3.asm**: tests two different flows of execution based on the value of r1.

Verified properties:

- Correct evaluation of branch conditions.
- Correct calculation of target addresses for branch instructions.
- Proper updating of the PC for branch instructions.
- Correct flushing of the pipeline on branch instructions.

7.4.5 Data hazards test

The following programs test the handling of data hazards in the microprocessor. In particular, they focus on the forwarding logic implemented in the datapath, as well as the stall mechanism for hazards that cannot be resolved through forwarding:

- **jr_fwd_18.asm**: tests the correct handling of forwarding case #18.
- **jr_fwd_19.asm**: tests the correct handling of forwarding case #19.
- **jr_fwd_20.asm**: tests the correct handling of forwarding case #20.
- **jr_fwd_21.asm**: tests the correct handling of forwarding case #21.
- **jr_stall_3_4.asm**: tests the correct handling of stalling cases #3 and #4.

Verified properties:

- Correct detection of data hazards.
- Proper forwarding of data between pipeline stages.
- Correct stalling of the pipeline for data hazards that cannot be resolved through forwarding.

7.4.6 Cache functionality test

The following programs test the correct functionality of the data cache:

- **cache_test.asm**: composed of many store instructions accessing subsequent memory words, testing the writing of data into the cache at the right lines and word offsets.
- **cache_test_2.asm**: composed of many load instructions accessing different cache lines twice. It tests the triggering of a cache miss on the first access (where the cache loads data from memory due to the reset_bit set) and a cache hit on the second access.
- **cache_test_3.asm**: miscellaneous cache test program, described in section 5.3.2.

Verified properties:

- Cache is accessed at the correct line and word offset based on the address.
- Cache miss is correctly triggered when accessing a line with the `reset_bit` set.
- Cache hit is correctly detected when accessing a line with a valid TAG.
- Cache misses when the `dirty_bit` is set trigger a write-back to memory before loading new data into the cache line.
- Cache misses when the `dirty_bit` is not set directly load new data into the cache line without writing back to memory.

7.4.7 Load and store instructions variations test

The assembly program `memory_rw.asm` tests all the possible variations of load and store instructions, accessing bytes, half-words and words, both signed and unsigned. First, some stores are performed to write specific values into memory. Then, loads are executed to read back the values from memory into registers, verifying that the data is correctly sign-extended or zero-extended based on the instruction used.

Verified properties:

- Load and store instructions correctly access data cache and memory.
- Data is correctly sign-extended or zero-extended based on the instruction used.
- Byte, half-word, and word accesses work correctly.

7.4.8 Stack operations test

The following programs test the correct functionality of stack operations in the microprocessor. There are no dedicated stack instructions, so the stack is managed using standard load and store instructions with register `r30` used as the stack pointer:

- **stack.asm**: example program performing push and pop operations on the stack.
- **stack_2.asm**: another example program testing stack operations.

Verified properties:

- Stack pointer (`r30`) is correctly updated on push and pop operations.
- Data is correctly written to and read from the stack memory locations.
- `jal` and `jr` instructions work correctly even with data dependencies.

7.4.9 Branch prediction test

The following programs test the correct functionality of branch prediction in the microprocessor. They focus on verifying that the BHT is updated correctly based on branch outcomes and that the pipeline is flushed appropriately on mispredictions:

- **branch_prediction.asm**: program involving multiple branches to test the branch prediction unit. The correct behaviour is verifiable through the waveforms and the BHT content dump file.

Verified properties:

- Stack pointer (`r30`) is correctly updated on push and pop operations.
- Data is correctly written to and read from the stack memory locations.
- `jal` and `jr` instructions work correctly even with data dependencies.

7.4.10 Complex programs test

These programs combine multiple aspects of the microprocessor's functionality to test its overall performance and correctness.

Fibonacci

`fibonacci.asm` is an assembly program that calculates Fibonacci series using a recursive approach. Registers `r1` and `r2` contain the two values to be summed, to obtain the next Fibonacci number. Register `r3` contains the position in the Fibonacci sequence to be calculated. Finally, register `r4` contains the result of the computation. Other registers are used as temporary storage during the recursive calls.

Verified properties:

- ALU correct behaviour in addition and comparison operations.
- Branch and jump instructions working properly.

Quick sort

`quick_sort.asm` is an assembly program that implements the quick sort algorithm to sort an array of integers stored in memory. The program is very complex and tests basically every feature of the DLX processor.

The content of the array to be sorted is initialized in the data memory before execution. At the first iterations of the algorithm, the data cache is filled with the values of the array, triggering multiple cache misses (because of the reset bits). Then the sorting algorithm is executed, involving numerous ALU operations with high data dependency, branch and jump instructions, as well as load and store operations (including stack operations).

Branch prediction is also tested, as the algorithm involves many branches whose outcomes are not easily predictable. The final result is verified by checking the contents of the data cache after execution.

To facilitate testing, `quick_sort_debug.asm` is also provided. This version of the program performs a final write-back of the sorted array from the data cache to the data memory, allowing for easier verification of the sorted result.

Verified properties:

- ALU correct behaviour.
- Branches and jumps working properly.
- Branch prediction working properly.
- Flush and stall mechanisms functioning correctly.
- Forwarding logic working correctly.
- Data cache functioning correctly during load and store operations.

CHAPTER 8

Synthesis and Physical Design

In this chapter, we describe the synthesis and physical design process of the DLX microprocessor. We first discuss the transformation of the VHDL description into a gate-level netlist, detailing the tools and optimization techniques employed to improve performance and minimize area. Subsequently, we address physical design aspects such as floorplanning, placement, and routing, highlighting their impact on timing, area, and power consumption. Finally, we present the synthesis and layout results, including timing analysis and resource utilization metrics.

8.1 Synthesis

8.1.1 Overview

The synthesis of the DLX processor is driven by the `DLX.t.scr` script, which automates the flow within Synopsys Design Vision. This script, written in the TCL-like command language of Design Vision, handles the analysis of VHDL source files, design elaboration, constraint definition, and synthesis optimization. It also manages report generation and the export of the final gate-level netlists.

8.1.2 Full script listing

Listing 8.1: Contents of `DLX.t.scr`

```
#####
# SCRIPT FOR SPEEDING UP AND RECORDING THE DLX SYNTHESIS
# Analyzing and checking VHDL netlist
# The analyze command is used for each file from bottom to top
#####
# Analyze custom types, constants and functions
analyze -library WORK -format vhd1 { ./src/000-globals.vhd }

# Analyze DATAPATH
analyze -library WORK -format vhd1 { ./src/a.a-datapath.core/02.a.a.a-ND2_1bit.vhd }
analyze -library WORK -format vhd1 { ./src/a.a-datapath.core/02.a.a.b-IV_1bit.vhd }
---
analyze -library WORK -format vhd1 { ./src/a.a-datapath.vhd }

# Analyze CONTROL UNIT
analyze -library WORK -format vhd1 { ./src/a.b-CU.vhd }
```



```

# Analyze DLX top-level entity
analyze -library WORK -format vhd1 { ./src/a-dlx.vhd }

#####
# Elaborating the top entity -- here supposed DLX
# Choose the architecture you want
elaborate DLX -architecture STRUCTURAL
#####
set_wire_load_model -name 5K_hvration_1_4

# Compilation and optimization with normal mapping effort using a 10 ns clock period
create_clock -name CLK -period 10.0 [get_ports Clk]
compile

# Reporting timing and area after the first synthesis
report_timing > reports/DLX_timeopt_10ns_t.rpt
report_area > reports/DLX_timeopt_10ns_a.rpt

# Compilation and optimization with high mapping effort using a 2 ns clock period
create_clock -name CLK -period 2.0 [get_ports Clk]
compile -map_effort high

# Save reports
report_timing > reports/DLX_timeopt_2ns_t.rpt
report_area > reports/DLX_timeopt_2ns_a.rpt

# Export results
write -hierarchy -format ddc -output outputs/DLX-structural-topt.ddc
write -hierarchy -format vhd1 -output outputs/DLX-structural-topt.vhd1
write -hierarchy -format verilog -output outputs/DLX-structural-topt.v
write_sdc outputs/DLX-structural-topt.sdc

```

Note: The script performs multiple **analyze** operations, each corresponding to a VHDL source file. Only a subset is shown here. The files are analyzed in hierarchical order, from low-level components to higher-level modules, ensuring all dependencies are correctly resolved.

8.1.3 Detailed Explanation

Header and Purpose

- The introductory comments define the script's purpose: to automate and record the DLX synthesis process.
- The *bottom-up* analysis approach is used, meaning low-level modules (e.g., gates, adders, multiplexers) are analyzed before composite blocks. This ordering allows the tool to resolve all dependencies during elaboration.

analyze commands

```

analyze -library WORK -format vhd1 { ./src/000-globals.vhd }
analyze -library WORK -format vhd1 { ./src/a.a-datapath.core/02.a.a-a-ND2_1bit.vhd }
...

```

- The **analyze** command instructs Design Vision to parse and check VHDL files, compiling them into the specified design library (here, **WORK**).
- Each file path points to a distinct VHDL module. Listing them explicitly ensures correct dependency resolution.
- Starting with foundational modules (packages, constants, basic gates) prevents undefined references during later elaboration.

Elaboration step

```
elaborate DLX -architecture STRUCTURAL
```

- The **elaborate** command builds the design hierarchy starting from the specified top-level entity (DLX).
- The **-architecture STRUCTURAL** flag selects the structural architecture, which likely instantiates lower-level datapath and control components.
- This step is crucial to ensure that the correct hierarchy is synthesized, particularly when multiple architectures exist for the same entity.

Wire-load model

```
set_wire_load_model -name 5K_hvration_1_4
```

This command specifies the wire-load model used to estimate interconnect delays and capacitances during synthesis. Accurate wire-load modeling is important for realistic timing estimation and to guide later stages of physical design.

First optimization pass (10 ns)

The first synthesis pass targets a moderate timing constraint (10 ns clock period), which typically results in smaller area and lower power consumption.

```
create_clock -name CLK -period 10.0 [get_ports Clk]
compile
report_timing > reports/DLX_timeopt_10ns_t.rpt
report_area   > reports/DLX_timeopt_10ns_a.rpt
```

- **create_clock** defines the timing constraint on the main clock input (**Clk**) with a 10 ns period (100 MHz).
- **compile** performs synthesis and logic optimization to meet this constraint.
- **report_timing** and **report_area** generate timing and area reports, respectively, stored in the **reports/** directory.

Aggressive optimization pass (2 ns)

The second synthesis pass applies a much tighter constraint (2 ns clock period) to achieve higher performance at the expense of larger area.

```
create_clock -name CLK -period 2.0 [get_ports Clk]
compile -map_effort high
report_timing > reports/DLX_timeopt_2ns_t.rpt
report_area > reports/DLX_timeopt_2ns_a.rpt
```

- The 2 ns clock period corresponds to a 500 MHz operating frequency.
- The `-map_effort high` option instructs the synthesizer to use more aggressive mapping and optimization techniques.
- The resulting timing and area reports document the design's performance under this more stringent constraint.

Exporting synthesized outputs

```
write -hierarchy -format ddc -output outputs/DLX-structural-topt.ddc
write -hierarchy -format vhdl -output outputs/DLX-structural-topt.vhdl
write -hierarchy -format verilog -output outputs/DLX-structural-topt.v
write_sdc outputs/DLX-structural-topt.sdc
```

The `write` commands export the synthesized design in multiple formats:

- `.ddc`: native Design Vision database file for incremental synthesis or later re-import.
- `.vhdl` and `.v`: gate-level netlists for verification or back-end implementation.
- `.sdc`: constraint file containing clock definitions and timing requirements for place-and-route tools.

8.2 Generated files and directories

When executed through `synthesis.sh` (which runs `design_vision -f DLX.t.scr | tee synth.log`), the following files are generated:

- `synth.log`: console log of the synthesis session.
- `reports/DLX_timeopt_10ns_t.rpt`, `reports/DLX_timeopt_10ns_a.rpt`: timing and area reports for the 10 ns run.
- `reports/DLX_timeopt_2ns_t.rpt`, `reports/DLX_timeopt_2ns_a.rpt`: timing and area reports for the 2 ns run.
- `outputs/DLX-structural-topt.ddc`: Design Vision database output.
- `outputs/DLX-structural-topt.vhdl` and `outputs/DLX-structural-topt.v`: gate-level netlists in VHDL and Verilog.
- `outputs/DLX-structural-topt.sdc`: exported timing constraints file.

8.2.1 Running the Synthesis

From the project root directory, synthesis can be launched with:

```
./synthesis.sh
```

This wrapper script:

- Removes any existing `work/` directory and creates a clean workspace.
- Loads the Design Vision environment using `/eda/scripts/init_design_vision`.
- Executes Design Vision in batch mode with `DLX.t.scr`, while logging the output to `synth.log`.

8.3 Summary

The `DLX.t.scr` script defines a complete, automated synthesis flow for the DLX processor. It performs hierarchical analysis of VHDL sources, applies timing constraints, executes synthesis and optimization with varying effort levels, and exports all essential outputs (reports, netlists, and constraint files). This structured and repeatable process ensures reproducibility and provides a solid foundation for subsequent physical design stages.

8.4 Synthesis results

As discussed previously, the results of the synthesis process are collected in the `reports/` directory. In particular, the timing reports `DLX_timeopt_10ns.t.rpt` and `DLX_timeopt_2ns.t.rpt` contain the detailed timing analysis for the two synthesis runs, while the area reports `DLX_timeopt_10ns.a.rpt` and `DLX_timeopt_2ns.a.rpt` summarize the resource utilization of the synthesized design.

From the timing reports, we observe that:

- For the first synthesis run, performed with a clock period constraint of 10 ns, the worst negative slack (WNS) is approximately **+4.08 ns** regarding a critical path with a 5 ns constraint (as it operates between the rising and falling edges of the clock). This indicates that the design comfortably meets the 10 ns timing target and could theoretically operate at a minimum clock period of about 1.72 ns, corresponding to a maximum frequency of roughly **581 MHz**.
- For the second synthesis run, performed with a tighter clock constraint of 2 ns and a higher mapping effort, the worst slack is **0 ns**. This implies that the synthesis process successfully met the 2 ns timing constraint without any violations. The corresponding clock frequency is **500 MHz**. It is worth noting that the maximum achievable frequency is slightly lower than in the previous case, likely due to the heuristics optimizations and trade-offs made to optimize the circuit from many perspectives, not only timing.

Regarding the area analysis:

- For the 10 ns synthesis, the total cell area is approximately **23,207.2 μm^2** .
- For the 2 ns synthesis, the total area is slightly smaller, around **22,508.1 μm^2** .

These results demonstrate that tightening the timing constraint to 2 ns leads to more aggressive optimization, which marginally improves performance while maintaining comparable area efficiency. Overall, the DLX design exhibits good scalability, meeting high-frequency targets with minimal impact on silicon area.

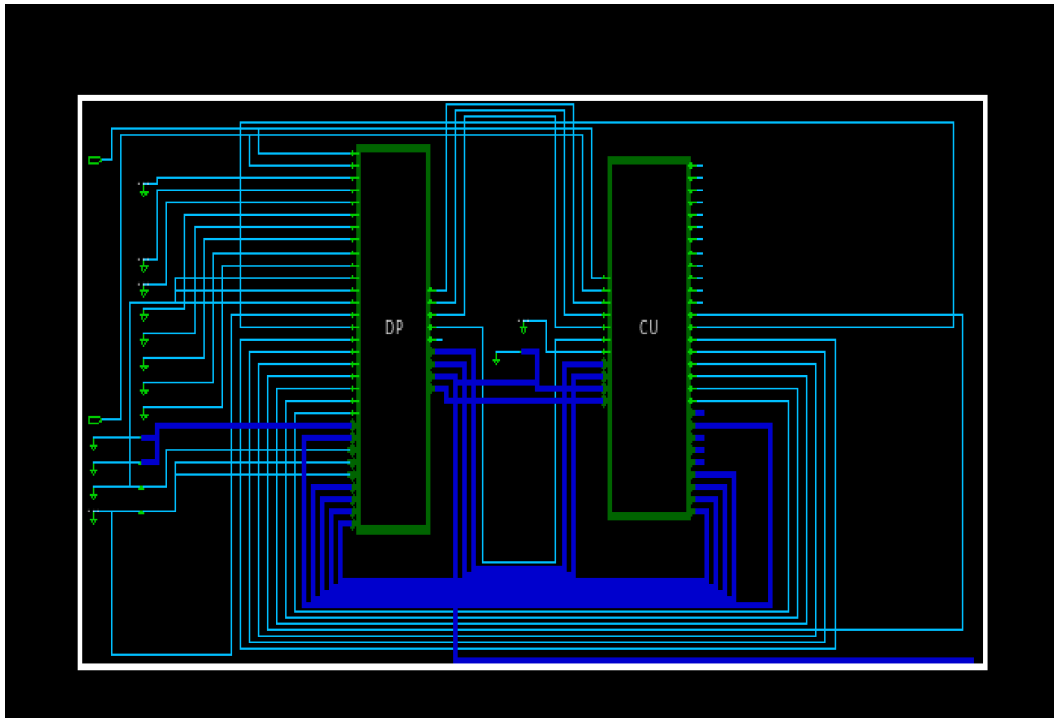


Figure 8.1: Schematic view of the synthesized DLX processor.

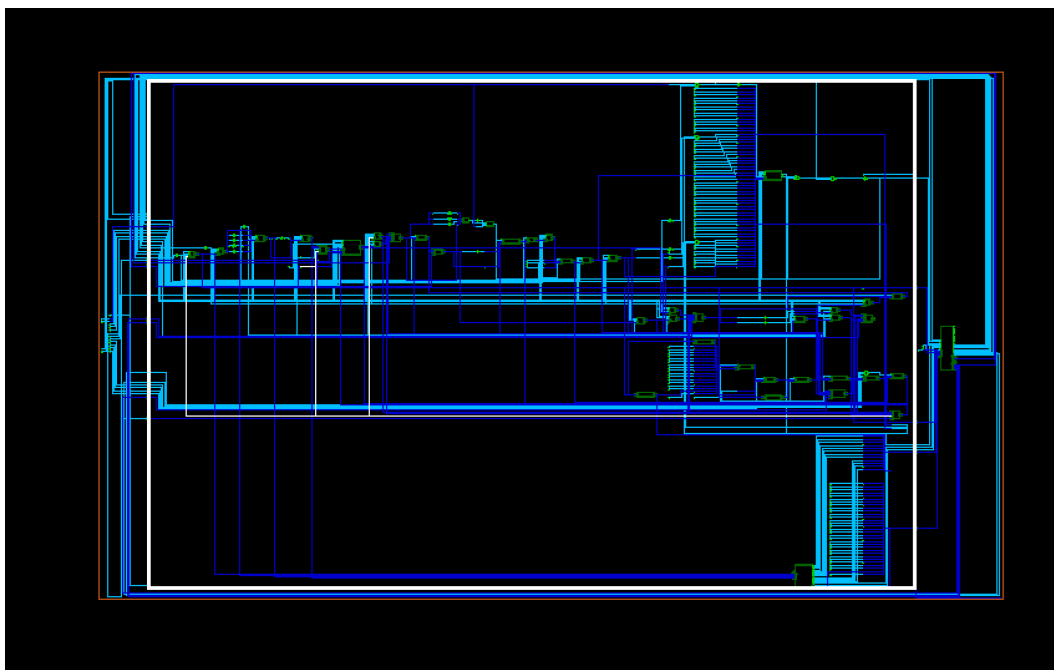


Figure 8.2: Schematic view of the synthesized DLX datapath.

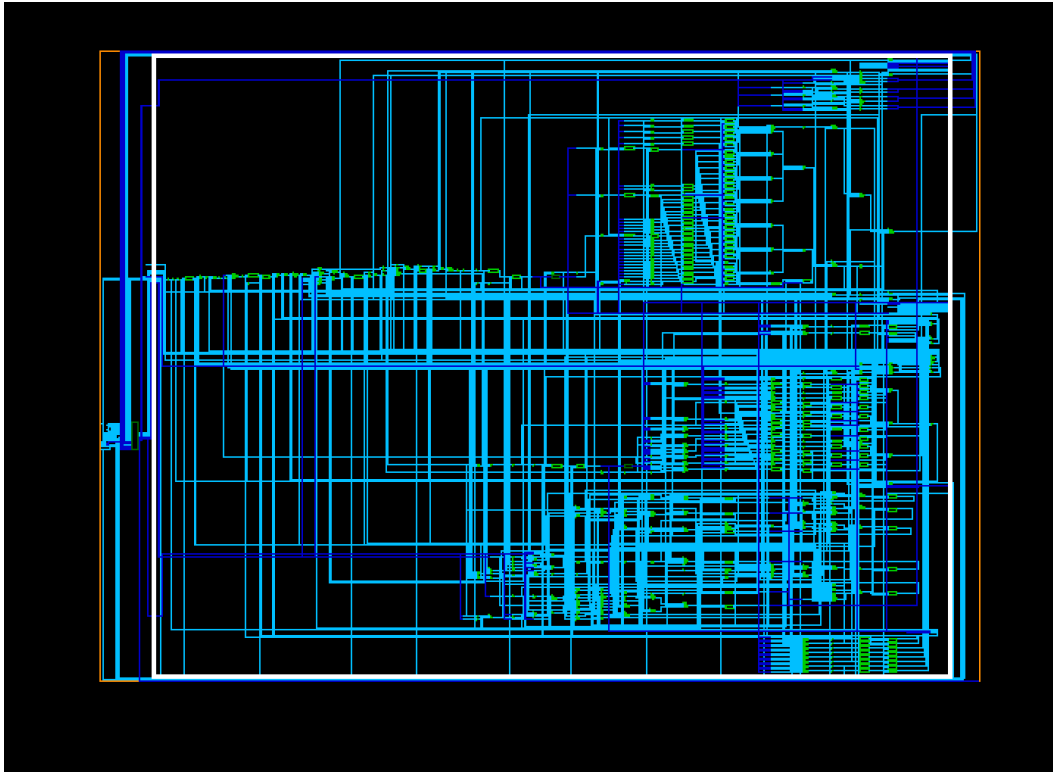


Figure 8.3: Schematic view of the synthesized DLX Control Unit

8.5 Physical Design of the DLX Processor

This section describes the physical design phase of the DLX processor, carried out using the Cadence Innovus implementation system. The activity followed the procedure outlined in Lab 6, with some phases omitted due to tool-related issues. The objective was to translate the synthesized DLX netlist into a complete standard-cell layout, including placement, clock-tree synthesis, and routing, while verifying that all physical and electrical constraints were satisfied.

8.5.1 Design Setup and Configuration

The process began by preparing the Innovus environment and importing the required design files. The environment variables were initialized with the following commands:

```
setinnovus
innovus
```

The design was then loaded through *File* → *Import Design*, selecting the configuration file `DLX.globals`. This file defined the references to:

- The post-synthesis Verilog netlist of the DLX processor.
- The standard-cell layout libraries (`.lef` files).
- The power supply definitions (`vdd` and `gnd`).
- The MMMC (Multi-Mode Multi-Corner) configuration via the `Default.view` file.

After loading, Innovus displayed the basic floorplan outline corresponding to the core area where the standard cells would later be placed.

8.5.2 Floorplanning

The floorplan defines the die dimensions, the placement area for the standard cells, and the space reserved for the power rings. This step was executed via:

`Floorplan → Specify Floorplan`

The aspect ratio was set to 1.0, the utilization factor to 0.6, and the core margins to 5 μm on each side. This configuration ensured sufficient routing space and an even cell distribution for subsequent design stages.

8.5.3 Power Planning

Power distribution was implemented using two metal rings around the core area: one for VDD and one for GND. The rings were created using:

`Power → Power Planning → Add Rings`

with metals M9 (horizontal) and M10 (vertical), both having a width and spacing of 0.8 μm .

To further improve power integrity, vertical stripes were inserted to distribute current evenly throughout the die:

`Power → Power Planning → Add Stripes`

The stripes were placed on M10, spaced by 20 μm , creating a dense and reliable power grid that connected the core to the external rings.

8.5.4 Standard-Cell Placement

In this phase, all synthesized logic cells were positioned within the defined rows of the core area. Placement blockages were first defined on layers M1–M8 to avoid overlap with the power and ground structures. Standard-cell placement was then performed via:

`Place → Place Standard Cell`

with the top routing layer limited to 6.

After placement, the tool reported key parameters such as total wire length and cell density to ensure proper utilization and minimal congestion.

8.5.5 Clock Tree Synthesis

Clock Tree Synthesis (CTS) was carried out to distribute the clock signal efficiently across the design, minimizing skew and insertion delay. The following commands were used to create and configure the clock network:

```
create_ccopt_clock_tree_spec
get_ccopt_clock_trees *
ccoptDesign
```

During CTS, buffer cells were automatically inserted to balance the load across the various pipeline registers of the DLX architecture, ensuring a uniform clock arrival time throughout the design.

8.5.6 Routing

Routing was executed using the NanoRoute engine to interconnect all placed cells according to the logical netlist:

```
Route → NanoRoute → Route
```

All routing layers up to M6 were utilized. Upon completion, Innovus provided detailed reports on the number of wires, vias, and the total routed length. Minor congestion warnings were observed in denser regions, though they did not impact overall connectivity.

8.5.7 Parasitic Extraction and Analysis

The post-routing phase included parasitic extraction to characterize interconnect resistances and capacitances. This step was performed using:

```
setExtractRCMode -engine postRoute -effortLevel low
extractRC
rcOut -spef DLX.spef
```

The generated `DLX.spef` file (Standard Parasitic Exchange Format) contains detailed information about the parasitic RC network for each routed net. This data is essential for accurate post-layout timing simulations and back-annotation.

8.5.8 Timing and Power Reports

Timing and power analyses were performed to evaluate the post-layout performance and to compare it with the synthesis-level expectations:

```
Timing → Report Timing
report_power > power.txt
```

The timing analysis provided a detailed view of the slack of one the critical paths, confirming that the design met the 2 ns clock period constraint with a worst slack of approximately 0.05 ns. The power report successfully summarized both dynamic and static power contributions, providing an overview of the processor's energy consumption after place-and-route. In particular, the report shows a total power consumption of about 12 mW, obtained by the sum of internal power (7 mW), switching power (4.6 mW) and leakage power (0.4 mW).

8.5.9 Final comments

Following the described flow, the physical design of the DLX processor was successfully implemented in Cadence Innovus. Despite omitting certain optimization steps due to tool issues, the layout was completed, routed, and verified without major violations. The results demonstrate the complete digital implementation process, from importing the synthesized netlist to generating the final layout ready for sign-off analysis.

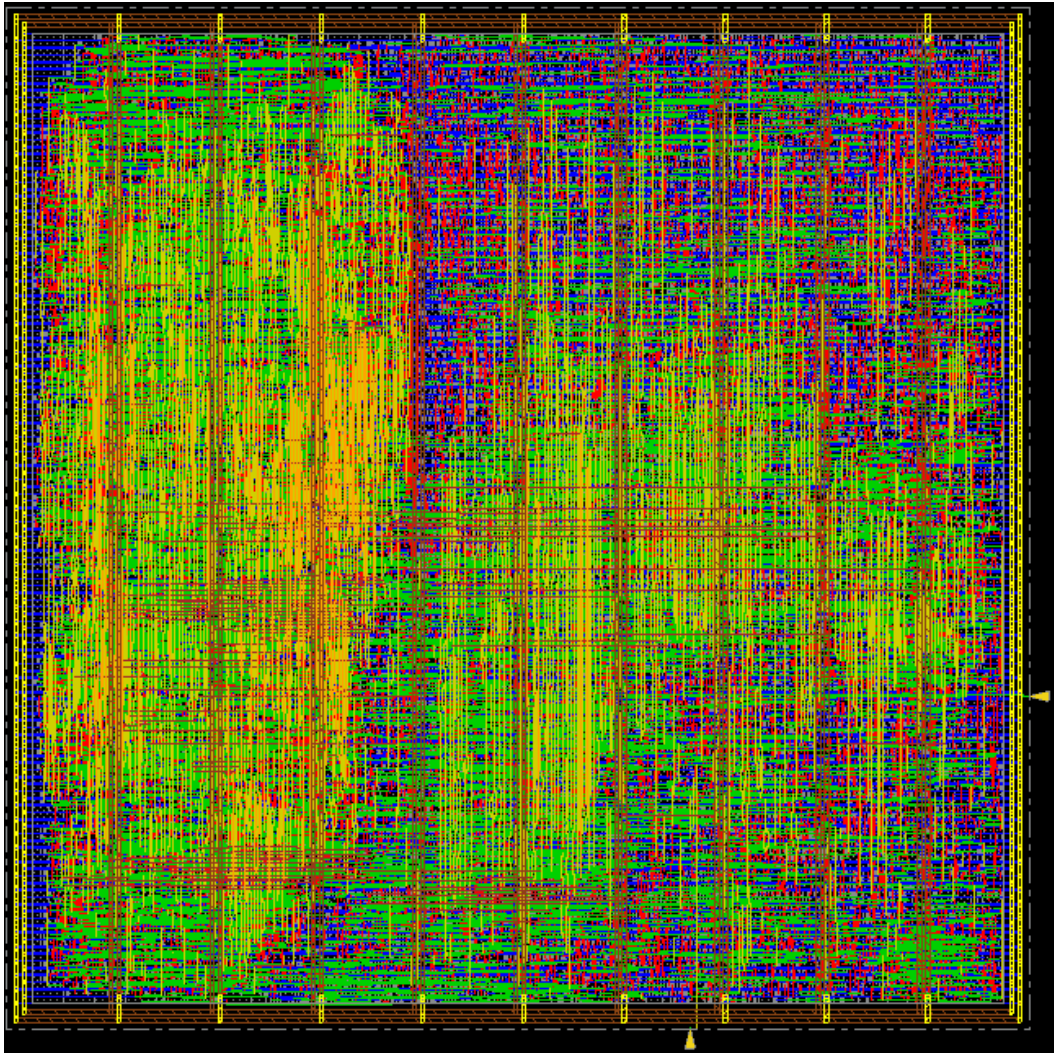


Figure 8.4: Final DLX layout overview.

CHAPTER 9

Conclusions

In this report, we have presented the design, implementation, and validation of a DLX microprocessor featuring a pipelined architecture with hazard management mechanisms, including a data cache and branch prediction capabilities.

The microprocessor was developed using VHDL and simulated using QuestaSim, allowing for thorough testing and verification of its functionality. The design process involved careful consideration of the overall architecture, datapath components, and control logic to ensure efficient operation and performance.

The implementation of hazard management techniques, such as forwarding, stalling, and branch prediction using a Branch History Table (BHT), significantly improved the microprocessor's ability to handle data and control hazards, thereby enhancing its throughput. The data cache was designed to reduce memory access latency, further contributing to the microprocessor's performance.

Validation of the microprocessor was conducted through a series of assembly test programs that covered various scenarios, including arithmetic operations, memory accesses, branching, and complex algorithms like quick sort and Fibonacci sequence calculation. The results obtained from these tests confirmed the correctness and reliability of the microprocessor's design.

Overall, this project has demonstrated the successful development of a pipelined DLX microprocessor with advanced features, showcasing the capabilities of VHDL for hardware design and simulation.

9.1 Lesson Learned

Throughout the development of this DLX microprocessor project, we have gained valuable insights and experience in various aspects of computer architecture and hardware design. One of the key lessons learned is the importance of a well-structured design process, starting from a clear understanding of the overall architecture and breaking it down into manageable components. This approach facilitated the implementation and debugging phases, allowing us to focus on individual modules before integrating them into the complete system.

Additionally, we have learned the significance and difficulty of hazard management in pipelined architectures. Implementing techniques such as forwarding, stalling, and branch prediction required careful analysis of data and control dependencies, as well as a deep understanding of the timing and interactions between different pipeline stages.

This experience has highlighted the challenges associated with ensuring correct operation while maximizing performance in a pipelined processor. Furthermore, the project has reinforced the importance of thorough testing and validation. Developing a comprehensive set of assembly test programs allowed us to systematically verify the functionality of the microprocessor and identify potential issues.

early in the design process. This iterative testing approach was crucial for achieving a reliable final product.

Finally, working with VHDL and simulation tools like QuestaSim has enhanced our skills in hardware description languages and digital design methodologies. We have learned how to effectively model complex systems, simulate their behavior, and analyze waveforms to diagnose and resolve issues.

Overall, this project has provided us with a solid foundation in computer architecture, hardware design, and VHDL programming, which will be invaluable for future endeavors in the field.

9.2 Future Work

Future work could involve additional features like the support of multicycle operations (e.g., multiplication and division), multi-threading with a windowed register file and interrupt handling.

9.3 Acknowledgements

We would like to thank Prof. Graziano, as well as the course's teaching assistants Giovanna Turvani, Fabrizio Riente and Deborah Volpe, for the material provided for the realization of this project, namely the lecture notes and laboratories of the Microelectronic Systems course, and for the opportunity given to us to work on this challenging and rewarding project.

Authors: Gabriele Arcidiacono, Stefano Galati, Emanuele Aquilia

APPENDIX A

Data Hazard Instances

In this appendix, we list all the data forwarding and stall cases that have been implemented in the designed pipelined processor to solve data hazards.

A.1 All cases of Data Forwarding

The following is a table listing all the cases that have been implemented in the designed pipelined processor to solve data hazards via data forwarding. Each record is identified by a unique code (#) and describes the case by specifying the source and destination pipeline registers and operations, the forwarding destination, and the condition on the registers involved. Recall that, for I-Type instructions, the source register specifier is RS1 and the destination register specifier is RS2.

#	SOURCE Pipeline Reg	SOURCE Op	DEST Pipeline Reg	DEST Op	Forwarded To	Condition
F1	EX/MEM	R-Type	ID/EX	R-Type, I-Type ALU, Load, Store, Branch	Top ALU input	EX/MEM.IR[RD] = ID/EX.IR[RS1]
F2	EX/MEM	R-Type	ID/EX	R-Type	Bottom ALU input	EX/MEM.IR[RD] = ID/EX.IR[RS2]
F3	MEM/WB	R-Type	ID/EX	R-Type, I-Type ALU, Load, Store, Branch	Top ALU input	MEM/WB.IR[RD] = ID/EX.IR[RS1]
F4	MEM/WB	R-Type	ID/EX	R-Type	Bottom ALU input	MEM/WB.IR[RD] = ID/EX.IR[RS2]
F5	EX/MEM	I-Type ALU	ID/EX	R-Type, I-Type ALU, Load, Store, Branch	Top ALU input	EX/MEM.IR[RS2] = ID/EX.IR[RS1]
F6	EX/MEM	I-Type ALU	ID/EX	R-Type	Bottom ALU input	EX/MEM.IR[RS2] = ID/EX.IR[RS2]
F7	MEM/WB	I-Type ALU	ID/EX	R-Type, I-Type ALU, Load, Store, Branch	Top ALU input	MEM/WB.IR[RS2] = ID/EX.IR[RS1]
F8	MEM/WB	I-Type ALU	ID/EX	R-Type	Bottom ALU input	MEM/WB.IR[RS2] = ID/EX.IR[RS2]
F9	MEM/WB	Load	ID/EX	R-Type, I-Type ALU, Load, Store, Branch	Top ALU input	MEM/WB.IR[RS2] = ID/EX.IR[RS1]
F10	MEM/WB	Load	ID/EX	R-Type	Bottom ALU input	MEM/WB.IR[RS2] = ID/EX.IR[RS2]
F11	MEM/WB	R-Type	EX/MEM	Store	Data Mem input	MEM/WB.IR[RD] = EX/MEM.IR[RS2]
F12	MEM/WB	I-Type ALU	EX/MEM	Store	Data Mem input	MEM/WB.IR[RS2] = EX/MEM.IR[RS2]
F13	MEM/WB	Load	EX/MEM	Store	Data Mem input	MEM/WB.IR[RS2] = EX/MEM.IR[RS2]
F14	MEM/WB	Jump and LR	EX/MEM	Store	Data Mem input	EX/MEM.IR[RS2] = LR
F15	MEM/WB	R-Type	ID/EX	Store	Bottom ALU input	MEM/WB.IR[RD] = ID/EX.IR[RS2]
F16	MEM/WB	I-Type ALU	ID/EX	Store	Bottom ALU input	MEM/WB.IR[RS2] = ID/EX.IR[RS2]
F17	MEM/WB	Load	ID/EX	Store	Bottom ALU input	MEM/WB.IR[RS2] = ID/EX.IR[RS2]
F18	ID/EX	Jump and LR	IF/ID	Jump Reg	Jump address input	IF/ID.IR[RS1] = LR
F19	EX/MEM	Jump and LR	IF/ID	Jump Reg	Jump address input	IF/ID.IR[RS1] = LR
F20	EX/MEM	R-Type	IF/ID	Jump Reg	Jump address input	EX/MEM.IR[RD] = IF/ID.IR[RS1]
F21	EX/MEM	I-Type ALU	IF/ID	Jump Reg	Jump address input	EX/MEM.IR[RS2] = IF/ID.IR[RS1]

Table A.1: All implemented Forwarding Cases

Now each forwarding case will be visually represented, identified with its code, in the following diagrams, with examples of the instructions involved in the hazard.

F1 - F5

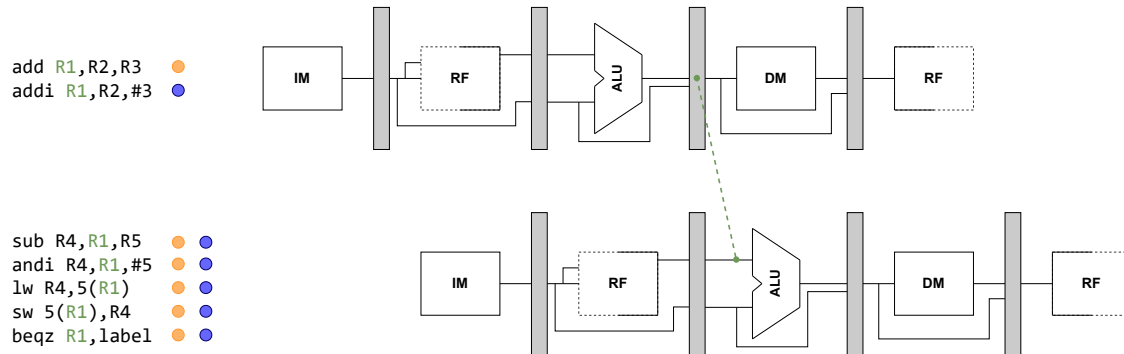


Figure A.1: Example of forwarding cases F1 and F5: forwarding from EX/MEM to ALU Input 1

F2 - F6

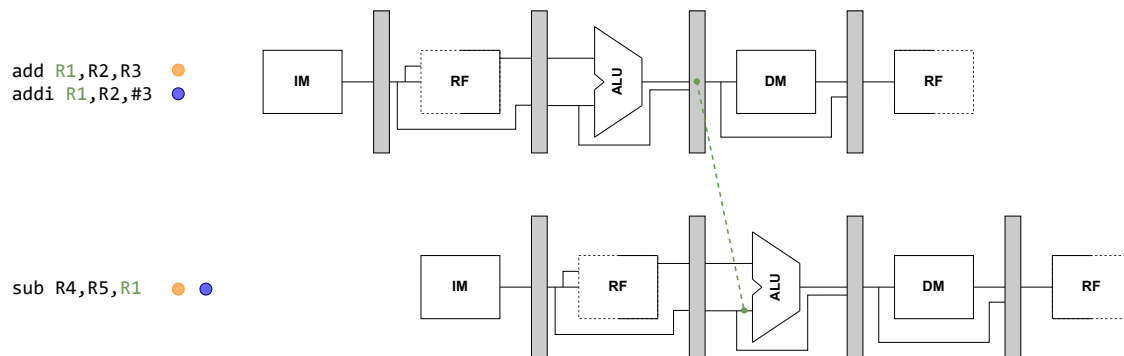


Figure A.2: Example of forwarding cases F2 and F6: forwarding from EX/MEM to ALU Input 2

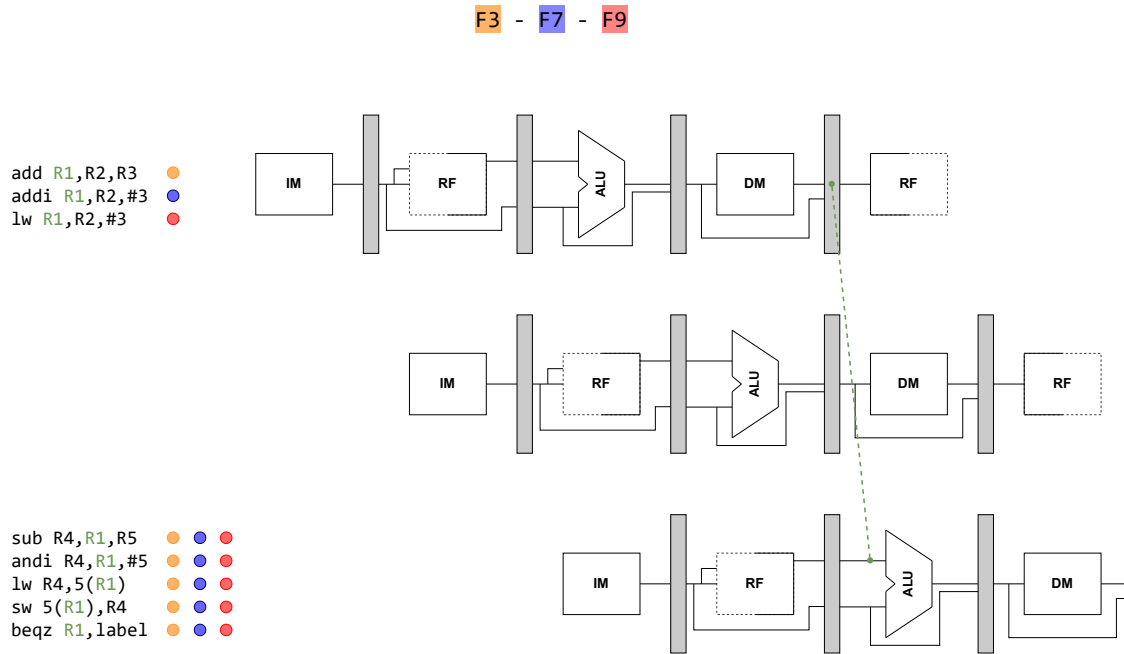


Figure A.3: Example of forwarding cases F3, F7 and F9: forwarding from MEM/WB to ALU Input 1

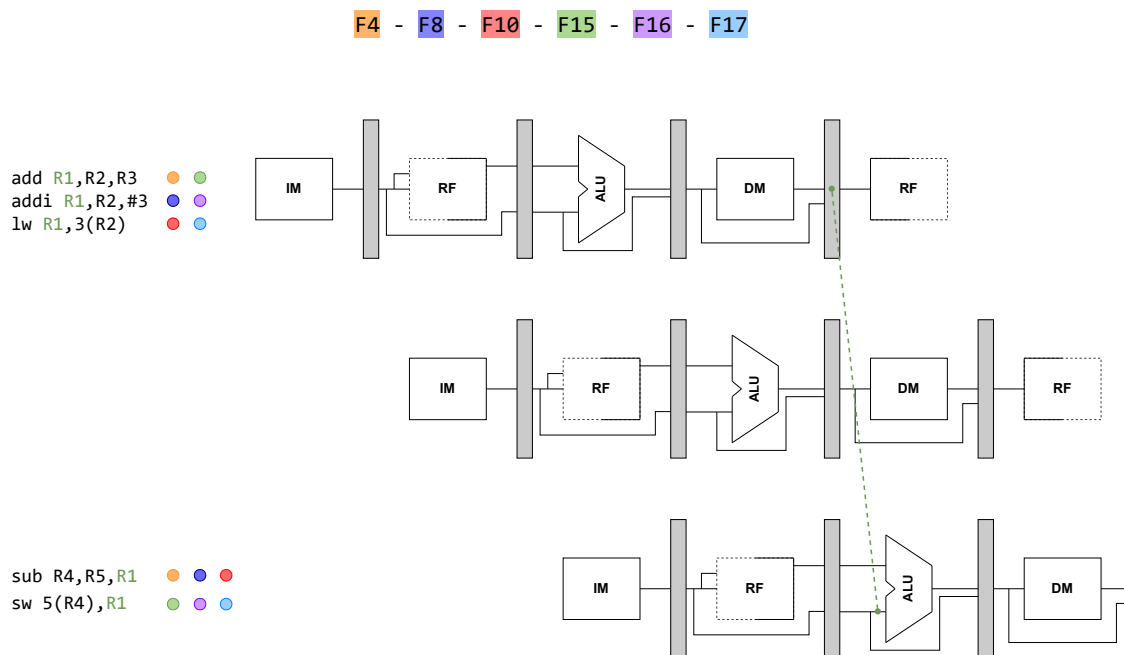


Figure A.4: Example of forwarding cases F4, F8, F10, F15, F16 and F17: forwarding from MEM/WB to ALU Input 2

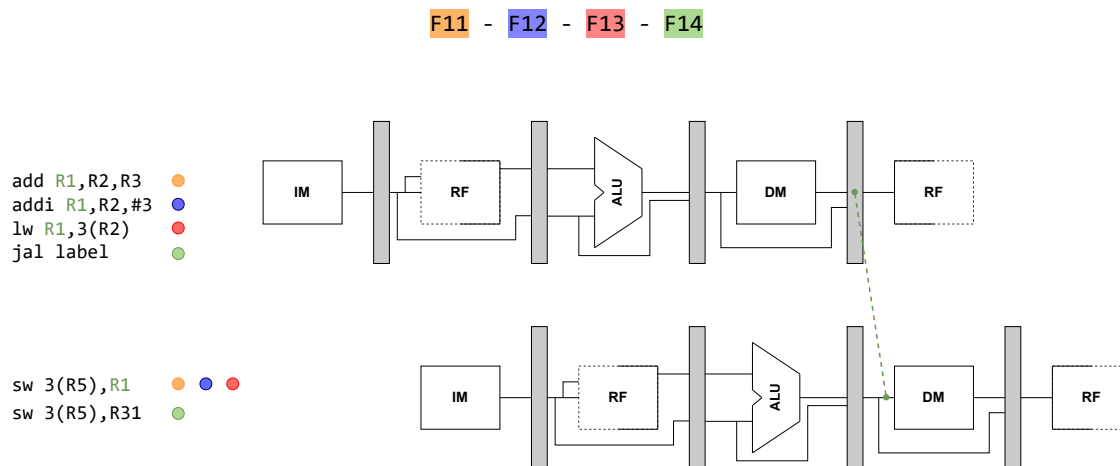


Figure A.5: Example of forwarding cases F11, F12, F13 and F14: forwarding from MEM/WB to Data Memory input

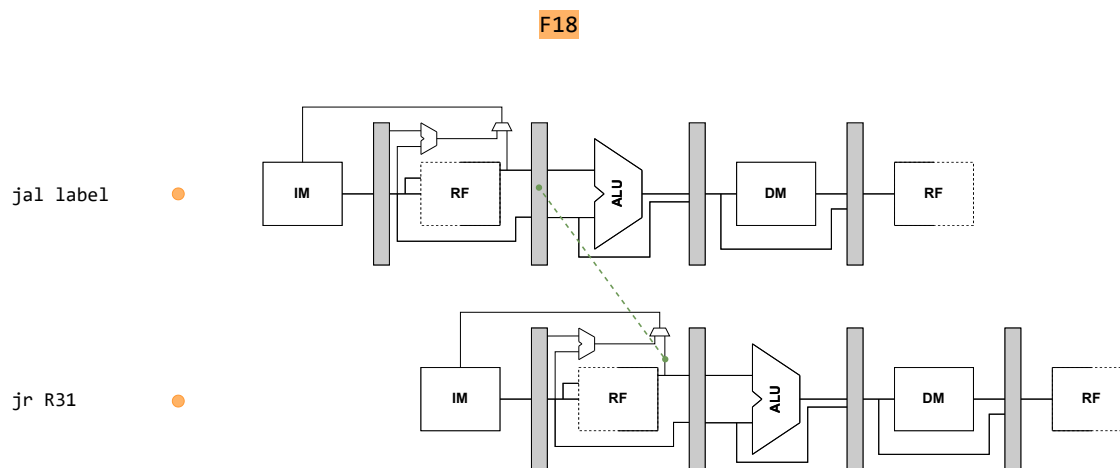


Figure A.6: Example of forwarding cases F18: forwarding from ID/EX to Jump Register address input

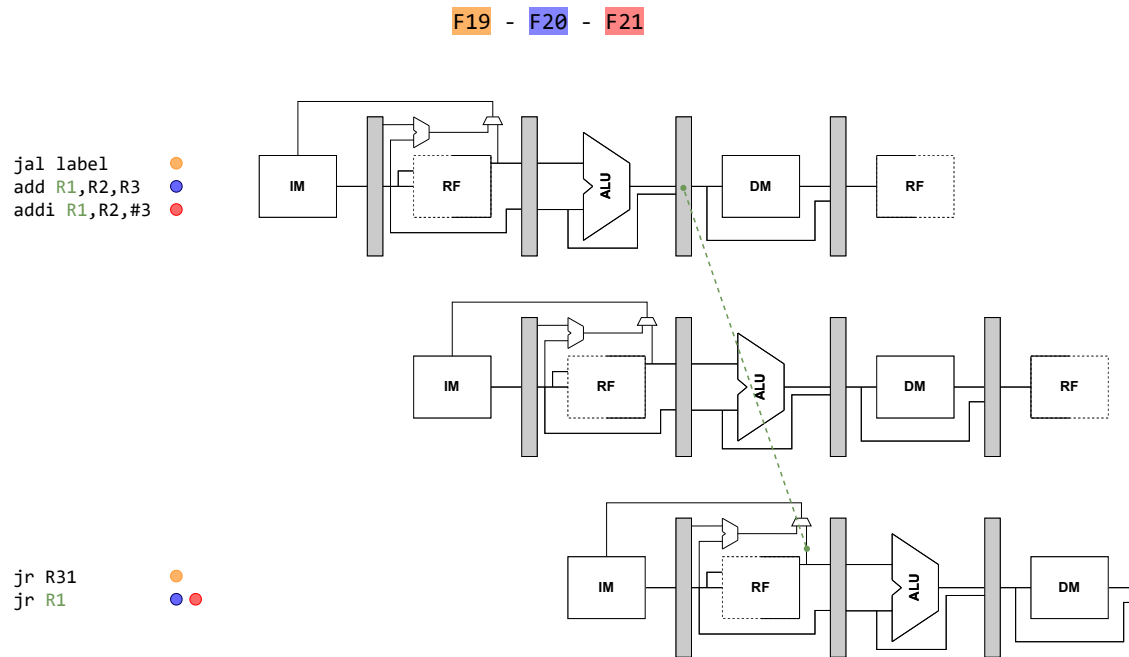


Figure A.7: Example of forwarding cases F19, F20 and F21: forwarding from EX/MEM to Jump Register address input

A.2 All cases of Stalling

The following is a table listing all the cases that have been implemented in the designed pipelined processor to solve data hazards via stalling. Each record is identified by a unique code (#) and describes the case by specifying the source and destination pipeline registers and operations, the condition on the registers involved, the pipeline register up to which the stall is applied, and the number of stall cycles introduced.

#	SOURCE Pipeline Reg	SOURCE Op	DEST Pipeline Reg	DEST Op	Condition	Stall up to	Stall cycles
S1	EX/MEM	Load	ID/EX	R-Type, I-Type ALU, Load, Store, Branch	$EX/MEM.IR[RS2] = ID/EX.IR[RS1]$	EX/MEM	1
S2	EX/MEM	Load	ID/EX	R-Type	$EX/MEM.IR[RS2] = ID/EX.IR[RS2]$	EX/MEM	1
S3	ID/EX	R-Type	IF/ID	Jump Reg	$ID/EX.IR[RS2] = IF/ID.IR[RS1]$	ID/EX	1
S4	ID/EX	I-Type ALU	IF/ID	Jump Reg	$ID/EX.IR[RS2] = IF/ID.IR[RS1]$	ID/EX	1
S5	ID/EX	Load	IF/ID	Jump Reg	$ID/EX.IR[RS2] = IF/ID.IR[RS1]$	ID/EX	2
S6	EX/MEM	Load	IF/ID	Jump Reg	$EX/MEM.IR[RS2] = IF/ID.IR[RS1]$	ID/EX	1
S7	EX/MEM	Jump and LR	IF/ID	Store	$IF/ID.IR[RS2] = LR$	ID/EX	1
S8	ID/EX	Jump and LR	IF/ID	R-Type, I-Type ALU	$IF/ID.IR[RS1] = LR$	ID/EX	2
S9	ID/EX	Jump and LR	IF/ID	R-Type	$IF/ID.IR[RS2] = LR$	ID/EX	2
S10	EX/MEM	Jump and LR	IF/ID	R-Type, I-Type ALU	$IF/ID.IR[RS1] = LR$	ID/EX	1
S11	EX/MEM	Jump and LR	IF/ID	R-Type	$IF/ID.IR[RS2] = LR$	ID/EX	1

Table A.2: All implemented Stall Cases

Now each stall case will be visually represented, identified with its code, in the following diagrams, with examples of the instructions involved in the hazard. It can be observed that some cases are implemented to cooperate with forwarding to completely resolve the hazard.

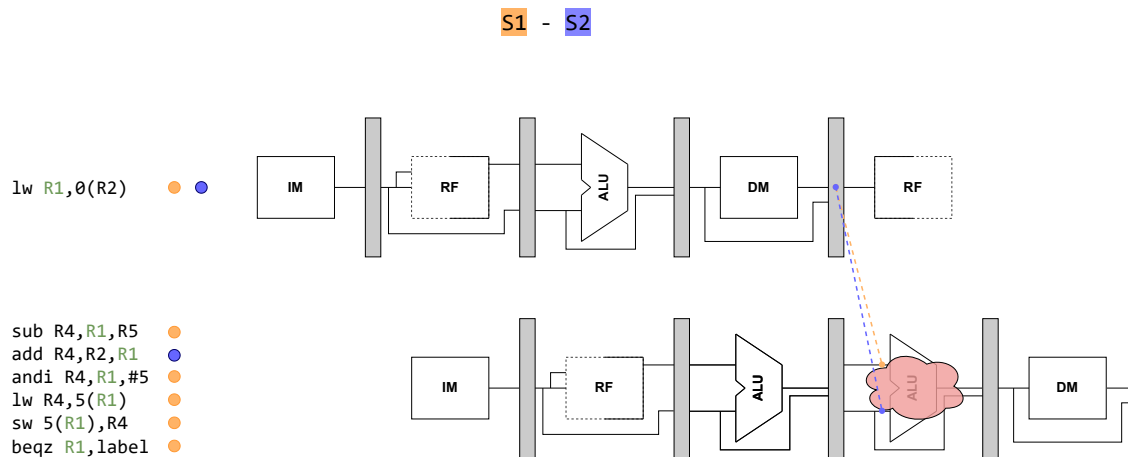


Figure A.8: Example of stall cases S1 and S2: stalling EX for 1 cc and then forward from MEM/WB to ALU Inputs

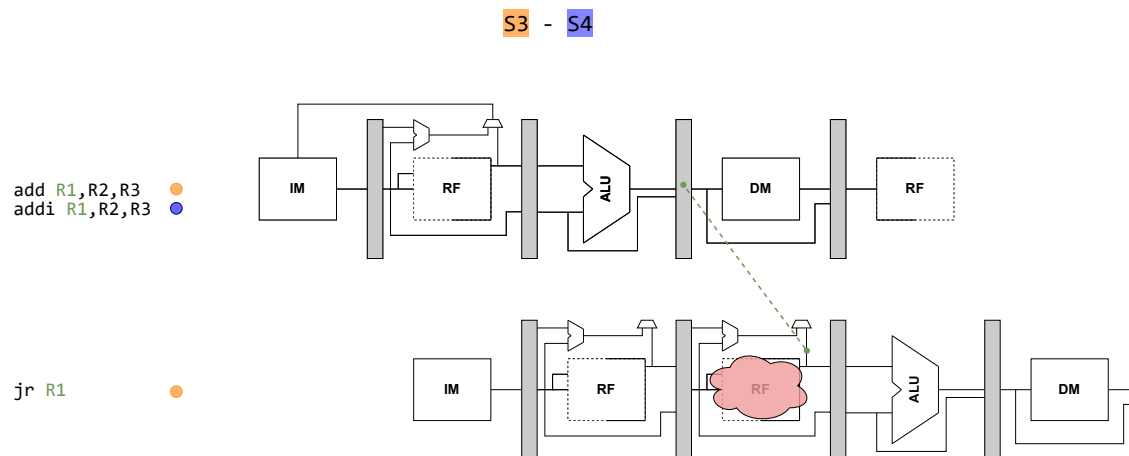


Figure A.9: Example of stall cases S3 and S4: stalling ID for 1 cc and then forward from EX/MEM to Jump Register address input

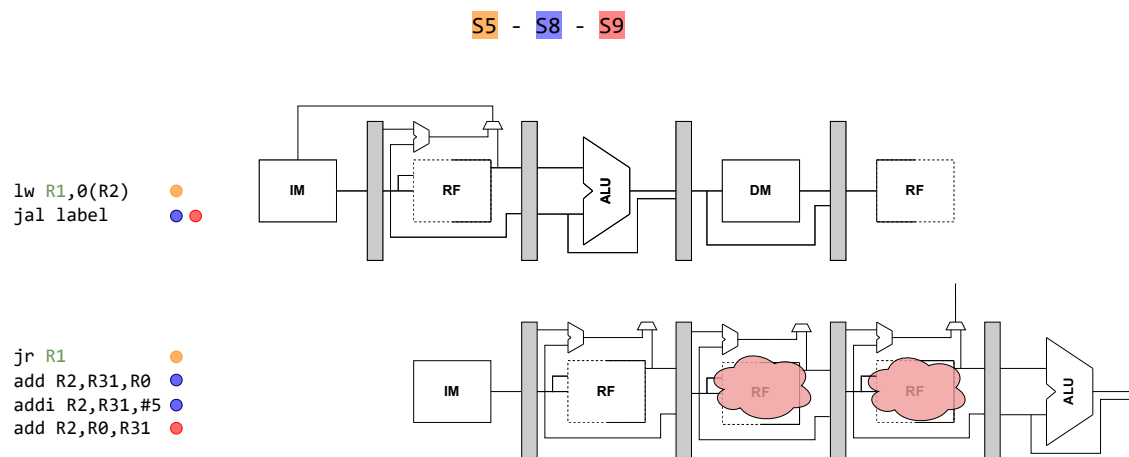


Figure A.10: Example of stall cases S5, S8, and S9: stalling ID for 2 cc

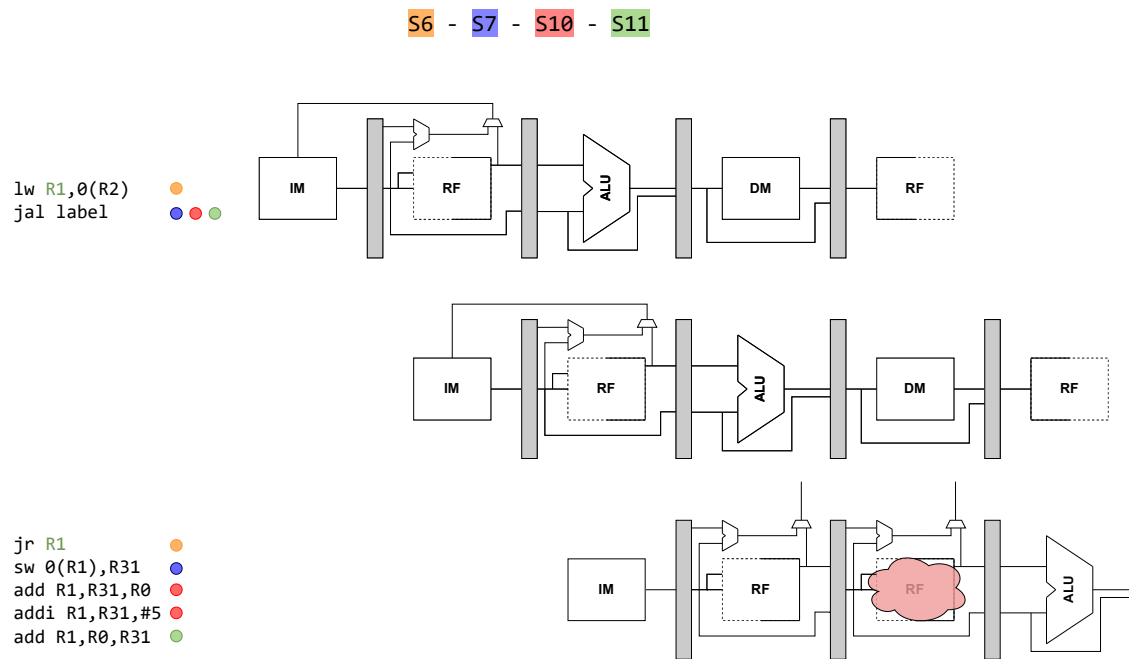


Figure A.11: Example of stall cases S6, S7, S10, and S11: stalling ID for 1 cc

APPENDIX B

Instructions encoding

In this appendix, we list all encodings of the DLX instructions implemented in our microprocessor design, as well as a brief description of the value of every control bit.

OPCODE	Instruction	Description
0	RTYPE	- UIS = 0: 16-bit immediate not used (don't care) - SUS = 0: 16-bit immediate not used (don't care) - SES = 0: No immediate used (don't care) - RF1 = 1: read RS1 - RF2 = 1: read RS2 - JUMPS = 0: don't care - BS = 0: ALU input B = out2 - AS = 0: ALU input A = out1 - CNDS = 00: branch logic disabled - DMS = 0, DMB = 00: don't care - DM_EN = 0: memory disabled - WBS = 01: write back ALU_OUT - WRF = 1: register file write enabled - RDS = 01: destination = IR[rd]
2	JTYPE_J	- SES = 1: 26→32 sign extender - RF1 = 0, RF2 = 0: no register reads - AS = 1, BS = 1: ALU = NPC + IMM - CNDS = 00: branch disabled - DM_EN = 0, WRF = 0: memory and write-back disabled
3	JTYPE_JAL	Like JTYPE_J, but WBS = 10 (write back PC+4), WRF = 1, RDS = 10 (R31).
4	ITYPE_BEQZ	- SES = 0: 16→32 sign extender - RF1 = 1, RF2 = 0 - AS = 1, BS = 1: ALU = NPC + IMM - CNDS = 01: equal-to-zero branch - WRF = 0, DM_EN = 0: no write-back, no memory
5	ITYPE_BNEZ	Same as BEQZ but CNDS = 11 (not-equal-zero branch).

Table B.1: Control signals for OPCODEs 0 to 5 (of the implemented instructions) explained

OPCODE	Instruction	Description
8, 10, 24–29	ITYPE ALU (signed)	ADDI, SUBI, SEI, SNEI, SLTI, SGTI, SLEI, SGEI - SUS = 0: signed immediate - AS = 0, BS = 1: ALU = RS1 op IMM - WBS = 01, WRF = 1, RDS = 00.
9, 11–14, 20, 22, 23, 58–61	ITYPE ALU (unsigned)	ADDUI, SUBUI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, SLTUI, SGTUI, SLEUI, SGEUI - SUS = 1: unsigned immediate - AS = 0, BS = 1: ALU = RS1 op IMM - WBS = 01, WRF = 1, RDS = 00.
15	ITYPE_LHI	- UIS = 1: immediate shifted left 16 bits - AS = 0, BS = 1 - WRF = 1, WBS = 01, RDS = 00.
18	ITYPE_JR	- RF1 = 1, WRF = 0 - AS = 0, BS = 1 - JUMPS = 0: register jump - No memory or write-back.
19	ITYPE_JALR	Like JR, but WBS = 10 (write back PC+4), WRF = 1, RDS = 10 (R31).
21	NOP	All signals inactive or don't care; no operation performed.
32–37	ITYPE Loads	Load instructions: LB, LH, LW, LBU, LHU - RF1 = 1, AS = 0, BS = 1 - DM_EN = 1, RnW = 1: read from memory - DMB = 00/01/10: byte/half/word - WBS = 00, WRF = 1, RDS = 00.
40–43	ITYPE Stores	Store instructions: SB, SH, SW - RF1 = 1, RF2 = 1 - DM_EN = 1, RnW = 0: write to memory - DMB = 00/01/10: byte/half/word - WRF = 0, WBS = 00.

Table B.2: Control Signals for OPCODEs 8 to 61 (of the implemented instructions) explained

OPCODE	Instr	UIS	SUS	SES	RF1	RF2	JUMPS	BS	AS	CNDS	DMS	DMB	RnW	DM_EN	WBS	WRF	RDS
0	R-TYPE	0	0	0	1	1	0	0	0	00	0	00	1	0	01	1	01
2	JTYPE_J	0	0	1	0	0	0	1	1	00	0	00	1	0	00	0	00
3	JTYPE_JAL	0	0	1	0	0	0	1	1	00	0	00	1	0	10	1	10
4	ITYPE_BEQZ	0	0	0	1	0	0	1	1	01	0	00	1	0	00	0	00
5	ITYPE_BNEZ	0	0	0	1	0	0	1	1	11	0	00	1	0	00	0	00
8	ITYPE_ADDI	0	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
9	ITYPE_ADDUI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
10	ITYPE_SUBI	0	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
11	ITYPE_SUBUI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
12	ITYPE_ANDI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
13	ITYPE_ORI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
14	ITYPE_XORI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
15	ITYPE_LHI	1	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
18	ITYPE_JR	0	0	0	1	0	1	1	0	00	0	00	1	0	00	0	00
19	ITYPE_JALR	0	0	0	1	0	1	1	0	00	0	00	1	0	10	1	10
20	ITYPE_SLLI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
21	NOP	0	0	0	0	0	0	0	0	00	0	00	0	0	00	0	00
22	ITYPE_SRLI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
23	ITYPE_SRAI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
24	ITYPE_SEQI	0	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
25	ITYPE_SNEI	0	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
26	ITYPE_SLTI	0	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
27	ITYPE_SGTI	0	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
28	ITYPE_SLEI	0	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
29	ITYPE_SGEI	0	0	0	1	0	0	1	0	00	0	00	1	0	01	1	00
32	ITYPE_LB	0	0	0	1	0	0	1	0	00	1	00	1	1	00	1	00
33	ITYPE_LH	0	0	0	1	0	0	1	0	00	1	01	1	1	00	1	00
35	ITYPE_LW	0	0	0	1	0	0	1	0	00	1	10	1	1	00	1	00
36	ITYPE_LBU	0	0	0	1	0	0	1	0	00	0	00	1	1	00	1	00
37	ITYPE_LHU	0	0	0	1	0	0	1	0	00	0	01	1	1	00	1	00
40	ITYPE_SB	0	0	0	1	1	0	1	0	00	0	00	0	1	00	0	00
41	ITYPE_SH	0	0	0	1	1	0	1	0	00	0	01	0	1	00	0	00
43	ITYPE_SW	0	0	0	1	1	0	1	0	00	0	10	0	1	00	0	00
58	ITYPE_SLTUI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
59	ITYPE_SGTUI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
60	ITYPE_SLEUI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00
61	ITYPE_SGEUI	0	1	0	1	0	0	1	0	00	0	00	1	0	01	1	00

Table B.3: Control Signals Table